RESEARCH REVIEW / LIVING EDITION

Scaling Law

A Problem-Driven Review

Resource allocation, data, inference, and post-training

Reconstructing the research conversation through problems, mechanisms, evidence, and remaining limits.

Edition date 2026-11-02

English edition | Primary research, technical reports, and community discussion

Primary-source links and navigation bookmarks are clickable. Display equations are typeset; unsupported notation is explicitly identified. Inline mathematics uses readable text notation. The maintained Markdown is the authoritative text.

Problem → Insight → Method → Evidence → Limitations → Subsequent branches

Contents

Page numbers match the PDF. Select a title or use the sidebar bookmarks to navigate.

Introduction: What Questions Do Scaling Laws Answer? 4

T1 Predictability, Budgets, and Training Recipes: What Is the Same Curve Actually Comparing? 6

T2 Data Bottlenecks: From Token Counts to Useful Information, Mixtures, and Training Order 12

T3 Architecture and Deployment: Distinguishing Parameters, FLOPs, GPU Memory, and Actual Costs 20

T4 Post-Training and Reinforcement Learning: From Feedback Objectives to Scalable Reasoning Policies 29

T5 Inference Budgets and Search: Turning More Compute into More Reliable Answers . . . 39

Chapter 6 - Theory and Capability Evaluation: Why Do Power Laws Arise, and What Can They Predict? 50

Chapter 7 - Vision, Multimodality, Diffusion, and Robotics: What Should Be Scaled to Improve the Target Capability? 59

Community Radar: Which Debates Should Change the Scaling-Law Survey? 69

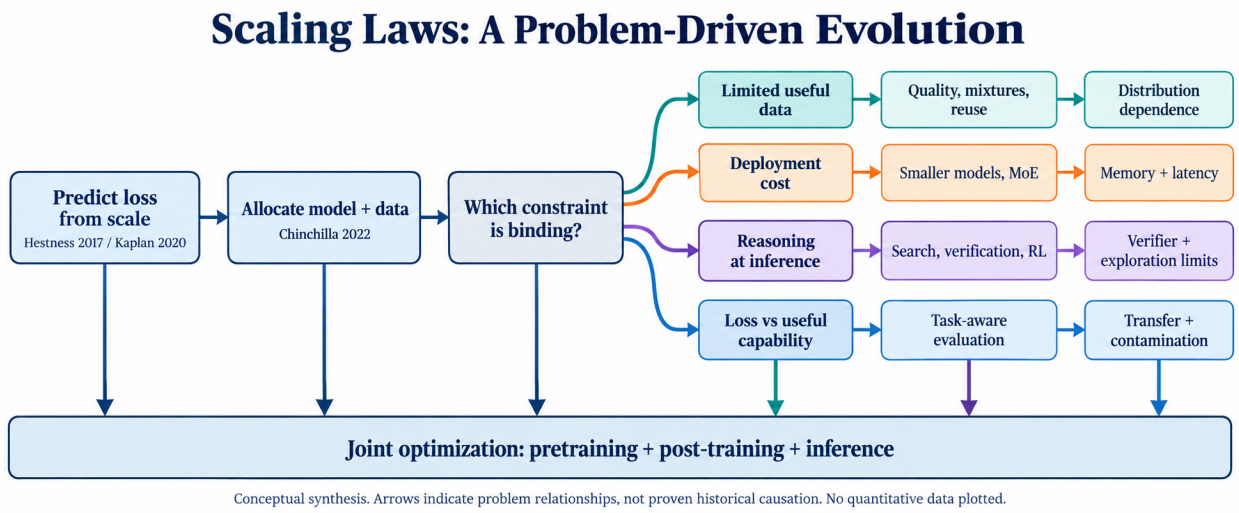
Synthesis: How Has the Research Focus Shifted, and What Evidence Matters Next? 76

Variables, Metrics, and Cost Conventions 79

The Reference Survey's Method and the Gaps in Our First Edition: A Full-Text Reading Audit 81

References 84

A Map of the Research Questions



A reading guide. The text distinguishes direct responses, shared bottlenecks, and parallel approaches; arrows do not automatically establish historical causation.

Edition v0.3.7 · Searches and verification through 2026-11-02 · English living edition

From predicting returns to scale to jointly allocating pretraining, post-training, and inference budgets. This edition contains 192 distinct primary-source records and a separate community radar. It is a representative narrative review, not an exhaustive systematic review; the cited experiments have not been independently reproduced.

The figure is a conceptual synthesis, not an experimental curve; arrows do not automatically establish historical causation.

Reading links: English homepage · Chinese edition · References · Translation status

Introduction: What Questions Do Scaling Laws Answer?

Training a large model begins with a resource decision: before spending the budget, can we estimate the benefits of adding model capacity, training data, or computation? If every configuration must be trained to completion before its performance is known, larger experiments become increasingly difficult to afford. Hestness and colleagues measured learning curves across tasks; Kaplan and colleagues subsequently connected language-model parameters, data, and training computation, turning the intuition that larger models might perform better into resource-performance relationships that could be fitted and compared. Hestness et al., 2017; Kaplan et al., 2020.

Predicting improvement, however, does not by itself identify the right allocation. Chinchilla showed that, at fixed training compute, allocating too much of the budget to parameters without a corresponding increase in training tokens can be inefficient. The central question shifted from whether to scale to which dimension to scale. Data scarcity and quality differences, sparse architectures, deployment costs, reward feedback, and inference-time computation subsequently added further constraints to this optimization problem. Hoffmann et al., 2022.

The central argument of this survey is that **scaling-law research evolves as researchers discover what existing resource models omit and solve the allocation problem again.** This is a synthesis of the literature, not a unified theorem proposed by any one paper.

Separate Three Questions That Are Often Conflated

Level	Object of study	What it can answer	What does not follow directly
Empirical regularities	How loss or error changes with scale under a specified configuration	Behavior in the measured range and the validity of controlled extrapolation	Validity at arbitrary scales, distributions, or tasks
Resource optimality	Choosing models, data, and algorithms under a budget	Which configuration is more efficient for the stated objective	That training optimality also implies deployment optimality
Capabilities and systems	Actual success under a task, interface, and evaluation protocol	Whether a system delivers reliably and at what cost	That one average score completely describes its capabilities

This distinction runs throughout the survey. Lower language-model cross-entropy, higher mathematical pass@1, finding a correct candidate among N samples, and delivering a correct answer to a user are different quantities. Even when all improve with computation, they need not share an exponent or conversion rule. Inference-time scaling in particular requires explicit accounting for selection and verification costs. Snell et al., 2024.

The Same Curve Can Address Four Different Decisions

To understand the subsequent branches, first identify where the budget is fixed. The first decision occurs before training: given total training compute, what model size, token count, and optimization schedule should we choose? The second arises once data and hardware constraints are known: when unique data are scarce, the model does not fit in memory, or context is too long, which variables remain available to increase? The third occurs before deployment: does a sufficiently large request volume justify longer training once in exchange for cheaper inference on every request? The fourth occurs when a request arrives: should this problem receive more generation, search, or verification, or should computation stop? The first three primarily change the system; the fourth allocates a per-problem budget within a trained system. Chinchilla; data-constrained scaling; lifecycle costs; Snell et al..

The following accounting framework connects these decisions; it is not presented as an already fitted, unified scaling law:

$$C_{\text{total}} = C_{\text{pre}} + C_{\text{post}} + Q\,\mathbb{E}_x[C_{\text{serve}}(x)].$$

Here, C_(serve) includes generation, selection or verification, and tool use; Q is the expected request volume. Optimizing monetary cost, latency, or energy requires separate accounting rather than inserting those quantities into the same FLOP sum. The request distribution also matters: at the same average compute, assigning equal budgets to easy and difficult problems may underperform allocation by marginal benefit. Predicting difficulty and deciding when to stop also incur costs. The task-adaptive methods discussed later approximate this incompletely known optimization problem.

This explains a recurring misinterpretation. An experiment showing continued gains from increasing D at fixed N does not determine whether a larger N is preferable at fixed C. Likewise, a model becoming more accurate with longer thinking does not establish that it is more efficient than another model at equal end-to-end cost. Apparently conflicting papers can be placed in the same research conversation only after identifying what was fixed, what changed, and what was measured.

Definitions used throughout the seven chapters appear in the glossary and cost conventions. In particular, data quality is not a universal scalar across tasks; sparse models require more than total parameter counts; verifier-assisted success cannot be attributed solely to the generator; and exact-match scores are not direct continuous measurements of internal capability.

A Map of the Research Questions

This Mermaid diagram is available in the Markdown edition; it is not rendered here.

The arrows represent the survey's explanatory connections. The text distinguishes direct responses to prior work, parallel approaches to the same bottleneck, and editorial synthesis; chronological ordering is not automatically treated as historical causation.

A more detailed account of paper relationships and branching rationales is available in the research conversation map (Chinese).

Reading Paths

For the classical allocation debate, start with the Kaplan-Chinchilla-reanalysis chain in predictability and budgets, then read data and deployment costs. To assess whether test-time scaling is taking over, begin with the proposers, verifiers, and allocation policies in post-training and inference-time computation, then examine capability evaluation and negative results. To follow new work, first identify the question raised in the community radar, return to the primary evidence, and decide which connection in the narrative should change.

This survey prioritizes representative studies that explain turning points rather than mechanically ranking papers by citations or social-media exposure. References to “current” or “latest” are relative to the latest incremental search date, November 2, 2026. Reading scope and unresolved issues are disclosed in the chapters and metadata.

T1 Predictability, Budgets, and Training Recipes: What Is the Same Curve Actually Comparing?

> Verification date: 2026-09-16. Scope: empirical pretraining laws, budget allocation, optimization across scales, overtraining, and capability extrapolation. Evidence comes from the original papers; this chapter does not claim exhaustive coverage through the verification date. Core methods, experiments, and limitations were checked in targeted readings of the full texts; experiments were not rerun, nor were all appendices independently recalculated. See the foundational sources and additional sources for sources and reading depth. Direct lines of inheritance are identified in the text; other connections are this survey's synthesis of how the research problems evolved.

1. The Initial Prediction Target Was the Marginal Return on Additional Resources

The early engineering problem was not a lack of awareness that larger models and more data might help. It was uncertainty about how much another order of magnitude of resources would improve performance, and whether a plateau reflected the task's limits or a problem with the implementation. Across translation, language, image, and speech tasks, Hestness et al. adjusted both model capacity and optimization settings to measure the relationship between dataset size and generalization error. They distinguished a small-data region resembling guessing, a region of power-law improvement, and eventual saturation. This made learning curves a planning tool, without proving that every task could improve indefinitely at the same slope. Hestness et al., 2017

Measuring data scaling alone was still insufficient: with a fixed budget, increasing the parameter count reduces the number of training tokens that can be processed. Kaplan et al. placed parameters, data, and training compute within one empirical framework, emphasizing that the greater sample efficiency of large models could allow an early-stopped large model to outperform a smaller model trained more fully. Their compute-optimal estimate was approximately N_(opt) proportional to C^(0.73), but it primarily counted

non-embedding parameters and used compute adjusted for batch size. This exponent cannot be detached from those definitions and compared directly with any later FLOPs curve. Kaplan et al., 2020, §1 and §6

From Loss Prediction to Practical Capabilities: GPT-3 and Gopher Exposed Another Layer of the Problem

GPT-3 was an important step in putting this resource perspective into practice: models ranging from 125M to 175B parameters received task examples without weight updates, demonstrating the potential for in-context learning to change with scale. It shifted attention from how much cross-entropy decreased to whether a single pretrained model could handle more tasks. These results could not, however, simultaneously establish optimal training budgets, the causal origins of each capability, and the absence of data contamination; the original report also disclosed shortcomings in its contamination filtering. Evaluation with a few prompt examples does not imply that the model never encountered relevant knowledge during pretraining. Brown et al., 2020, §2-4

Gopher extended comparisons across model sizes to a broad range of tasks. Knowledge and reading comprehension improved more clearly, whereas some logic and mathematics tasks benefited less, and performance on some tasks even worsened with scale. Pretraining loss is therefore a compressed measure of model behavior, while capabilities emerge from the interaction of the model with task distributions, prompts, and evaluation. The point is not that average loss is useless, but that an average cannot by itself identify which capabilities have entered a regime of predictable improvement. Rae et al., 2021/2022, §4.3

2. Chinchilla Revised Resource Allocation, Not Predictability Itself

Chinchilla directly revisited parameter/token allocation under a fixed compute budget. It estimated optimal configurations through three approaches: training-curve envelopes, IsoFLOP curves, and a parametric loss model. A 70B-parameter model trained on 1.4T tokens demonstrated the benefits of training a smaller model for longer, using a training compute budget comparable to Gopher's. One commonly used fit is

$$L(N, D) = E + AN^{-\alpha} + BD^{-\beta}, \qquad C \approx 6ND.$$

Here, D counts processed tokens, L is defined on a specified validation distribution, and the constants and exponents come from a particular training recipe. The expression 6ND is only an approximation for training dense Transformers; it does not include every context-related or systems overhead. Hoffmann et al., 2022, §3-5

This survey's algebraic explanation substitutes D=C/(6N) and sets the derivative to zero, yielding

$$\alpha A N^{-\alpha} = \beta B D^{-\beta}, \qquad N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}, \quad D_{\text{opt}} \propto C^{\alpha/(\alpha+\beta)}.$$

The optimal allocation balances the marginal returns from two reducible error terms. Only when their exponents are similar do the budget exponents for parameters and data both approach one-half. Roughly 20 tokens per parameter is thus an empirical approximation for a particular training objective and recipe,

not an upper bound beyond which additional data becomes useless. The original paper's three fitting methods did not produce identical results either. Tables 2 and 3 of the original paper

Why Can Smooth Curves Produce Different “Optima”?

Porian et al. directly investigated the differences between Kaplan and Chinchilla, examining output-layer costs, warmup, and how learning rate and batch size change with scale. After adjustments, their allocation trends on two datasets were closer to Chinchilla's. Observed scaling effects can therefore be entangled with insufficient optimization of small models or inconsistent cost accounting; they cannot all be interpreted as physical properties of model capacity. Porian et al., 2024; revised version consulted

The fitting procedure itself also needs auditing. Besiroglu et al. reconstructed Chinchilla's data from figures and repeated its third, parametric fitting approach, identifying issues with the parameters and confidence intervals. Their new fit was closer to the trends from the paper's first two methods. This was a check based on limited available material, not a full replication using complete original training logs. The relatively robust conclusion that models and data should grow together should be distinguished from the more fragile claim that a particular set of constants can extrapolate precisely across four orders of magnitude. Chinchilla Scaling: A replication attempt, 2024

Equally Good Fits Can Give Different Forecasts

A better optimized fit need not identify a unique forecast. Mlodozeniec and Frei (25 October 2026) distinguish solver error from the Rashomon effect: different parameters can fit the observed ladder nearly equally well yet extrapolate differently. Their branch-and-bound method bounds fitting loss over parameter boxes, discards impossible regions, and can certify near-optimality and the prediction range of acceptable fits within a declared domain.

The study reanalyses four published ladders, including digitized Chinchilla and Llama 3 points, alongside synthetic examples. A crucial boundary separates the demonstrations: the real five-parameter prediction bands use local search and are inner approximations, because tight certified envelopes exceeded the available budget. The certified synthetic envelopes therefore should not be generalized into complete certification of every reported real-data forecast. Guarantees are conditional on the chosen power-law family, parameter bounds, fitting objective and tolerance; they neither model run-to-run randomness nor establish statistical confidence or correct functional form. This extends the earlier replication debate: report optimization error and viable forecast variation separately, then test forecasts on new scales.

3. Optimization Is Also a Scaling Variable: Tensor Programs and μP

These disagreements raise a deeper question. If the best learning rate and update magnitude change with width, a curve connecting models trained with uncalibrated hyperparameters may be measuring the quality of tuning. Yet searching for hyperparameters from scratch for every large model consumes budget that could otherwise support the final training run. **What needs to transfer is not only model capability, but also the training recipe.**

Tensor Programs IV approached this issue through infinite-width parameterizations. Some limits turn a network into a kernel model with approximately fixed features; other parameterizations preserve feature learning. This distinction provides a theoretical foundation for the subsequent μP work, but does not

directly imply the loss exponents of real language models. It addresses how networks can maintain nontrivial learning dynamics as width changes. Yang & Hu, 2020/2022

Tensor Programs V turned this idea into μTransfer: tune a narrow model, then transfer suitable base hyperparameters to a wider model. The mechanism is not simply to multiply the global learning rate by a constant. It coordinates initialization, learning rates, and readout scaling across different tensors so that updates have an appropriately sized effect on activations. For example, under its Adam recipe, if a hidden matrix's width multiplier is m, its learning rate is eta_(base)/m; vectors and input and output layers follow different rules. Extracting just one of these scaling formulas while retaining the other settings of standard parameterization does not implement μP. Yang et al., 2022, Table 3 and Appendix B

The paper used a roughly 40M-parameter proxy to search for hyperparameters for a 6.7B GPT-3-style model, reporting a tuning budget of approximately 7% of the large model's pretraining FLOPs. This suggests that small models can help choose the optimization settings behind a loss curve, as well as estimate the curve itself. Experimental details matter, however: the large-model comparison used different positional encodings, and numerical issues led the μP run to use FP32 while the baseline used FP16. Consequently, the full downstream difference cannot be attributed solely to parameterization. The theoretical basis for width transfer is also stronger than for transfer across depth, batch size, or sequence length; regularization hyperparameters do not have the same guarantees. §6.1 and §7.4 of the original paper

This survey therefore separates the cost of a scaling experiment into finding a recipe and executing it. The primary value of μP is to reduce the former and reduce optimization mismatch across scales, rather than to claim that changing parameterization improves every power-law exponent. This also explains why new scaling papers should report tuning budgets and failed runs: these costs determine whether a method can genuinely guide the next expensive training run.

Effective Learning Rates Add Another Experimental Control

Matching nominal learning rates need not match optimization dynamics. Yu et al. (29 October 2026) track each parameter block's effective rate as nominal rate divided by its norm. Replaying recorded norms or adjusting blockwise rates largely aligns weight-normalized and weight-decayed training, suggesting that some apparent normalization gains arise from the induced schedule. Their AIR schedule affinely adjusts inverse-square-root decay to a finite horizon.

Under Muon and fixed-norm comparisons, MoEs with 110M and 790M active parameters train on 128B and 500B tokens respectively. Matching warmup duration, terminal rate and the cumulative rate sum, while allowing peaks to differ, yields about 0.01 lower terminal loss than cosine. These are individual trajectories, with schedule shapes selected during evaluation and unequal tuning sweeps. The supporting theory optimizes alignment under fixed-target, constant-coefficient dynamics, not language-model loss generally; transferring recorded trajectories also differs from choosing a deployable schedule beforehand. This adds a control to scaling experiments: inspect realized relative updates and schedule selection, rather than attributing a curve shift to normalization or capacity alone.

4. Checking a Curve Requires More Than Final Weights

One of OPT's contributions was to document restarts, loss anomalies, and hardware problems that are difficult to infer from a final curve. Real training budgets include recovery and the work required to keep runs operating; they are not determined solely by successfully processed tokens. Its open models support external analysis, but data, implementations, and recipes still differ between OPT, GPT-3, and other families. Matching nominal parameter counts does not fully control these variables. Zhang et al., 2022, §2.5

Pythia made the ability to study scaling a more explicit design objective: eight sizes, each trained on both the original and a deduplicated version of the data, for 16 models in total. It controlled data order within each data version and provided 154 checkpoints per model. This lets researchers ask when a behavior emerges and whether it changes consistently with training time and model size. The trade-off is that a uniform schedule does not automatically provide compute-optimal training for every size. A suite suited to studying dynamics has not necessarily solved the optimal budget-allocation problem. Biderman et al., 2023, §2

OLMo extended openness to data, training code, logs, checkpoints, and evaluation. Its connection to the problem addressed by Pythia is clear: testing an empirical law requires allowing others to reconstruct the process that generated its data points. **This survey's assessment** is that open model suites are part of the measurement infrastructure for scaling laws. Their scientific value should not be judged solely by their leaderboard positions at release. Groeneveld et al., 2024

5. Beyond the Training Optimum: LLaMA and the New Problem of Overtraining

Chinchilla optimizes the training cost of reaching a given loss. LLaMA explicitly included inference cost in its design motivation: a smaller model may be better suited to extensive subsequent use even if it requires more pretraining tokens. Its results made training beyond Chinchilla-style ratios a practical requirement. However, cross-family performance comparisons also change data and recipes, so they are not ablations of parameter count alone. Touvron et al., 2023, Introduction

Here, overtraining means exceeding the tokens-per-parameter ratio that is optimal for training compute. It does not mean that validation performance has already deteriorated from overfitting. Gadre et al. therefore asked whether loss retains a stable structure away from the optimal frontier. They measured 104 models with 11M-6.9B parameters across three types of corpora, covering token multipliers M=D/N from 5 to 640. Setting alpha=beta=2eta in the base model and substituting M and C ~= 6ND gives

$$L(C, M) = E + (aM^{\eta} + bM^{-\eta})C^{-\eta}.$$

This expression separates scaling compute from choosing training duration. Within the range where the fit holds, changing M may primarily shift the curve's position without disrupting a common compute exponent. This is not an a priori guarantee: the approximate exponents, data recipes, and optimization settings in the experiments all require examination. Gadre et al., 2024, §2-3

The authors also mapped validation loss to average error across a set of downstream tasks. Two boundaries matter: the task set was selected using small-model performance, and an average relationship cannot replace predictions for individual tasks, let alone automatically cover post-training.

The experiments were also preceded by a hyperparameter search, so the small-model budget needed for prediction should not be described as the total cost of the study. This survey interprets the result as providing useful interpolation and extrapolation tools for deployment-oriented training, rather than establishing that arbitrarily long training yields the same returns. Experimental setup and §6 of the original paper

6. Llama 3: Separating Budget Prediction from Task Prediction

The Llama 3 report illustrates how scaling laws can become a decision procedure for a large training run. The researchers trained smaller models of roughly 40M-16B parameters, spanning approximately 6 x 10^(18) to 10^(22) FLOPs, found minimum losses along IsoFLOP curves, and estimated how the optimal token count changed with compute. Extrapolation suggested a configuration of about 402B parameters and 16.55T tokens; considering the flat region around the minimum, they ultimately selected 405B. This is closer to the engineering purpose of scaling laws than fitting a line after the fact: identifying candidate configurations and the basis for choosing between them before training the target model. The Llama 3 Herd of Models, §3.2.1

Capability prediction is a separate layer. The first step fits the relationship between training compute and the negative log-likelihood of correct answers on the target task; the second maps that NLL to accuracy. The second step uses not only small models but also larger Llama 2 models as anchors, despite differences in data and tokenizers. Successful extrapolation in the report therefore validates a particular pipeline. It does not prove that all large-model tasks can be predicted precisely using only small models trained on the same distribution. Ibid., §3.2.1

The report also exposes a hidden variable in token accounting: tokenizer improvements increased the number of characters represented by each token. Two models trained on a trillion tokens need not have processed the same amount of text. Loss comparisons must also respect their units: per-token cross-entropy over different vocabularies cannot be treated directly as the same quantity. Large-model technical reports should therefore be broken into separately testable claims, examining the controls for configuration selection, validation-loss prediction, task mapping, and systems implementation, rather than compressing the success of an entire model into “scaling works.” Ibid., §3.1-3.3

Scaling Ladders Must Preserve Data Reuse

Scaling ladders must preserve the regime whose behavior they predict. Microsoft AI’s 15 October 2026 report restricts each source’s unique examples in smaller runs so that its repetition count matches the final run, while keeping tokens per active parameter fixed. This source scaling addresses a recipe-transfer failure: small runs seeing fresher data may favor insufficient regularization for the larger repeated-data run. The report also jointly tunes learning rate and weight decay, then validates fitted prescriptions at an intermediate scale.

Evidence types remain separate. Figure 2 reports a ladder extrapolation to MAI-Base-2, whereas the advertised equal-loss FLOPs advantage uses another fit that includes the completed model and interpolates its matching point. It is not an independently validated prediction of that advantage, a wall-clock speedup, or a capability-wide ranking. Internal bits-per-byte evaluations and a jointly changed data, optimizer and architecture recipe also limit causal attribution. This adds a practical check to budget

forecasting: reproduce data reuse and optimization conditions in the small-scale experiments, then distinguish forecast validation from retrospective efficiency estimation.

7. Which Cost Should Be Optimized Next?

This line of work has advanced from a single learning curve to a conditional experimental procedure: specify the data distribution and evaluation objective, ensure comparable optimization quality across scales, fit under an explicit cost definition, and finally test on scales or tasks excluded from fitting. This procedure also explains why apparently contradictory optimal ratios may both be valid: they may optimize different training recipes or serve different deployment requirements.

For a model that will provide a long-running service, the objective must also include request volume, generation length, and hardware constraints. T3 Architecture and Deployment develops this topic. When training tokens no longer correspond to new useful information, the analysis must incorporate unique data volume, mixture proportions, filtering, and generation mechanisms, as discussed in T2 Data. Tasks that have not yet entered a reliable extrapolation regime should retain their uncertainty, rather than having it replaced by a smoother average curve.

T2 Data Bottlenecks: From Token Counts to Useful Information, Mixtures, and Training Order

Verification date: 2026-10-19. Scope: unique data volume, deduplication, selection, domain mixtures, data curricula, supply forecasts, and synthetic feedback. The chapter distinguishes experimental evidence in the original papers from this survey's synthesis of mechanisms; data processing and training experiments were not independently rerun. See the foundational sources and additional sources for reading depth. Works checked only at the abstract level, including TinyStories, Phi-3, and Scaling Laws for Transfer, remain in the source registry but are not used to support this chapter's technical conclusions.

1. Data Becomes a Bottleneck Because D Carries Too Many Meanings

In the dense-pretraining approximation C ~= 6ND, D first denotes the number of tokens processed during computation. This accounting variable can directly represent statistical dataset size only if every token supplies independent, homogeneous new information. Actual corpora contain repetition, domain differences, errors, and overlap between training and test data; identical token counts can also come from different tokenizers. **This survey organizes the discussion** by separating several questions: how much new content is available, which content to retain, how to allocate training proportions, in what order to train, and who generated the new content. Together these factors change the return curve, but they are not a single “quality parameter.”

The training runs used for Chinchilla's scaling-law fitting analysis covered less than one epoch, so its formula alone cannot determine the value of repeatedly training on a finite corpus. This was not true of every domain in the final 1.4T-token model's training data: Table A1 reports 1.24 epochs for MassiveWeb and 3.40 for Wikipedia. The analytical runs must be distinguished from the final training recipe. Muennighoff et al. directly separated unique data volume, repetition count, and model capacity, fitting

diminishing marginal returns to repeated tokens. In some experimental settings, the gap after four epochs of repetition was small, while further repetition showed substantial diminishing returns, and individual runs deteriorated; their saturating fit cannot fully represent that deterioration. This revises the assumption that all tokens are equivalent, but does not establish a threshold under which repeating any dataset four times is safe. Hoffmann et al., 2022, §5; Muennighoff et al., 2023, §3 and §5-6

Repetition Value Depends on the Reference and Fixed Budget

How many epochs to use depends first on which budget stays fixed. Chai and Xiong, 4 October 2026, §§3-7 distinguish two references: one pass over the same corpus asks whether extra compute helps; fresh data at matched model size and total tokens measures repetition's opportunity cost. Their CommonCrawl grid spans 23M-361M non-embedding parameters, 5-300 total tokens per parameter, and 1-16 epochs. They model excess loss over fresh data as Delta L=A z^p, with z=(R-1)N_(total)/U: R is epochs, U unique tokens counted as prediction targets, and N_(total) includes embeddings. The excess-loss fit uses 113 of 140 repeated-data configurations above the 0.002 bpb resolution threshold. In a separate fixed-U sweep, larger models reach minimum loss after fewer epochs; when U grows proportionally to non-embedding parameters N, the preferred epoch count changes little on CommonCrawl.

Counts still do not determine the outcome: at matched model size, unique corpus and epochs, consecutive shard replay raises loss relative to spacing repeats across epochs. The main grid has one run per configuration, uses no weight decay or dropout, and measures held-out bits per byte. Its fitted minimum is not a universal capability ceiling under finite data. The contribution clarifies references, fixed quantities and replay schedules; it does not establish a universal repetition threshold.

Recipes Shift Repetition Returns, but Sequential Time Still Matters

Beyond corpus size and repetition count, the training recipe can move the repeated-data frontier. Min, 11 October 2026, §§3-6 jointly tunes batch size, learning rate, weight decay and stopping epoch in the main 1B-token DCLM-Baseline experiment, then extends the recipe to 0.333B-3B unique tokens using the selected learning rates. Models are nominally 153M-1.5B dense networks. Validation loss is U-shaped in batch size in the main 1B-token experiment. Smaller batches are consistent with useful stochastic regularization, but smaller is not always better. Repacking documents each epoch, tying embeddings and applying embedding decay further improve results: a fixed recipe's repetition limit is not an intrinsic data limit.

Smaller batches require more sequential updates, motivating separate GPU and deadline constraints. However, the main time axis uses optimizer-step equivalents, and comparisons using exactly a specified GPU count do not imply that increasing the available-GPU upper bound is harmful. Hardware latencies and the 70B/10T scenario are explicitly simulations or distant extrapolations. The nominal models become smaller after embedding tying, and recipes are validation-selected. The evidence supports retuning repeated-data training, without establishing a universal batch size or measured end-to-end training time.

The options in a data-limited setting are therefore not restricted to enlarging the model or stopping training. Repeating existing data, relaxing filters, adding heterogeneous domains, and generating new examples can all increase the nominal D, but introduce different biases. Comparisons should report total

processed volume, unique data volume, and its sources together. Otherwise, an apparent improvement in data efficiency may simply reflect undisclosed repetition or distribution shifts.

2. Deduplication First Repairs Counting and Evaluation, Before Addressing Quality

Lee et al. treated repeated long strings and near-duplicate documents separately, comparing language models before and after deduplication. In their C4 experiments, deduplication reduced copying of training text during generation and improved the efficiency of training-data use. The problem is not merely that duplicates consume budget: overlap between training and validation sets can also make validation loss appear better than generalization to genuinely new examples. A scaling curve fitted to such measurements may be smooth while incorporating increasingly strong memorization. Deduplicating Training Data Makes Language Models Better, methods and experiments

This does not contradict the finding that deliberate repetition can still help when data is limited. Deduplication research primarily removes unintended duplication from corpus collection and overlap with evaluation data. Data-constrained research instead asks how much value another pass provides once the training set is clearly defined. **This survey's synthesis** is that the basic data unit and the source of repetition should be specified before calculating epochs. Uncontrolled repetition in naturally collected corpora should not be conflated with traceable resampling in an experimental design.

Dolma provides infrastructure for checking this distinction: a large open corpus and modular quality, content, and deduplication pipelines allow processing steps to be replaced and evaluated separately. Its analysis also suggests that the content excluded by different filters need not overlap substantially. Processing steps that seem individually mild can, in combination, leave an insufficient data supply. The net benefit of filtering therefore depends on retention rate, the available corpus pool, and training demand, rather than on using as many filtering rules as possible. Soldaini et al., 2024, filtering and deduplication analysis

3. From Random Expansion to Active Selection: When Is a Target Distribution Needed?

Sorscher et al. posed a challenge: randomly enlarging a dataset may be an inefficient baseline. If a selector knows which examples are most informative, can the learning curve improve faster? Their theoretical models demonstrate the possibility of going beyond power laws under ideal selection, while image experiments reveal that practical selection metrics do not transfer easily. Selection cost, class coverage, and the number of training epochs all affect the net benefit. These results therefore cannot be presented as evidence that general-purpose language models already achieve exponential error reduction. Beyond neural scaling laws, 2022

DSIR addresses a more specific and actionable condition: **samples from the target domain are already available**. It represents documents using low-dimensional features such as hashed n-grams, estimates the density ratio between the target and the raw corpus in that feature space, and performs importance resampling. Intuitively, feature combinations that are common in the target but rare in the raw pool receive higher weights. This offers a cheaper route to data selection than running a large quality model

on every document. Data Selection for Language Models via Importance Resampling, methods and experiments

Its evidence comes primarily from continued pretraining for domain adaptation and comparisons with matched training-token budgets. Target samples determine what to select, while the low-dimensional approximation determines which distributional differences are visible. The method does not discover an unknown general-purpose target on the researcher's behalf, nor does n-gram matching guarantee semantic correctness. This creates a substantive branch in the research: a known deployment domain permits optimization of distribution matching, while unknown future tasks call for broader domain coverage and robust objectives. There is no single optimal filtering score for both.

4. FineWeb: Why Is “Educational Quality” Not Quality for Every Task?

FineWeb turned web cleaning into a set of choices that could be tested through pretraining outcomes, producing a corpus of roughly 15T tokens. FineWeb-Edu then applied model-assisted filtering for educational value, retaining around 1.3T tokens. Its significance lies not only in releasing a larger dataset, but also in connecting filtering thresholds with the capabilities of trained models and evaluating processing steps with comparable models and training settings. Penedo et al., 2024, §4

The central trade-off lies in the definition of “good.” Educational text can strengthen academic-knowledge and reasoning evaluations, but also changes the coverage of nonacademic websites, social language, and different language varieties. The paper's comparisons show that perplexity across domains does not uniformly favor either the educationally filtered version or the broader web version. An advantage on one benchmark therefore cannot be recast as greater learning value for every kind of text. A filter selects a portfolio of capabilities. Ibid., §4.2 and §6

DataComp-LM constrains this problem from another direction: it fixes the experimental framework and compares data processing across multiple model sizes, aiming to keep architecture and training configuration from becoming hidden variables. It supports the value of model-assisted filtering while acknowledging limitations in combination searches, run variation, large-model experiments, and evaluation coverage. FineWeb and DataComp-LM are two approaches to the shared problem of attributing gains to data; one algorithm does not directly replace the other. DataComp-LM, 2024, §6

This survey therefore treats “high-quality data” as a conclusion that must specify an objective and a control: lower loss on a designated domain under the same training budget, or higher scores on a particular set of downstream tasks? Is the scoring model's execution cost included? Does the filtered corpus remain large enough to support long training? Without fixing these conditions, a smaller curated dataset cannot be compared fairly with a larger, more diverse one.

5. Landmark: How DoReMi Turns Unknown Downstream Tasks into a Mixture-Optimization Problem

Choosing domain proportions is cheaper than scoring individual documents, but challenges remain. Sampling in proportion to corpus volume overrepresents large domains, while the domain with the highest average loss may simply be noisy rather than the best place to spend further training. DoReMi's key insight is to compare the current model's loss with a reference model, focusing on “excess loss” that may still be learnable rather than absolute difficulty. Its domain-level objective can be summarized as

$$\min_{\theta} \max_{\alpha \in \Delta} \sum_{i} \alpha_i \, \mathbb{E}_{x \sim D_i}[\ell_\theta(x) - \ell_{\text{ref}}(x)].$$

alpha is the domain-sampling distribution. The actual algorithm also includes token-level normalization, excess-loss clipping, weight smoothing, and averaging over training. This abbreviated formula cannot substitute for those details in a reproduction. Xie et al., 2023, §3 and Algorithm 1

The method first trains a reference model, then uses an approximately 280M-parameter proxy to estimate domain weights, and finally trains an 8B model on the resulting mixture. What transfers is the data allocation, not a direct enlargement of the proxy's weights. In experiments on the Pile's 22 domains and a separate GLaM dataset, the authors compared domain losses and downstream results. They reported an average gain of roughly 6.5 percentage points across five generative one-shot tasks, as well as fewer training steps to reach baseline performance. The additional small-model compute was approximately 8% of the compared large model's pretraining compute, so this search cost should remain part of the assessment. §4 of the original paper

Why might this mechanism work? **This survey's explanation** is that the reference model provides a baseline for differences in intrinsic difficulty across domains, helping avoid spending all resources on irreducible error, while distributionally robust optimization exposes domains neglected by the current mixture. Its limits arise from the same mechanism: a weak or biased reference model makes excess loss less reliable; coarse domain partitions average away differences within a large domain; and content that a small model considers worth learning may not offer the same marginal return to a large model. The paper's success across scales provides empirical support, not a theorem covering every proxy and target size.

6. Direct Successor RegMix: Learning Recipe Rankings Instead of Closely Simulating Training

RegMix explicitly compares against proxy-training methods such as DoReMi and proposes a different proxy: train many extremely small models across a wide range of mixture proportions, then regress validation loss against the proportion vector. If small models rank candidate recipes approximately as large models do, the proxy need not reproduce large-model training step by step. Liu et al., 2024/2025, §1 and §3

Its experiments trained 512 small proxies, each with roughly 1M non-embedding parameters and 1B training tokens, fitted a regressor, and validated mixture rankings using multiple 1B models. The 1M count has a specific definition: it must not be read as a million parameters for the entire model including word embeddings. Search cost was approximately 2% of a single compared 1B-model training run,

demonstrating the value of using small proxies to cover a broad configuration space. Methods and experiments of the original paper

The limitations are equally specific. The primary optimization target was Pile-CC validation loss because it correlated with downstream performance in those experiments; it may not suit medical, code, or multilingual services. Comparisons with DoReMi also renormalized its domain weights over the available domains, which the authors acknowledge may affect the baseline. The justified conclusion is that small-model ranking can reduce search costs within the studied candidate domains, objectives, and scale range, not that RegMix has universally superseded DoReMi. §5 and comparison notes in the original paper

Both methods assume that recipes can transfer across scales, but estimate them differently: DoReMi uses reference losses and dynamic robust optimization, whereas RegMix uses ranking patterns from many small experiments. The earlier Data Mixing Laws also attempts to fit the functional relationship between domain proportions and loss directly. This survey checked that work only at the abstract and introduction level and does not use it to add stronger cross-scale conclusions. Ye et al., 2024

From Ranking Transfer to Joint Prediction: Does the Best Mixture Change with Scale?

Stable rankings do not predict the target-scale loss or rule out a moving optimum. Shukor et al., 2025, §§2-6 jointly model target-domain loss using parameter count N, training tokens D, and domain proportions h. In their additive form, only the bias term depends on the mixture, so a scale-independent optimum follows from the model structure. The joint form also makes the coefficients of the parameter and data terms depend on h, allowing budget-dependent optima. The advance is to turn recipe transfer into competing, testable fitting assumptions, not to establish that every task's optimum must move.

The paper tests unseen mixtures and larger models in language, native multimodal, and vision pretraining. Its language experiments fit 412M-1.4B models and evaluate 3B/7B models. Greater flexibility does not improve every domain: the joint form reduces Wikipedia's mean relative validation-loss prediction error from the additive form's 4.45% to 2.09%, but raises GitHub's from 1.17% to 2.51%. These are loss-prediction errors, not task-accuracy gains. In the “10-20 runs” analysis, the six/eight-domain experiments fix model size; this does not establish that 20 training runs suffice for arbitrary cross-scale searches. Most fitting uses a constant learning rate. The optimized 7B runs switch to a cosine schedule, so their downstream results must be distinguished from numerical extrapolation of the original fitted curves.

The approach still assumes a fixed mixture during training, omits repetition in scarce domains, and uses target loss as an indirect measure of downstream capability. **This survey's synthesis** is that it adds explicit coupling between scale and mixture without resolving data exhaustion or changing training stages. That connects it to the curriculum question below, rather than replacing research on training order. Original paper, §§5-6 and §8

New Evidence: Small-Model Weight Responses Need Not Extrapolate Monotonically

Jane Street's September 14, 2026 study adds a condition to recipe transfer. On an internal text benchmark, researchers randomly assigned sequences log-uniform weights from 0.01 to 10, trained three model families for three epochs, and fitted an effective weight exponent p^* from loss reductions on the same training data. Weight responses tended to rise and then fall with model scale; additional epochs also shifted the peak. This suggests checking whether proxy and target models occupy similar weight-response regimes. It is neither a direct comparison with DoReMi or RegMix nor a scaling law for held-out generalization gains. Internal data limit external verification, and adjusting weights using 1/p^* remains an untested proposal. Renda & Mani, article; estimator note

7. Static Proportions Are Still Insufficient: SmolLM2 Introduces Training Stage as a Variable

When small models are trained for a long time, the best corpus for an early stage may not be the best for a later one. SmolLM2 trained its 1.7B model on roughly 11T tokens, adjusting the proportions of web, code, mathematics, and synthetic text over four stages. Building on open data infrastructure such as FineWeb, it advances the question from choosing one best mixture to deciding what to feed the current checkpoint next. Some of the report's data experiments begin from intermediate checkpoints, because weaker, randomly initialized small models may not reliably assess the late-stage value of difficult data. SmolLM2, §4.3-4.7

This offers a caution for proxy design: the usefulness of data may interact with capabilities the model already possesses. Selection based solely on extremely small proxies trained from scratch may miss examples that require foundational knowledge before they become useful. The report does not, however, establish a universal curriculum-learning law. Later stages combine data changes, learning-rate decay, and additional compute, so final gains cannot all be attributed to one data source. Training also encountered loss spikes that were difficult to explain fully, indicating that the complete recipe still contains factors not fully modeled. Ibid., stage analysis and ablations

SmolLM2 also illustrates the interface between pretraining and post-training: a base model's advantage does not necessarily translate automatically into an instruction model's advantage when existing instruction data is reused, creating a need for corresponding post-training data. **This survey's synthesis** is that data optimality depends on at least three factors: model size, the current training stage, and the ultimate use case. Total token counts alone cannot distinguish them. To become useful engineering tools, future mixture laws should include proxy checkpoints and training stages in their verifiable definitions.

8. The Data Wall Is a Conditional Forecast, Not an Established Event

The 2024 revision by Villalobos et al. combines estimates of the effective stock of public human-generated text with growth in training demand. Under assumptions including continuation of existing trends, it gives a possible supply-constraint window of roughly 2026-2032, with a median estimate around 2028. This is neither real-time confirmation of conditions in 2026 nor a claim that all text in the world will run out on a particular day. It concerns training supply under specified assumptions about sources, availability, and effectiveness. Will we run out of data?, v2

This forecast has an important connection to overtraining. To reduce subsequent inference costs, developers may train smaller models on more tokens, changing data demand under the same compute budget. Conversely, broader definitions of usable data, repeated training, transfer, or generation of new data change the supply model. It is therefore reasonable for the interval to shift with demand objectives and data efficiency. The continued release of new systems does not by itself falsify the data-wall hypothesis. What needs updating is the original forecast's input assumptions and observable data. Ibid., §2-3

This survey consequently distinguishes three levels: whether the stock of web content continues to grow; how many useful examples remain after licensing, cleaning, deduplication, and target matching; and whether models are becoming more efficient at learning from those examples. These belong respectively to supply, processing, and learning algorithms. Total internet byte counts alone cannot determine the remaining room for language-model training.

9. For Synthetic Data, the Feedback Mechanism Matters More Than the Source Label

Shumailov et al. studied recursive training in which a model's outputs train the next generation, demonstrating model collapse through accumulating errors such as finite-sampling error and the progressive loss of tail information. The version checked here is arXiv v3; an unread formally published version is not being presented as evidence of full-text review. The conclusions concern particular feedback mechanisms and do not imply that any inclusion of synthetic data necessarily causes degradation. The Curse of Recursion, v3 consulted

Gerstgrasser et al. directly tested a crucial condition: is the original real data replaced, or retained and accumulated together with data from successive generations? In their tasks and theoretical settings, accumulation can prevent the corresponding degradation. Data volume and compute also increase with accumulation, however, so this does not prove that all strategies can avoid collapse under a fixed budget. This is a research chain with direct problem inheritance: the successor changes the data-update mechanism on which the previous conclusion depended. Is Model Collapse Inevitable?, 2024

This discussion ultimately converges with DoReMi, FineWeb, and data curricula: generators, filters, and samplers all reshape the training distribution. Updates to the survey should record real-data retention, verification methods, tail coverage, and total cost, rather than treating “synthetic” as a binary label sufficient to explain success or failure. In particular, settings with additional feedback, such as code tests or mathematical verification, involve a different information process from a loop that merely imitates the previous generation's text. Their empirical research chains should be read alongside the post-training chapter, rather than inferred directly from collapse experiments.

Wild AI Web Text: Added Tokens Can Hurt Even When Human Data Are Retained

Recursive replacement is not the only risk. WildAI (Russell et al., 2026), §§4-6 fixes the human-data budget and adds naturally occurring web text labeled as AI-generated by detectors. Additions increase tokens and compute; this is not fixed-compute substitution. Its model separates a saturating benefit from a harm term: initial gains under data scarcity can reverse as additions grow. It fits 726 models of 19.9M-268M parameters and holds out 74 models at 477M/973M. Appendix D.1 also compares candidate forms on these sizes; we therefore do not treat the reported errors as a final test independent of model selection. This extends the mixture question: an added token's value can become negative, depending on the target distribution and existing budget.

Evaluation mixtures can conceal that harm. Among runs whose loss rose on the human-labeled FW26 subset, a validation mixture containing 22.3% AI-labeled tokens reported improvement for 95.5%. Reporting target-domain losses separately therefore matters when selecting a recipe. The evidence concerns English web text, detector-defined splits, sub-1B models and pretraining next-token loss. It does not establish that deliberately generated, verified synthetic data are generally harmful, that downstream capabilities deteriorate, or that the same thresholds apply to frontier models.

10. How Should This Branch Continue to Be Updated?

New work worth tracking should answer a specific unresolved question: does it reduce the computational cost of data selection, demonstrate transfer of mixture proportions to larger models, incorporate training stage, preserve more tail coverage under the same budget, or revise a forecast of data supply? Claims of higher-quality tokens without controls or defined objectives are difficult to accumulate into knowledge.

This also provides a practical accounting framework for reading papers: examine unique example counts, processed tokens, selection costs, proxy-search costs, target-domain loss, downstream coverage, and evaluation independence together. A selector may improve benchmark scores without expanding the total data supply; a synthetic mechanism may expand supply without providing new verifiable information. Keeping these accounts separate prevents the research conversation from restarting at “the data got better” with every new model release.

T3 Architecture and Deployment: Distinguishing Parameters, FLOPs, GPU Memory, and Actual Costs

Verification date: 2026-10-19. Scope: sparse experts, training parallelism, exact-attention implementations, KV representations and serving, long context, and lifecycle budgets. This chapter discusses algorithmic cost, hardware performance, and model quality separately; acceleration factors reported in individual papers are not treated as constants that generalize across hardware. See the foundational sources and additional sources for original sources and reading depth. No systems benchmarks were run.

1. When Parameters Need Not Be Activated, Scaling Laws Need New Coordinates

In dense models, parameter count usually represents both storage capacity and per-token computation. MoE breaks this coupling: a model can contain many experts while activating only a few for each token. Doubling model size therefore no longer implies doubling computation per step. Architecture comparisons must distinguish at least total parameters, active parameters, data volume, FLOPs, memory, and communication. The relationships below are this survey's accounting distinctions, not a claim that these quantities already share a universal conversion factor.

GShard combined conditional computation with automatic sharding compilation, distributing experts across devices. Its multilingual translation experiments demonstrated the possibility of expanding capacity while increasing computation more slowly. They also showed that parameter sparsity does not eliminate systems complexity: routing, cross-device data movement, and compilation at scale still need to be handled. These results concern multilingual translation and cannot be transferred without qualification into claims about equal-budget gains for general-purpose autoregressive language models. Lepikhin et al., 2020, §1-3

Switch Transformers directly simplified routing: each token selects one expert rather than several, reducing routing and communication, while selective high precision mitigates numerical instability in the router. Expert capacity requires buffer space; too little causes overflow tokens to be dropped, while too much wastes computation and memory. Its T5-style objective, C4 data, and TPU comparisons establish measurable speed-quality trade-offs, rather than proving that per-step cost remains constant no matter how many experts are added. Fedus et al., 2021/2022, §2

From Runnable Architectures to Predictable Configurations

Clark et al. separated active computation from total capacity to study shared regularities in routed language models, but their main experiments fixed training at 130B tokens and did not jointly optimize data volume. Fine-Grained MoE explicitly identified this limitation and brought tokens, parameters, and expert granularity into joint optimization. Finer experts increase the flexibility of combinations, but also add routing overhead. This chain has evidence of direct inheritance and shows why a sparse-model scaling law cannot be obtained merely by replacing the parameter count in a dense formula with active parameters. Clark et al., 2022; Scaling Laws for Fine-Grained Mixture of Experts, 2024

Mixtral demonstrated how this architecture could enter open autoregressive models: two of eight experts are selected per layer, with roughly 47B total parameters and 13B active parameters per token. The report explicitly notes that serving memory depends on total parameters, while routing and memory access also affect device utilization; batched workloads more readily achieve higher arithmetic intensity. Describing its deployment cost simply as that of a 13B model therefore omits major constraints. Jiang et al., 2024, §2 and §3

Sparsity Also Changes Hyperparameter Transfer

Choosing a sparse architecture also changes the optimization recipe. Tian et al., 8 September 2026 add the expert activation ratio A=E_(act)/E_(tot) to hyperparameter prediction; this differs from the active-to-total parameter ratio. They fit

$$\eta^* = k_\eta C^{\gamma_\eta} A^{\delta_\eta}, \qquad B^* = k_B D^{\gamma_B} A^{\delta_B}.$$

Here, eta is peak learning rate, B global tokens per update, D training tokens, and C=MD non-embedding training FLOPs, and M the architecture-specific analytically computed non-embedding FLOPs per token. Within their setting, greater sparsity favors a lower learning rate at fixed C and a larger batch at fixed D.

The study reports 1,800 runs, approximately 10M-324M activated parameters, and grouped validation holding out activation ratios or model scales. A frozen prediction falls near the observed optimal region for a 12B-total/324M-activated model with A=1/64, trained on 159B tokens. Activated size remains at the fitted range's boundary: this is one joint extrapolation in sparsity, tokens, and compute, not independent validation along every axis. The experiments share a hybrid attention backbone, Muon optimizer, corpus, and warmup-stable-decay schedule. Overlapping intervals across candidate functional forms and limited seed coverage leave uncertainty; these coefficients do not establish a universal recipe or downstream-capability law.

Expert Capacity Depends on Language Coverage

Expert capacity must be allocated against the diversity it serves. Dag et al. (v3, 21 October 2026) jointly model active parameters, expert count, language coverage and repetition-adjusted exposure. Their main grid contains 64 Top-1 configurations with 93M-590M active parameters. Across the same twelve languages, expanding coverage from twelve to two hundred produces a smaller mean loss penalty with 32 experts than with a dense model: the reported reduction is 83%, at matched active parameters and target-language exposure. Total tokens are not held equal in that comparison.

The fitted law is tested on held-out configurations, but each evaluated language already has fitted coefficients; this is not prediction for unseen languages. Larger-model and longer-horizon extrapolation remains conditional, and restoring average quality can leave individual languages worse off. Nor does logical compute establish physical cost: in the authors' implementation, matching model size, language coverage and training horizon gives median GPU-hours of 1.69 times the dense baseline for 32 experts. Main-grid seed coverage is limited, and downstream accuracy interactions vary by task. The result makes coverage a resource-allocation variable without establishing universal expert benefits.

2. Landmark: DeepSeekMoE Moves from More Capacity to More Useful Experts

When conventional experts are coarse-grained, one expert may handle unrelated knowledge, while several experts may redundantly learn information needed everywhere. DeepSeekMoE addresses these two issues through fine-grained segmentation and shared experts. It reduces each expert's intermediate dimension to 1/m of its original size, increases the number of experts to mE, and correspondingly increases the number activated per token to mk, aiming to keep total expert parameters and active computation comparable. Of these mk active slots, it then reserves |S| for always-active shared experts handling common knowledge, selecting only mk-|S| non-shared experts each time. Shared experts consume part of the existing activation budget. Dai et al., 2024, §3

Omitting residual connections and normalization, the layer output can be illustrated as

$$y(x) = \sum_{s \in S} f_s(x) + \sum_{i \in \mathrm{Top}_{mk-|S|}(g_{\bar{S}}(x))} g_i(x) f_i(x).$$

S is the shared-expert set, and bar(S) contains the remaining, non-shared experts; the second term selects mk-|S| paths only from the non-shared set. The idea is not to equate the number of combinations with capability. It is to assign common and specialized content to different paths while allowing specialized paths to combine more flexibly. The original paper conducted architectural ablations in a roughly 2B-parameter, 100B-token setting before scaling to 16B parameters and 2T tokens. Comparisons with dense references trained on the same corpus come closer to isolating the architectural benefit than cross-company model leaderboards do. §4-5 of the original paper

Two limitations remain. First, expert removal and performance ablations can provide evidence of specialization, but cannot prove that every expert has a clear, independent semantic responsibility recognizable to humans. Second, the paper uses a dense model with the same total parameter count as a strong reference; this survey does not call it a strict performance upper bound for every task and training budget. Finer experts can also cause a token to contact more devices, so matching parameters and FLOPs does not eliminate communication differences.

Direct Successor V2: Once the FFN Becomes Cheaper, KV and Communication Become Bottlenecks

DeepSeek-V2 explicitly inherits this expert structure while addressing two new constraints. Its fine-grained routing controls communication by limiting the number of devices involved in processing a token. On the attention side, it introduces MLA, jointly compressing keys and values into a low-dimensional latent representation. Let the cached latent dimension per layer be d_c, the key dimension dedicated to positional encoding be d_R, and the number of layers be ell. Its cache contains approximately (d_c+d_R)ell elements per token, reducing storage relative to the standard multi-head cache of 2h d_hell elements. DeepSeek-V2, §2

One mechanism deserves close reading. If the up-projections in a low-rank compression can be absorbed into the query/output matrices, the full KV representation need not be reconstructed each time. RoPE's position-dependent transformations, however, disrupt this matrix rearrangement. V2 therefore decouples the position-dependent component rather than merely claiming that low-rank KV is sufficient.

This also explains why MLA's benefits should be verified together with its architecture and implementation. The reported cache reductions and throughput gains are not automatically plug-and-play results for arbitrary models. Ibid., §2.1.2-2.1.3

Direct Successor V3: Load Balancing Itself Can Interfere with the Model Objective

V3 retains MLA and DeepSeekMoE but moves the main expert load-balancing mechanism outside the auxiliary loss. It adjusts routing biases according to expert overload or underload, using those biases to affect selection while retaining the original affinity scores as the weights for expert outputs. It is important to specify that a small sequence-level auxiliary balancing loss remains. “Auxiliary-loss-free” should not be interpreted as the complete absence of balancing terms from the training objective. DeepSeek-V3, §2.1

Its larger-scale implementation also combines FP8, DualPipe, and inter-node communication optimizations. The evidence establishes that the complete recipe can run, supported by local ablations; it does not fully isolate the contribution of every component. The reported 2.788M H800 GPU hours cover the official training stages and correspond to roughly USD 5.576M at the assumed rental price, explicitly excluding preliminary research and ablations. This figure cannot be described as the project's total research and development cost, nor can it be derived solely from the active-parameter ratio. Ibid., Table 1 and §3

3. Fitting a Model into Memory Does Not Mean the Cluster Runs It Quickly

The systems branch faces another limitation: even if the algorithmic budget permits a larger model, a single device may lack memory for its parameters, gradients, and optimizer states. Megatron-LM exploits the structure of Transformer layers for tensor parallelism, reducing redundant communication in naive partitioning, and combines it with data parallelism. In its 8.3B-parameter, 512-GPU experiment, approximately 76% refers to scaling efficiency relative to the selected single-GPU baseline, not to achieving 76% of theoretical peak hardware performance. This distinction directly affects how a paper's FLOPs translate into procurement or operating budgets. Shoeybi et al., 2019/2020

ZeRO instead addresses replicated storage in data parallelism. It progressively shards optimizer states, gradients, and parameters, allowing the cluster's aggregate memory to serve one model more efficiently. It does not make all communication free or eliminate every activation and temporary-buffer overhead. A discussion of theoretically accommodating a trillion parameters and evidence of completed training at that scale represent different levels of evidence. ZeRO can be combined with tensor parallelism because they address different memory and communication bottlenecks. Rajbhandari et al., 2019/2020

This survey treats such work as defining the feasible region for scaling experiments. A mathematically better N,D configuration may run more slowly than a FLOPs-suboptimal configuration if it forces the model across low-bandwidth links, into overly small microbatches, or through frequent recomputation. Comparisons of training optima should therefore specify whether they optimize an ideal arithmetic budget or an actual budget for a particular cluster and deadline. The former cannot automatically stand in for the latter.

Low-Precision Training: Optimization Error Beyond Capacity Loss

Lower precision changes both arithmetic cost and which errors training can remove. Kim et al., 13 October 2026, §§3-4 augment effective-parameter and effective-data models with separate persistent optimization-error terms for forward and backward quantization. Gradients evaluated at quantized weights can oscillate, while backward quantization can introduce bias. Convex analysis and a one-dimensional example motivate the form; they do not prove that real LLMs necessarily share this floor. The authors fit their 280M-910M non-embedding-parameter Qwen3 grid and published Quartet data, holding out larger models or token budgets in separate extrapolation splits.

If the residual terms persist in the target regime, more compute shrinks other loss components until raising precision can become preferable to further enlarging a low-precision model. Yet increasing optimal precision with budget remains a conditional prediction: the cost model counts only matrix multiplication, assumes format speedups and equal block costs, and joint mixing relies on transfer assumptions across layers and passes. This adds precision to resource allocation without replacing end-to-end training-time measurements.

4. Landmark: Why Can FlashAttention Do More Arithmetic Yet Run Faster?

Standard attention computes QK^T, softmax, and multiplication by V. Conventional implementations write a quadratically sized intermediate matrix to high-bandwidth memory, or HBM, and then read it back. On GPUs, this data movement can cost more than some arithmetic. FlashAttention therefore begins by reorganizing exact computation rather than changing the model approximation: it places input tiles in faster on-chip SRAM and maintains global normalization with an online softmax. Dao et al., 2022, §2-3

The key to the online computation is that statistics from two blocks can be merged. If the old block's maximum and exponential sum are m,ell, and the new block's are tilde(m),tilde(ell), the merged statistics are

$$m' = \max(m, \tilde{m}), \qquad \ell' = e^{m-m'}\ell + e^{\tilde{m}-m'}\tilde{\ell}.$$

Rescaling the accumulated output accordingly removes the need to store the full attention matrix in HBM. During backpropagation, the necessary intermediate results are recomputed from the input blocks. Arithmetic work can increase, yet reduced reads and writes make execution faster. This remains exact attention with quadratic arithmetic complexity and should not be conflated with linear attention. Ibid., Algorithm 1 and Theorems 1-2

For BERT-large at sequence length 512, the paper reports roughly 15% end-to-end acceleration relative to the then-current MLPerf 1.1 training record. For GPT-2 at length 1K, it reports approximately 3× acceleration relative to its selected implementation. These factors depend on sequence length and baseline; they are not common gains for all Transformers. The analysis explicitly includes SRAM capacity in IO complexity and does not claim identical benefits on every GPU or at every head dimension. §4.1 of the original paper

This survey's assessment is that this work changes the interpretation of cost in scaling laws. The same model, tokens, and theoretical FLOPs can incur different wall-clock costs because memory access is

organized differently. This must be recorded separately from improvements in the model's statistical efficiency.

Direct Successor FlashAttention-2: One IO Optimization Does Not Remove Every Bottleneck

The second generation found that the first was still limited by thread-block occupancy, shared-memory communication between warps, and non-matrix operations. It reduces non-matrix FLOPs such as normalization, adds parallelism along the sequence dimension, and redistributes work among warps. Roughly 2× attention-kernel acceleration and higher peak utilization on A100 demonstrate that the same FLOP count still does not imply the same time per FLOP. Dao, 2023, §3-4

This explicit successor chain does not replace an incorrect algorithm with a correct one; it removes successive systems bottlenecks while preserving exact outputs. It also leaves a new boundary: a kernel-level speedup does not directly yield an equally large speedup for the entire training run. Other layers, communication, the optimizer, and data loading still consume time. Survey updates should record hardware, tensor shapes, precision, baseline versions, and whether measurements are kernel-level or end-to-end, rather than retaining only a speedup factor.

5. Serving Has Two Memory Bottlenecks: Storage per Token and Wasted Capacity

Autoregressive decoding repeatedly reads the KV states of previous tokens, giving it different resource characteristics from prefill, which can process input tokens in parallel. MQA lets different query heads share one set of keys and values, reducing cache and bandwidth requirements while changing the model's representational capacity. GQA directly extends this approach by grouping query heads and sharing KV within each group. It offers a compromise between standard multi-head attention and a single shared group, and also studies converting existing multi-head checkpoints through mean pooling followed by further training. Shazeer, 2019; Ainslie et al., 2023, §2-3

Ignoring buffers and other overheads, for batch size B, context length T, layer count ell, KV head count h_(KV), head dimension d_h, and bytes per element b, KV storage can be written as

$$M_{\mathrm{KV}} \approx 2BT\ell h_{\mathrm{KV}} d_h b.$$

This survey derives the expression from tensor shapes to illustrate the interactions among longer context, larger batches, fewer KV heads, and precision changes. GQA's conversion result using 5% additional training compute comes from its particular model settings; it does not guarantee lossless conversion of arbitrary checkpoints. MLA further changes this accounting through the different low-dimensional representation described above.

PagedAttention addresses an orthogonal problem. Even if the KV size per token is unchanged, reserving contiguous space for requests of different lengths creates fragmentation and unused capacity. It divides KV into fixed-size blocks, allocates them on demand, permits noncontiguous storage and prefix sharing, and uses vLLM's scheduling to increase the batch size that can be served. Its benefit is more effective use of space, not compression of each token's representation. Kwon et al., 2023, §1-4

MQA/GQA/MLA can therefore be combined with paged management: the former change representation, while the latter changes allocation. Both must nevertheless be evaluated against the workload. Long

requests, diverse generations, shared prefixes, low-latency single requests, and large offline batches benefit differently. Pope et al.'s inference analysis considers parallel layouts, batch size, context, and latency requirements together, studying input processing and token-by-token generation separately. The highest-throughput configuration may fail an interactive latency requirement, and the lowest-latency configuration may not minimize cost per token. Efficiently Scaling Transformer Inference, 2022

6. Context Scaling: Fitting the Window Does Not Mean Using the Information Well

Even after improving attention implementations, long context still requires compatible training distributions and positional mechanisms. Xiong et al. continued pretraining from Llama 2 checkpoints for approximately 400B tokens, adjusting positional encoding, long-sequence settings, and data mixtures. Smaller models used a longer training window, while larger models used another length to control costs. This provides empirical evidence for extending short-context models while also showing that enlarging the window is not a free configuration change. Effective Long-Context Scaling of Foundation Models, §2 and §4

In comparing data mixtures, the paper found that text quality and the training recipe could matter more than simply increasing the proportion of long documents. The survey should not compress these results into a claim that adding long documents is sufficient. Different tasks require retrieval, integration, or reasoning across passages and need separate evaluation. The bottleneck has shifted from whether memory can hold the sequence to whether training teaches the model to use the additional information effectively. Ibid., data and training-curriculum ablations

Mamba takes a different architectural branch. Instead of using attention to retain the full history for item-by-item retrieval, it selectively updates a state using input-dependent state-space parameters. Its basic form is h_t=bar(A)_t h_(t-1)+bar(B)_t x_t, y_t=C_t h_t; selection makes information retention content-dependent. A hardware-friendly scan addresses the computational problem created when input-dependent parameters invalidate the fixed convolutional form. The trade-off is that history is compressed into a finite state. Linear sequence complexity does not automatically guarantee fine-grained retrieval equivalent to attention. Gu & Dao, 2023/2024, §2-3 and §5

The original report studies sequence modeling across language, DNA, and audio. Results involving million-length sequences cannot be recast as million-token general-purpose language understanding without distinguishing modalities. **This survey's synthesis** is that long-context research should jointly measure at least length, information density, tasks assessing effective use, and execution cost. A model's maximum supported window is only one condition and cannot by itself represent the scale of its capabilities.

New Development: Allocating Computational Depth across Training Stages

Chen et al. (September 16, 2026; revised September 17) shift recurrence from inference budgets to pretraining allocation: increase core passes during training, with boundary operators normalizing states and reinjecting early representations. After architecture-specific tuning, Untied-Grow's loss-matched compute advantage over a standard model increases from 1.30× near 10^(18) FLOPs to 1.55× near 10^(20) on FineWeb; FineWeb-Edu shows a similar trend, with a held-out 7.4B run testing extension beyond the fitted range. This supports architecture-dependent fitted exponents, beyond constant shifts, within a limited range. However, fits share a baseline-estimated loss floor and batch size remains fixed. The roughly 20× GPT-3 comparison uses different data and evaluation pipelines, so it is not a controlled architectural gain; the distant forecast at 10^(25) FLOPs remains unverified. We follow v2's power-law wording, not exponential performance growth. Chen et al., v2, §3-4 and Appendix C

7. Deployment Objectives Ultimately Feed Back into Pretraining

Sardana et al. add inference demand to a Chinchilla-style objective. If a model will be used extensively, the serving savings of a smaller model can compensate for longer pretraining. At sufficiently high demand, a smaller model trained on more data can minimize total cost. However, curves fitted over conventional tokens-per-parameter ranges may also overestimate the benefit of additional data in the regime of extremely long training. Beyond Chinchilla-Optimal, 2023/2024

A simplified dense-model accounting expression illustrates this change:

$$C_{\text{life}} \approx 6ND_{\text{train}} + 2ND_{\text{infer}}.$$

It omits attention length, the prefill/decode distinction, memory, communication, and hardware utilization, and serves only to explain why the objective changes. Thus, overtraining in T1 and data curricula in T2 are not exceptions to scaling laws; they reallocate resources under new demand constraints. If post-training causes a model to generate longer reasoning for each request, the inference budget changes again.

Which Loss Should Be Predicted after Compression?

The same deployment objective opens another route: compress an existing model instead of choosing a new pretraining scale. Cheng et al., 2026, §§5-6 study teacher-forced reference-answer loss changes in mathematics, code and question answering after pruning, quantization and distillation. Sharing the pruning-density response halves configuration measurements; across the tested Pythia, OLMo-2 and Wanda settings, math and code prediction errors remain within 0.020 nats/token of full-grid regression. Each setting still refits coefficients: the form and measurement strategy transfer, not a ready-made numerical recipe. Including loading, reference measurements and calibration, the latter two settings use 59% and 70% of full-grid GPU time, not half the compute.

Finer prediction need not improve decisions: on the tested QA candidate sets, a fixed method priority matches numerical selection's regret, the loss gap to the best feasible candidate under the same storage budget. Generated exact match can also disagree with loss rankings, while nominal storage budgets do not measure deployment latency. The work primarily advances measurement efficiency and objective selection; it does not establish a universal compression-capability law across families.

The shared conclusion this survey draws from these lines of work is that systems innovation can make previously infeasible configurations feasible, architectural innovation can change the relationship between capacity and cost, and deployment demand can change what counts as optimal. The most valuable updates from subsequent papers identify which constraint was revised, which controls establish the net benefit, and where the next bottleneck appears. Reporting more total parameters, fewer active parameters, or higher throughput in one experiment is insufficient to establish that argument.

T4 Post-Training and Reinforcement Learning: From Feedback Objectives to Scalable Reasoning Policies

Verification date: 2026-10-12. Organized around problems, mechanisms, experimental conditions, and limitations; a logical connection does not automatically establish direct historical influence. Newly added literature is recorded in expansion-frontier.json, and earlier turning points and 2026 material in frontier.json. Reading depth is recorded for each source; this chapter does not claim exhaustive coverage of all papers available by this date.

1. The Starting Point Is Misaligned Objectives, Data, and Feedback, Not Insufficient Model Size

Pretraining scaling usually examines loss over a predictive distribution. Actual users care about whether an answer follows their intent, a proof is valid, or a program passes its tests. These objectives are correlated, but they are not the same objective. Post-training begins with the problem of using limited additional data and compute to reorganize the knowledge a base model already possesses into more appropriate behavior. Three budgets must be recorded together: generating training examples, judging their quality, and updating parameters. Reporting only the last hides the costs of search and teachers.

InstructGPT made this mismatch concrete: first use demonstrations for SFT, then train a reward model on human rankings, and finally optimize preferences with constrained PPO. Its finding that answers from a smaller model were preferred applies to the prompt distribution used in the paper's human evaluation; it does not mean that smaller models generally have stronger knowledge or mathematical capabilities. It shows that the training objective can change the relationship between parameter count and user utility, while leaving a bottleneck: rewards come from finite annotations, yet the policy actively seeks regions to which the reward function assigns high scores. InstructGPT

DPO addresses the complexity of this pipeline by reformulating a particular KL-regularized reward optimization problem as preference classification. Let pi_(ref) be the reference policy, and Delta_theta the difference between the preferred and dispreferred responses' log probabilities relative to that reference policy. Its core loss is

$$\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma(\beta \Delta_\theta).$$

This reparameterization reduces the need for a separate reward model and online RL, but it does not make preference data automatically cover every error the future policy might generate. Its theoretical correspondence depends on assumptions such as the reward-preference model; “simpler to implement” and “exploration is no longer needed for any task” are different conclusions. DPO, §4

When a task has an executable checker, feedback can shift from “which answer do people prefer?” to “does the answer satisfy the conditions?” Tulu 3 retains SFT and DPO, then applies RLVR to mathematics and instruction-constraint tasks. This illustrates complementarity, rather than the wholesale replacement of preference learning by mathematical rewards. It also finds that similar training rewards need not yield similar test performance, and that a stronger initial model is often advantageous. OLMo 2 connects this kind of post-training with public pretrained models, data, and checkpoints, making it easier for researchers to analyze which stage produced the gains. Tulu 3, §6, OLMo 2, §5

The base model's starting conditions therefore cannot be treated as background noise. Llemma continues training Code Llama on mathematical text and code; MetaMath expands existing mathematical problems through reformulation, backward questions, and related methods; DeepSeekMath connects mathematical corpus construction, continued pretraining, supervised fine-tuning, and RL in a single training chain. Together, these works remind us that the “zero” in subsequent “zero-supervision RL” usually means that a particular training stage uses no human reasoning traces. It does not mean that the model has received no large-scale training on knowledge, code, or synthetic data. Synthetic problems add forms of expression and approaches to solving problems, but this is not equivalent to adding the same number of independent new facts. Llemma, §2, MetaMath, §3, DeepSeekMath, §2-4

2. Reward Overoptimization: Why More Optimization Need Not Improve the Real Objective

Before verifiable reasoning became a central focus, RLHF had already exposed another bottleneck: a reward model (RM) learns an approximation to preferences, and continually increasing its score need not continually improve the objective. In 2022, Gao, Schulman, and Hilton turned this phenomenon into a measurable scaling problem. They used a fixed 6B “gold RM” to generate comparison labels, trained proxy RMs of different capacities, and then either updated a policy with PPO or selected candidates with best-of-N. Both routes exhibited regions where the proxy score kept rising while the gold score first rose and then fell. Failure can therefore occur outside the training stage as well. Original paper, §1-2 and §3.5.

The key advance was to examine **optimization strength and scorer capacity jointly**. The paper measures policy displacement by the square root of KL divergence and fits the following empirical relationships to gold reward relative to the initial policy:

$$d = \sqrt{D_{\mathrm{KL}}(\pi \| \pi_0)}.$$

$$R_{\mathrm{BoN}} = d(\alpha_{\mathrm{BoN}} - \beta_{\mathrm{BoN}} d).$$

$$R_{\mathrm{PPO}} = d(\alpha_{\mathrm{PPO}} - \beta_{\mathrm{PPO}} \log d).$$

The coefficients vary smoothly with RM parameter count, and larger RMs generally support higher peak gold rewards; more labels can also mitigate overoptimization. This reframes “is more optimization useful?” as “how much optimization can the current scorer withstand?” These are empirical expressions for a specific setting, however, and the PPO expression does not apply near the origin. KL measures distributional displacement; it is not a common FLOPs budget across algorithms. Differences in gains at the same KL also cannot be interpreted as differences in efficiency at the same actual compute budget.

Distributional change and the costs of training and sampling must be accounted for separately. §3.1-3.3 and §4.1.

The most important boundary is that **"gold" is defined as the reference only within the synthetic experiment. It is itself a proxy for human preferences, not actual human intent.** The study does not fully measure the second layer of mismatch between labeled preferences and real intent. The connection to verifier vulnerabilities in the next chapter is this review's synthesis across branches: both more PPO updates and more inference-time search concentrate pressure on errors in the scoring rule. Scaling compute must therefore be accompanied by checks for improvement in independent evaluations, rather than only increases in the optimized score. This also motivates subsequent attention to new feedback and scorer updates, instead of assuming that a fixed RM can be reused indefinitely. §4.3-4.5.

How Much Optimization Can Preference Data Support?

Aouad, El Gadarri, and Farias ask how preference data limits the optimization pressure a reward model can support. Under independent Gaussian response rewards, reward-independent comparison pairs, and a reward-blind reference policy, for fixed overlap and reward scale, with sufficiently many effective comparisons and a sufficiently large response space, their bounds give attainable reward gains of order

$$\sqrt{\min\{\log M, K\}}.$$

Here M counts comparisons and K bounds expected KL divergence, not FLOPs. The lower bound uses a constructed comparison set, proxy, and reference policy; it does not guarantee that ordinary reward-model training attains the bound or prove that every policy deteriorates beyond it.

Experiments use a 70B gold reward model and 0.6B-4B Qwen3 proxies. Increasing M from 600 to 300,000 adds prompts while retaining 300 comparisons per prompt. Evaluation uses 128 disjoint prompts with 8,192 responses each. The principal fit concerns the gold-reward peak selected over an optimization grid as a function of M, using gold scores rather than deployable gold-free tuning or a validated two-dimensional forecast for arbitrary (M,K). Exponential tilting reweights fixed response pools; it is not an actual policy-training experiment. This connects feedback supply to optimization limits, while leaving human preferences, adaptive data collection, and compute-optimal allocation unresolved. Original paper, §§3-6 and Appendix B

Does a More Accurate Static Judge Make a Better Online Rewarder?

Scaling a rewarder also requires deciding what it should predict. Raychev et al. compare Qwen3 generative judges of 0.6B-14B in creative-writing online DPO. Thinking judges trained with GRPO score higher on fixed Gold labels, yet Answer-Only SFT judges produce higher peak and top-five-average policy Elo in seven of eight matched configurations; five Elo gaps have conditional confidence intervals wholly above zero. Intermediate Thinking checkpoints can also be better rewarders than later, more accurate judges. This extends the overoptimization problem to rewarder selection: static agreement need not predict the usefulness of feedback on an evolving policy.

The comparison changes both format and training objective; it does not identify reasoning as the cause. Gold is a learned 32B model trained on Reddit vote-derived preferences, and a different learned judge's 80-comparison follow-up is inconclusive overall. Peaks are selected retrospectively by Gold; the

bootstrap does not cover retraining uncertainty. Parameter sweeps and in-sample fits are neither compute-matched allocation rules nor universal scaling laws. Complete per-run token-cap provenance is also unavailable, so the evidence supports evaluating rewarders inside the optimization loop, not claiming superior human writing quality or lower total cost.

3. Key Turning Point One: Why Can Self-Training Improve, and Why Can It Run Out of Its Own Answers?

When only a few reasoning demonstrations are available, directly expanding human annotation is expensive. STaR's insight is to turn the paths on which a model occasionally answers correctly into new training data: generate reasoning and answers, retain successful examples, then fine-tune. For failed problems, the paper also tries giving the model the correct answer so that it can construct a rationale backward, then removing the answer hint during training. In the original method, each round fine-tunes again from the original pretrained model to reduce overfitting from accumulated training. It does not simply continue training the same checkpoint indefinitely. STaR, §3-4

A common objective illustrates the mechanism. Let x be the problem, y the complete response, R(x,y) the nonnegative reward returned by a checker, and pi_t the current policy. If we retain only high-reward responses, the training distribution changes from pi_t to

$$q_{t+1}(y \mid x) = \frac{R(x,y)\pi_t(y \mid x)}{\mathbb{E}_{y' \sim \pi_t}[R(x,y')]}.$$

Subsequently maximizing E_(q_(t+1))log pi_theta(y | x) moves more probability mass onto good solutions already found. ReST-EM organizes this family of methods through an expectation-maximization perspective: binary rewards correspond to supervised learning after rejection sampling, and the paper studies the roles of model size, generation volume, and iteration in mathematics and code tasks. ReST-EM, §3-5

The formula also directly exposes a boundary. If every reward in the current sampled batch for a problem is zero, the empirical estimate of the normalizing denominator is zero, and that batch provides no positive examples. This does not mean that the true expectation in the formula is zero. If the checker accepts examples with incorrect reasoning but a correct answer, the incorrect steps will also be reinforced. Answer hints, teacher models, tools, and search can all introduce new information into the process; gains under these conditions cannot be attributed to a model creating knowledge from nothing. Conversely, failing to find an answer with finite sampling does not prove that the base model's probability of answering the problem correctly is strictly zero. These are two distinct issues in the debate over the boundaries of a model's support.

ReST's Grow-Improve procedure addresses another cost problem: first generate data, then make multiple offline improvements using the same batch, rather than resampling after every parameter update. Its original experimental evidence comes mainly from machine translation and learned quality feedback; ReST-EM connects verifiable problem-solving to explicit reward reweighting. The former cannot simply be treated as an equivalent experiment with binary mathematical rewards. What the two share is the design principle of making fuller use of expensive generated examples. ReST, ReST-EM

The next steps in this branch naturally diverge: increase sampling to expand positive-example coverage; use search to find better paths through intermediate states; use teachers to supply trajectories that the current policy struggles to discover; or change the RL update so that different steps in a failed example receive different credit. Identifying which of these produced an improvement is more explanatory than calling all of them “self-evolution.” STaR itself also obtains different gains across datasets, so it does not establish a law under which every self-training round necessarily improves performance at a fixed rate. STaR, §4-5

4. Key Turning Point Two: Does a Process Reward Predict Correctness or Future Success?

Final-answer feedback is cheap, but it tells us only whether the whole trajectory passes. Cobbe et al.'s verifier work separates generation from selection: first produce multiple answers, then train a model to identify a correct solution. On GSM8K, this makes additional sampling usable. The paper also distinguishes single-generation performance from coverage under repeated sampling, showing that overtraining the generator can harm sample diversity. The resulting question is: if we generate ten answers and one is correct, can we reliably find it? Training Verifiers, §4

Checking only outcomes can also accept correct answers containing incorrect steps. Uesato et al. measure both answer errors and reasoning-process errors and compare outcome with process feedback. Their results reveal differences among annotation cost, final accuracy, and process reliability in particular GSM8K experiments; they do not prove that process supervision is superior at every budget. Let's Verify Step by Step subsequently became an important turning point in the human process-labeling branch. Annotating every step is expensive, however, motivating automatic process supervision. Uesato et al., §2-4

Math-Shepherd's approach is to sample continuations from each prefix. For a prefix s, a common soft label can be written as

$$\widehat{V}^{\pi}(s) = \frac{1}{M} \sum_{j=1}^{M} \mathbf{1}\{\text{correct}(y_j)\}.$$

Here y_j denotes a response continued from prefix s under the current policy, with j indexing the sample. A hard label can instead indicate “at least one success.” This allows training a scorer for reranking or RL without step-by-step human annotation. Crucially, the expression estimates **the probability of completing the task from this point, given the subsequent policy**. It is not directly the probability that “this step is logically correct.” Math-Shepherd, §3

Consider two illustrative prefixes. Prefix A makes a correct but difficult algebraic transformation, after which the continuation model often fails. Prefix B first makes a mistake, then corrects itself and reaches the right answer. Scoring by subsequent success rate may rank B above A; scoring by strict step correctness should reverse the order. This is not a paradox in which a scorer “suddenly stops working”: the two objectives are different to begin with. Recoverability is valuable when guiding search, but can be misleading when explaining where a proof first went wrong. A review must distinguish a value model, an error localizer, and a final-answer verifier at the level of their objectives.

The Lessons of Developing Process Reward Models subjects this distinction to empirical checks. It compares Monte Carlo continuation labels, model judgments, and human supervision, showing that continuation success and step correctness are not interchangeable. It also cautions that evaluating a PRM only through best-of-N answer accuracy can make an outcome-oriented scorer look particularly strong. The paper combines consensus filtering with observations of both selection tasks and step-error identification. This supports the claim that label construction and evaluation objectives should match; it does not imply that all Monte Carlo labels are useless. PRM Lessons, §2-4

This distinction explains why a single kind of PRM did not come to dominate every use case. Some models primarily estimate search value, some locate the first error, some score complete proofs, and others expand the generative checking process into a reasoning chain of its own. Their labeling costs, deployment latency, and applicable tasks differ. Ranking them all by a single “reward model accuracy” obscures the interface conditions that determine their practical effects.

5. From External Search to Learning to Use a Budget: o1 and R1

OpenAI's official o1 report provides an important empirical signal: increasing reinforcement-learning training compute and increasing test-time thinking compute both improve the reasoning performance shown. This expands the question from “how should we search?” to “how should we train a policy that makes good use of additional compute?” The report does not disclose a training recipe sufficient for full reproduction or a unified fitted formula, however, and smooth growth in the figures cannot be treated as a validated law for unlimited extrapolation. Learning to reason with LLMs, 2024.

DeepSeek-R1 makes this direction more open to inspection. R1-Zero applies RL directly to the already pretrained DeepSeek-V3-Base, using verifiable feedback such as mathematical answers and code tests, together with format rewards. GRPO estimates advantages from within-group returns, reducing the need for a separate critic. Behaviors such as long reasoning and self-checking emerge during training, but readability problems and language mixing motivate R1's cold-start data and multistage SFT/RL recipe. “Zero” here does not mean learning from zero knowledge, and the training conditions of R1-Zero must not be conflated with those of the final R1. DeepSeek-R1, §2.

Automatic verification reduces the dependence of feedback scaling on individually annotated examples, while introducing a structural boundary. Checkable final answers are relatively well suited to mathematics and code; open research, long-horizon actions, and open-ended writing may lack rewards that are reliable, cheap, and difficult to exploit. This review therefore concludes that scaling post-training requires, at a minimum, usable tasks, verifiable feedback, and a policy that produces useful exploration together. RL step count alone cannot describe it.

6. Key Turning Point Three: Where Do GRPO's Savings Come From, and Where Is Optimization Bias Hidden?

Long reasoning imposes memory, value-estimation, and credit-assignment costs on PPO. DeepSeekMath's GRPO uses relative rewards among multiple answers to the same question as a baseline in place of a separate critic. For a group of G responses, a simplified outcome advantage is

$$A_i = \frac{R_i - \bar{R}}{\mathrm{std}(R_1, \ldots, R_G) + \varepsilon}.$$

The original formulation averages the clipped token surrogate by length within each sequence, then averages across the group, with a reference-policy constraint. Removing the value model does not remove sampling: obtaining a reliable relative signal still requires multiple rollouts, and the training-data distribution also determines which groups have nonzero signals. DeepSeekMath, §4

Long sequences magnify the effects of seemingly harmless normalization in this objective. Dr. GRPO points out that if every token gradient in a response is multiplied by 1/T_i, each token in a short positive example receives a stronger push, while each token in a long negative example receives a weaker penalty. The within-group reward standard deviation also changes the relative weights of different problems. The paper replaces each response's own length with a fixed length scale and removes this reward-standard-deviation normalization, studying how these choices affect training. “Debiasing” here concerns a particular objective and normalization convention; it does not mean that every policy-gradient estimate thereby becomes unbiased, stable, or universally optimal. Understanding R1-Zero-Like Training, §3

An objective-level comparison makes the scaling implication clearer. Extending the same kind of negative trajectory from two thousand to eight thousand tokens reduces the penalty per token under a sequence-averaged objective. If those additional tokens are repetitive loops, training may not discourage them as strongly as we expect. Observing that length and reward increase together does not immediately establish that longer thinking is a necessary cause of reasoning improvement. Length weighting, truncation, and sample filtering must be examined. This is an audit question derived from the objective, not a definitive claim about the behavior of every long-reasoning model.

DAPO addresses related issues through a large-scale training recipe: raise the upper clipping bound to preserve room for exploration; use dynamic sampling to filter problems with all-correct or all-wrong groups that lack a within-group relative signal; aggregate the loss globally at token level; and handle overlong truncation to reduce the noise from treating unfinished responses directly as semantic errors. These four designs target different failure mechanisms. Rollouts discarded by dynamic sampling still consume compute, and global token normalization differs from Dr. GRPO's fixed denominator. They must not be described as the same modification. DAPO, §3

The shared value of the two approaches is to bring “can RL keep scaling?” back from algorithm names to inspectable statistical questions. What fraction of all generated examples provides a useful training signal? How much weight do long responses actually receive in the objective? Is the training set increasingly reduced to moderately difficult problems? Is exploration lost because probability concentrates too early? These quantities determine whether doubling the rollout budget produces nearly twice as much useful data. Comparisons of GPU hours or total FLOPs must also include filtering, failures, and resampling.

7. Why Do Sequence Objectives, Critics, and Initial Models Still Matter?

GRPO's group baseline did not end objective design. GSPO changes the importance weight to a length-normalized form of the whole-sequence probability ratio:

$$s_i = \exp\left(\frac{1}{T_i}\sum_t \log \frac{\pi_\theta(y_{it} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x, y_{i,<t})}\right),$$

then applies clipping at sequence level. The authors specifically study MoE training stability: routing changes can make local token ratios highly unstable, while sequence aggregation offers another means of control. Note that s_i is a length-normalized ratio, not an ordinary joint probability-density ratio for the whole sequence. Empirical stability also does not imply that all token-level importance sampling is theoretically invalid. GSPO, §4-5

Another branch returns to training the critic carefully. Open-Reasoner-Zero uses a simplified PPO recipe, rule-based rewards, and particular GAE settings, reporting scalable training. VAPO focuses on value-estimation bias and sparse terminal rewards in long reasoning, using value pretraining and length-dependent credit-assignment designs. They show that removing the critic first is an engineering choice, not proof that critics have no lasting value. Nor are the two methods the same recipe, and fewer training steps cannot be directly interpreted as a proportional reduction in FLOPs. ORZ, §2-3, VAPO, §3-4

Group-relative RL can save a critic yet retain a synchronization cost: in asynchronous training, early completions wait for the rest of their prompt group. Li²SAO addresses the opposite problem of single-rollout learning-a critic that is initially too inaccurate to guide updates. It first trains the actor with group-relative advantages and the critic with centered returns plus outcome ranking, then switches to single-rollout GAE.

In its Qwen3-4B mathematical setting, 500 updates reach mean AIME pass@1 of 67.0%, versus 63.6% for GRPO. On eight A800 GPUs, this run takes 28.9 hours versus 36.4 for GRPO with six rollout GPUs; the new method shares four trainer GPUs between actor and critic. All collect 128 trajectories per update, but response lengths and prompts per batch change. These are configuration-specific wall-clock observations, not equal-FLOP comparisons or a universal scaling law. Evaluation samples sixteen answers per problem; independent training-seed uncertainty is not established. The theory’s exact-value identities also do not guarantee convergence or unbiasedness of the full asynchronous implementation.

A simple order-of-magnitude example illustrates the difficulty of credit assignment: if a terminal signal is repeatedly multiplied by 0.95 as it propagates to earlier positions, only about 0.006 remains after one hundred steps. Actual GAE is more complex than this illustration, but it explains why early decisions in long responses may struggle to receive effective feedback. Shortening trajectories, estimating process values, changing advantage propagation, and adding intermediate rewards address the same problem at different points. Denser intermediate rewards also create more opportunities for misdefinition and exploitation.

SimpleRL-Zoo compares multiple open base models and training choices, finding that the effects of format rewards, prompt templates, and data difficulty depend on the initial model. This means that the appearance of reflective words in a model's output is insufficient evidence that training created a previously absent capability: comparisons must use the same base model, sampling budget, and prompting conditions. ORZ's data ablations also remind us that more training steps and greater problem diversity are not interchangeable resources; a narrow data distribution may reach a bottleneck first. SimpleRL-Zoo, §2, ORZ, §3

Experimental tables for RL scaling should therefore report at least the base-model version, distribution of training problems, samples per problem, total rollout tokens, update tokens, retention fraction, and test decoding settings. If a model upgrade, a reward-parser change, and a longer context occur together, a

steeper training curve is insufficient to attribute the gain to the algorithm itself. The preceding methods provide local evidence from several scalable training systems; this is not yet enough to combine them into a fixed power-law exponent across base models, tasks, and reward interfaces.

8. When Support Is Insufficient: What Do Teacher Trajectories and Self-Reward Each Add?

If the current policy struggles to sample successful trajectories, changing the gradient formulation alone may be insufficient. LUFFY places off-policy reasoning from a stronger teacher and on-policy samples from the current policy in the same training framework. It adjusts policy shaping so that key teacher actions do not produce almost no effective learning signal merely because their current probability is too low. The question it answers is “how can external successful trajectories effectively participate in exploration and updates?” Teacher generation cost, teacher capability, and sample selection are therefore part of the method. These gains cannot establish that RL without external information can cross the same boundary. Learning to Reason under Off-Policy Guidance, §3

If the main difficulty is the cost of preference annotation, a separate self-rewarding branch emerges. Self-Rewarding Language Models have the model generate candidates, score them using evaluation prompts, and then construct preference pairs for iterative training; the process still starts with human demonstrations and evaluation seeds. CREAM addresses self-scoring noise through regularization based on preference consistency across model iterations. The latter can suppress unstable labels, but it cannot identify every stable shared error: if two stages endorse the same wrong answer, agreement does not make that answer true. Self-Rewarding LMs, §2, CREAM, §3

Three kinds of feedback should therefore be distinguished: checkable feedback from executors or formal systems, learned scorers trained on external labels, and preferences produced by model self-evaluation. All can support training, but their error correlations differ. In particular, when the generator and scorer improve together, rising training reward may reflect better answers or simply increasing agreement between the two. The next stage requires more independent held-out checks, error-type analysis, and tests under distribution shift.

9. Verifiers Need Compute Too: From Assigning Scores to Checking Reasoning

Binary or stepwise scorers often compress a complex proof into a very small output. If checking itself requires multistep reasoning, a verifier can reason before judging. ThinkPRM supervises models with generative verification traces and studies the benefits of additional verification compute. Its low-label results depend on checking processes generated by a strong teacher. The method includes filtering verification traces using existing step labels; it does not acquire verification ability from scratch using only a few bare labels. Its reported MATH-500 selection and search experiments use a sampled subset of 100 problems (§4.1 and Appendix E.5), rather than the full 500-problem benchmark. Process Reward Models That Think, §3 and Appendix E

DeepSeekMath-V2 further connects complete proofs, self-analysis, a generative verifier, and a meta-verifier that checks whether critiques are valid, then uses these signals to train generators and checkers. It addresses more than final numerical answers that can be directly compared as strings. Its

high-compute evaluation also includes multiple candidates, repeated verification, and iterative refinement. The corresponding results belong to the budget of the entire system, rather than an ordinary single response. Adding another verifier does not automatically provide a guarantee of formal correctness; ultimate reliability still depends on labels, checking procedures, and independent review. DeepSeekMath-V2, §2-3

This forms the most important connection between post-training and inference budgets: a stronger verifier can improve the current selection and produce better training examples; a better generator in turn changes the error distribution that the verifier faces. When both scale together, one should therefore be frozen for ablations, with cross-evaluation on old and new generation distributions. Evaluating only the current generator paired with the current verifier cannot reveal whether the verifier has actually learned more general judgment.

10. Does RL Change the Range of Solvable Problems or the Sampling Probabilities?

The gains from o1/R1 raise a question that pass@1 alone cannot answer: does performance improve because of new capabilities, or because existing solutions become easier to sample? Yue et al. compare the full pass@k curves of base and RL-trained models. In their tested mathematics, code, and visual-reasoning settings, RL models lead at small k, but base models may cover more solvable problems at large k. Perplexity and coverage analyses support the interpretation that current recipes primarily redistribute the probabilities of existing trajectories. Does RL Really Incentivize…, v5, §3-5.

What this result directly challenges is the inference that improved pass@1 proves an expansion of capability boundaries. Failure at finite k cannot prove that the probability of a correct solution in the distribution is strictly zero. Moreover, pass@k uses correctness judgments available during evaluation and cannot be directly equated with successfully selecting an answer in deployment. “RL can never produce new capabilities” is therefore much stronger than the experiments support. Holding sampling temperature, length, prompts, and checking rules constant is also necessary for comparison.

In 2026, Curriculum RL proposes a mechanism-level remedy: if every rollout in a group fails on a difficult problem, within-group normalized rewards provide no differentiating learning signal. It first uses sampling to locate problems near the boundary, then provides a small amount of targeted teacher guidance, and finally consolidates the gains with RL. The paper reports improvements in both pass@1 and pass@256. This is evidence that “teacher guidance + curriculum + RL” expands the empirically solvable set. **It cannot establish that pure RL without external information has been proved to exceed the base model's ceiling.** Curriculum RL, §3-5, 2026-06.

Meanwhile, post-training itself is beginning to receive more systematic scaling measurements. Tan et al. conduct 54 experiments on Qwen2.5 models from 0.5B to 14B, finding that larger base models are more effective under a fixed post-training budget in the tested conditions, and that repeated use of high-quality problems can also work. However, “test loss” here is defined as normalized error rate, not language-modeling cross-entropy, and the pretraining investment inherited by larger base models is not matched. This provides a conditional relationship for a particular RL recipe, not an “RL Chinchilla constant.” Scaling Behaviors of LLM RL Post-Training, §2-3, 2025-09.

This debate depends on both the definition of capability and the measurement protocol; see Evaluation and the Limits of Extrapolation. A gap remains between failing to discover a solution through sampling and a correct solution having strictly zero probability. Finite experiments cannot eliminate that gap.

11. Connections to Other Chapters

- **Pretraining/data → post-training**: the base model's probability of correct trajectories and its knowledge coverage determine whether self-training has positive examples and how difficult RL exploration will be; relevant works include Llemma, DeepSeekMath, and SimpleRL-Zoo.
- **Post-training → inference budgets**: training changes single-sample success rates and candidate correlations, thereby changing the sampling scaling curve; an old curve cannot be held fixed to extrapolate a new policy.
- **Inference budgets → post-training**: search, tools, and verifiers turn additional compute into reusable trajectories; relevant works include ReST, ReST-MCTS*, and rStar-Math.
- **Evaluation → both stages**: step correctness, recoverability, final answers, and preferences are different metrics; relevant works include Uesato, Math-Shepherd, and PRM Lessons.
- **Architecture → inference budgets**: depth with shared weights increases per-problem compute while also changing latency and training stability; relevant works include Universal Transformers, Coconut, and recurrent-depth models.

These connections reconstruct the literature around problems. Except where papers explicitly discuss an inheritance relationship, the arrows should not be interpreted as established direct historical influence among authors.

The next chapter turns to Inference Budgets and Search: post-training changes the candidate distribution, but the system must still decide how often to generate, search, and check for each problem. Updates to this chapter should prioritize controlled scaling experiments with matched base models, data, and complete cost accounting. A new model's high score alone cannot identify which training component produced the gain.

T5 Inference Budgets and Search: Turning More Compute into More Reliable Answers

> Verification date: 2026-10-19. Covers representative works that help explain mechanisms; much of the 2026 material remains in preprint form. Experiments have not been independently reproduced. Sources, versions, and reading scope are recorded in frontier.json and expansion-frontier.json; the Hugging Face authors' blog is recorded in the editorial primary-source collection.

Pretraining invests compute in shared parameters, whereas inference allocates compute to a particular problem. Chain-of-Thought changes a single generation process through demonstrations of intermediate steps; Self-Consistency reduces the instability of a single path through multiple-path sampling and answer aggregation. They open serial and parallel directions, respectively, but leave cost and correct selection unresolved. CoT, Self-Consistency

This chapter also treats search over unfinished prefixes separately, because it can reallocate the budget midway through a response. A 2026 evaluation framework treats the model, prompt, decoder, external evidence, selector, and stopping rule together as the system under evaluation. Many curves below therefore describe empirical relationships between a particular system's accuracy and its budget; they do not automatically constitute universal power laws across models. Hariri et al., §2-3

1. From Sample Count to Success Rate: A Verifier Separates Coverage from Successful Selection

The most direct form of inference scaling is to generate more times. Large Language Monkeys studies repeated sampling, finding that coverage continues to grow with sample count on some tasks and fitting empirical curves. But an oracle knows which samples are correct, while majority voting or reward models in deployment do not. The paper observes that practical selection methods can saturate before coverage does. Thus, an improvement in “pass@k” answers only a question of existence, not whether the answer ultimately delivered to the user is more reliable. Large Language Monkeys, §3-4

Three different metrics are needed here. Coverage is the fraction of problems for which at least one correct sample has been generated; selection accuracy is the fraction for which the selector delivers a correct answer. The final per-problem success rate must also be constrained by the real budget, including all candidates, all verification, latency, and tool calls. A method can substantially improve the first metric without improving the second, or improve the second while being a poor tradeoff on the third. A plot whose horizontal axis is only sample count erases cost differences caused by response length and verifier size.

How Do Large Language Monkeys Get Their Power (Laws)? offers an important explanation. For a fixed problem, if the success probability of each independent sample is p, the failure probability is (1-p)^k, not a power law. Averaging over problems of different difficulties gives

$$F(k) = \int_0^1 (1-p)^k f(p)\,dp.$$

When the success-probability density in the difficult-problem region is approximately f(p) proportional to p^(alpha-1), then for large k the integral can decay as k^(-alpha). An aggregate power law can arise from mixing problem difficulties; it does not require the computation for each individual problem to follow a power law. How Do Large Language Monkeys Get Their Power (Laws)?, §2-3

This explanation yields two testable implications. First, changing the problem set changes the distribution of difficult problems and may therefore change the fitted exponent. Second, under the independent-sampling model, if p is truly zero for some fraction of problems, a failure floor remains that even unlimited sampling cannot eliminate. The second implication follows under the formula's assumptions; finite experiments do not establish that a real problem “will never be answered correctly by the model.” Changes to temperature, prompts, tools, or post-training also change the original p and correlation structure, so the old curve no longer has the same claim to extrapolation.

From Independent Sampling to Coordinated Coverage

Independent draws can revisit the same unsuccessful strategy, making branch coordination a separate question from increasing sample count. PTTS addresses this with a planner that jointly generates four outlines and a frozen executor that completes each outline; groups are repeated to collect 64 solutions. Its RL variant rewards the planner when any branch succeeds, using shorter executor rollouts during training: 4k tokens versus a 10k-token evaluation cap per solution. Across five mathematics benchmarks with Qwen3-1.7B/4B executors, the reported pass@64 gain over repeated sampling reaches 13.4 percentage points for the 1.7B model. PTTS, v1, §§3-4

This extends the coverage branch, not evidence that a deployment selector chooses better answers. The study generates 64 solutions and analytically estimates smaller-k coverage from that pool; coordinated groups also change the samples' dependence structure. Consequently, the reported pass@32 advantage over repeated sampling's pass@64 should not be read as a directly measured halving of end-to-end compute. A deployment comparison must count planning, executor generation, selection and amortized planner training, then measure latency or FLOPs. Likewise, theoretical inclusion of repeated sampling concerns the best policy achievable within the policy class; it does not guarantee that every trained planner improves performance.

2. From Repeating Whole Answers to Searching Intermediate States

Independent sampling repeats many unproductive prefixes. If an intermediate step can be identified as wrong, generation can stop early; if a prefix appears more promising, more budget can be concentrated there. Tree of Thoughts separates thought units, candidate generation, state evaluation, and search strategy, applying tree search to tasks such as Game of 24, writing, and crosswords. This changes "think a few more times" into "decide where to think from next." Gains depend on usable task decomposition and state evaluation; not every long text naturally provides a reliable search tree. Tree of Thoughts, §3

LATS places reasoning and action within the same search process, combining environmental feedback, MCTS, and reflection. An external environment can supply information missing from model self-evaluation, but also changes the cost structure: search nodes may involve real tool calls, and rollback or retries may not be cheap. Search gains obtained in resettable experimental environments require fresh examination before extending them to irreversible actions. This last point is an inference about deployment conditions, not a safety guarantee that the paper provides for every real system. LATS, §4

Search with small models also faces the difficulty that both their own generation and their own judgment are weak. rStar builds search from multiple reasoning actions and brings another model into mutual verification, attempting to reduce dependence on the generator's self-evaluation alone. However, agreement between two models is not statistically independent evidence: shared training data and similar reasoning habits can still produce correlated errors. Effectiveness must be checked through actual accuracy; it cannot be guaranteed by the definition of agreement itself. rStar, §3

When search trajectories can train the model in turn, extra compute on one problem no longer serves only that response. ReST-MCTS* jointly updates a policy and a process value model, constructing supervision from searched paths and information such as distance to success. rStar-Math further combines code-augmented reasoning, a process preference model, and multiple rounds of self-training. These systems amortize search costs over many subsequent inference requests, but a small parameter

count does not imply low total cost: large-scale trajectory generation, evaluation, and retraining all belong in the accounting. The models also start from existing pretrained weights, so “self-training” must not be described as acquiring mathematical knowledge from scratch. ReST-MCTS, §3, rStar-Math, §3

Strong feedback interfaces further change the available strategies. DeepSeek-Prover-V1.5 uses Lean checking results for training and, during search, retains accepted proof prefixes and continues generation from valid states. This partly transfers error localization from learned approximate judgments to a formal tool. But a proof checker verifies derivations within a formal statement; it cannot automatically confirm that a natural-language problem has been fully and accurately formalized. The scaling capability here comes from the combination of model, search, and verification environment, not just longer natural-language chains. DeepSeek-Prover-V1.5, §2-3

3. Under the Same Budget, Should We Sample, Search, or Revise?

The next question is: **should the same budget be spent on more independent candidates, deeper search, or sequential revision?** Snell et al. organize methods around improving the proposer or using a verifier, selecting strategies according to difficulty relative to the model. In their PaLM-2 and MATH setting, appropriate budget allocation can achieve roughly four times the efficiency of a particular best-of-N baseline; under some conditions, a small model with additional inference compute can outperform a model with about fourteen times as many parameters. The limitations are equally important: gains on the hardest problems are small; stronger search can exploit verifier weaknesses; and the cost of difficulty estimation is excluded from the main comparisons. The paper therefore supports conditional substitution between budgets, not the general claim that “small models can replace large models simply by thinking longer.” Snell et al., §3, §5, and §7.

Wu et al. approach the issue from another direction, comparing the FLOPs-accuracy frontier across model size, voting, verification, and tree search, and proposing Rebase to allocate search budgets. Their analysis shows that voting can saturate because it is constrained by the answer distribution: a frequent wrong answer does not automatically become correct as sampling increases. The optimal combination of “scale the model” and “improve the inference algorithm” therefore changes with the budget. On the mathematical tasks tested, a smaller model with appropriate search can be more cost-effective. Wu et al., §1 and §3-4.

The Hugging Face authors' blog connects this question to reproducible experiments. On MATH-500, it pairs 1B/3B proposers with an 8B PRM, comparing candidate budgets from 1 to 256 across five random seeds. DVTS preserves search diversity through multiple independent subtrees, reducing the risk that one path consumes the budget too early. This provides an engineering bridge from research strategies to implementations with open models and tools. Candidate budgets in the comparison are not matched end-to-end FLOPs, and the generators and verifiers have different parameter counts. Thus, “a 3B system outperforms a 70B model with a single response” does not directly mean lower total compute cost. Parameter count, GPU memory use, throughput, and cumulative inference cost need to be reported separately. Hugging Face authors' blog

Execution schedule adds another budget dimension. On 100 GSM8K prompts, repeated three times, the September 2026 study Sample Count Is Not Enough tests Phi-3-mini-4k-instruct and Qwen2.5-1.5B-Instruct on A100-SXM4 80GB GPUs (temperature 1.0, top-p 0.95, maximum 512

generated tokens per candidate). Holding eight candidates fixed, eight sequential single-candidate calls consume 4.64-4.86× the gross GPU-device energy and incur 5.77-6.12× the P95 latency of one eight-candidate batch. Generated-token volumes differ by at most 1%; schedules sample independently. Energy includes idle consumption during measurement but excludes whole-node energy. This strengthens the budget-accounting requirement: report call structure and measured systems costs alongside candidate count. These are conditional Hugging Face batch-generation results, not a universal scaling exponent or a FLOPs-equivalent comparison; dependent search steps and continuous serving need separate evaluation. Sample Count Is Not Enough, v2, §III-VII

4. Where Should the Verification Budget Go?

Search makes verifier errors more consequential: an early false negative can prune an entire correct branch, while an overvalued prefix can absorb a large share of the budget. This differs from reranking completed answers once. A scorer that predicts recoverability is suitable for judging the value of further exploration; locating the first logical error requires different labels. The distinction between Math-Shepherd and PRM Lessons now becomes a compute-allocation problem rather than a terminological dispute.

The total inference budget can be conceptually decomposed as

$$C_{\text{total}} = C_{\text{proposal}} + C_{\text{verification}} + C_{\text{environment}} + C_{\text{coordination}}.$$

This is an accounting identity, not an already fitted scaling law. More verification compute may allow better use of a weaker generator's candidates, or its benefits may saturate quickly because scoring and generation share errors. Spending the entire budget on verification also encounters insufficient candidate coverage: if every answer is wrong, even a perfect selector cannot deliver a correct one.

System comparisons should therefore ideally show three curves together: scale verification compute with a fixed generator; scale generation compute with a fixed verifier; and jointly allocate the two under the same total cost. PRM label volume, verification-model parameters, checking tokens per candidate, and candidate count are different axes. None should be singled out as if it alone constituted “test compute.” ThinkPRM and DeepSeekMath-V2 make this dimension more explicit, but their results in particular mathematical settings do not yet establish a single optimal ratio for open-domain fact verification.

Why Can Submission Fail Even When a Correct Candidate Exists?

Beyond verification cost, a selection protocol can discard usable outputs. SELF-POT (Yang et al., 2026), §3.3 and Appendix J.2 replays selection on fixed programming pools: five models each face 100 tasks, giving 500 scheduled cells; 498 pools are retained and two missing records remain failures. Falling back to the first nonempty candidate when judging returns no valid nonempty answer increases correct submissions from 376 to 437, retaining all spent judge costs. This output-aware fallback is a post-hoc amendment, not a preregistered intervention.

Public-example selection reaches 453/500 while removing judge-stage logical API charges; hidden correctness labels score outcomes but never guide selection. Coverage, selection accuracy and successful submission therefore also depend on protocol reliability. However, this is a retrospective replay on fixed candidates, not freshly generated, matched-budget end-to-end trials. Logical API charges

are neither invoices nor hardware costs; local test execution is accounted for separately. The result supports explicit failure recovery, not a general claim that more test-time compute is cost-effective on every task.

Harness Search: Same-Task Feedback Does Not Establish Transfer

Once extra inference also rewrites an agent's prompts, tools, or memory, the question is whether it improves a reusable harness or merely adapts to the evaluated tasks. Wang et al. separate these objectives in their October revision. On Terminal-Bench 2.1, they use three models, two runs and a 45/10/34 train/validation/test split. After K=5 search rounds and validation-based harness selection, mean test pass@1 is 63.2 for both initial and selected harnesses. Search-method comparisons match rollout budgets, not total FLOPs: harness-editing cost is excluded. The negative result is conditional. Task-specific adaptation improves game scores, but ARC-AGI-3 caps environment actions, excluding internal reasoning, whereas EdgeBench uses a 12-hour window that includes summarization and editing. EdgeBench also reports the best score observed, including host-side evaluations hidden from the agent; this is not blind selection of a final submission. The implication is to report feedback access, search cost, held-out transfer, and selection protocol separately. Neither task-specific gains nor this limited negative result establish a general law for reusable agent improvement.

Scaling Harnesses During Training: Learning Transfer through Structured Variation

A separate response to harness specialization changes training rather than searching the harness at test time. Bai et al. factor execution into five components with three variants each, select 18 of 243 configurations using pairwise-balanced design, and expose each task to two nearby harnesses over successive RL updates. With Qwen3-4B and 30B-A3B, PPO and GRPO, they evaluate disjoint tasks under 18 held-out combinations and external harnesses. For 4B GRPO, mean pass@1 across three benchmarks in unseen Claude Code reaches 37.22%, versus 33.38% for RL trained through OpenClaw; scores average three evaluation runs, not three independent trainings.

The main RL comparison matches 300 updates and 128 rollouts per update, not measured total FLOPs or wall-clock cost. Structured 18-harness training also beats exhaustive 243-harness training at this budget; the latter includes the white-box test configurations and is not held-out generalization. Thus diversity organization, not simply harness count, matters. This supplies conditional training-time transfer evidence, not a refutation of the earlier test-time harness-search result or a universal scaling exponent.

Environment Composition: Which Training Trajectories Should Scale?

Execution-framework diversity leaves another allocation question: which environments should supply the training trajectories? SynthAPI controls apps A, instructions per app T, profiles per instruction S, and database depth D (data batches including the core batch), with N=ATS tasks. It trains 849 Qwen3.5 students across four sizes using SFT, not RL. A composition-dependent logistic fit predicts normalized diagnostic performance more accurately than task count alone: held-out MAE is 0.0285 versus 0.1706 when the highest-compute quarter is excluded from fitting. Larger budgets favor more tool and database diversity.

This supports modeling composition alongside quantity, with important boundaries. Fits are separate for each size; diagnostics use AppWorld and SynthAPI holdout, and score normalization uses the observed maximum across the full trained population. Reported compute is C ~= 6ML, where M is student

parameter count and L counts supervision tokens; it excludes synthesis, teacher sampling, and evaluation costs and is not measured end-to-end FLOPs. Four evaluation trials per task do not establish training-seed uncertainty. Transfer uses a shared Python interface, and the fitted law is validated on one synthesis pipeline.

5. From “Think Longer” to “When Should Thinking Stop?”

The distillation branch answers a different question: instead of rediscovering all reasoning behaviors, transfer reasoning traces generated by a strong model to a smaller one. s1 applies SFT to Qwen2.5-32B-Instruct using one thousand examples selected for difficulty, quality, and diversity, then uses budget forcing to control stopping or append “Wait.” The initial paper reports an AIME24 increase from 50% to 57%, while explicitly noting that further extension saturates and is constrained by context length. It demonstrates that a small amount of carefully selected training data can effectively change behavior. But teacher-generation cost, base-model pretraining cost, and test-time cost cannot be counted as zero, and particular mathematical results cannot be described as surpassing the teacher or R1 across all capabilities. s1, §2-6.

Long chains often contain repeated checking, multiple solutions, or unnecessary continuation. Do NOT Think That Much for 2+3=? analyzes this overthinking and tries supervised or preference optimization with shorter correct traces. In the study, the “first occurrence of a correct solution” can be located using offline reference answers; an online system generally lacks that oracle. Showing afterward that a reasoning segment was unnecessary therefore does not establish a deployable real-time stopping rule. Overthinking, §2-3

ThinkPrune puts the budget constraint directly into training: it provides the corresponding reward only when a response is correct within a given token limit, then gradually tightens the budget. This makes the accuracy-length relationship an optimizable choice instead of assuming that every part of the original long chain is necessary. The paper's checkpoint selection allows a certain relative drop in validation accuracy, so its conclusion is a tradeoff between cost and accuracy, not lossless compression for every task. ThinkPrune, §3

As budget forcing becomes a cheap baseline, new failure modes are easier to observe: additional reasoning can repeat completed work, introduce incorrect steps, or change a correct answer into a wrong one. Across training settings with different reasoning lengths, TOPS finds that the optimal length varies by task. It first teaches the model multiple effort levels, then selects the shortest correct responses under different budgets for self-improvement. This changes the objective from “encourage length” to “provide enough compute for the problems that need it.” The evidence still comes mainly from mathematics and a limited set of base models, and identifying the shortest correct trace depends on correctness feedback available during training. Thinking-Optimal Scaling, §3-5.

This does not contradict DAPO's effort to avoid blunt penalties for overlong truncation. The former explicitly changes the task to “solve within the budget,” whereas the latter tries to prevent training-system truncation from mislabeling unfinished but reasonable reasoning as failure. One studies deployment constraints, the other feedback noise. The same length phenomenon may require opposite treatments under different objectives. The appropriate comparison plots the accuracy frontier under budget

constraints and separately examines easy, difficult, and out-of-distribution problems, instead of reporting only the average token reduction.

More broadly, a reduction in average length may result from abandoning a few long-tail difficult problems early. When a task has a strict latency limit, tail latency may matter more than the mean; when errors are costly, confidence calibration may matter more than saving a few tokens. These reflect changes in the deployment objective. There is no “best thinking length” independent of the task's value function. Fixed and per-problem adaptive budgets should be compared on the same problem set with the same cost accounting.

6. From Single-Trajectory Stability to Adaptive Allocation across Problems

Work in 2026 separates this chain of remedies into two distinct problems. **The first is trajectory stability.** Min-Seek addresses repetition and instability after forced extension by retaining shorter segments of previous thinking and reindexing positional-encoding-related KV-cache states. On two R1-distilled models and five types of reasoning tests, it obtains more stable curves than budget forcing that continually accumulates history. Its ability to sustain generation comes from memory management; this does not mean that accuracy on difficult problems increases indefinitely with time. The scale of the experiments and the use of a single generation per configuration also limit the strength of the conclusions. Min-Seek, §3-4, 2026-01 preprint; published in Findings of EACL in 2026-03.

The second is allocating budgets across problems. Adaptive Test-Time Compute Allocation first estimates the utility of different sampling budgets offline, then uses a lightweight classifier to predict a budget for each problem. In its mathematical experiments, this is more effective than assigning every problem the same number of samples. A key distinction is that the hardest problems do not necessarily deserve more compute: the problems worth additional budget are those that have not saturated and genuinely respond to extra computation. The study still requires an offline utility table and uses discrete budgets; it samples 200 problems each from MATH and GSM8K, with an 80/20 split. Routing features include normalized entropy obtained from one LLM call, while the main cost comparisons are measured in sampling budgets. Thus, the lightweight classifier's cost is not the full routing cost; feature extraction, generation length, and actual latency still require end-to-end accounting. Nor do these results guarantee budget transfer. Zhai et al., §2 and §5-6, 2026-04.

In this review's synthesis, the next stage should predict “the expected gain from one more unit of computation,” not just problem difficulty. Stopping, switching methods, or seeking external evidence may also be more valuable than continuing to generate.

7. Does More Compute Have to Mean More Readable Text?

Explicit CoT encodes intermediate states as text, making them easy to supervise and inspect, but requiring an autoregressive decoding cost for each computational step. The architecture branch therefore asks a different question: can internal computational depth increase without every step becoming a word? As early as 2018, Universal Transformers used shared transformations recurrently across depth and studied adapting computation by position. “Fixed parameters, greater iterative depth” is therefore not a concept that originated with the era of reasoning models. Universal Transformers, §2

Quiet-STaR studies generating auxiliary rationales at positions in ordinary text, learning when and how to think from their contribution to predicting subsequent tokens. It generates discrete reasoning text, even though that text need not appear directly in the final answer; being invisible to the user does not make it continuous latent reasoning. The work extends reasoning training beyond manually selected reasoning problems to a broader range of text positions, while leaving additional sampling costs and the faithfulness of intermediate explanations unresolved. Quiet-STaR, §3-4

Let's Think Dot by Dot provides a mechanistic counterexample: in small models specifically trained on controlled synthetic tasks, filler tokens without semantic content can still carry hidden computation. This shows that “the output does not look like thinking” does not imply that no computation occurred. However, the experimental training distribution includes the relevant tasks and filler patterns. The result cannot establish that adding a string of dots to any frontier model will generally improve reasoning. Let's Think Dot by Dot, §3-4

Coconut more directly uses the last hidden state of the previous stage as input to the next, replacing some linguistic reasoning steps with continuous states. Training uses existing CoT as a curriculum, gradually removing earlier textual steps and replacing them with latent steps, then applying a learning signal to the remaining text and answer. This is not equivalent to hiding an ordinary language model's output: the fed-back input, training path, and computation graph all change. Removing explicit text with continuous states does not guarantee cheaper training, because serial latent stages and backpropagation still incur costs. Coconut, §3-5

Coconut's results also require a version qualification. This review reads the experiments and appendices of v4, updated on 2026-08-23, which include studies of larger models than the early versions. Gains are not consistent across models, tasks, and controls. The authors' interpretation that latent space retains multiple paths is suggestive, but cannot be promoted directly into the claim that “every hidden state performs a complete parallel tree search.” Comparisons with textual CoT must also control training data, total forward computation, stopping strategies, and supervision to identify whether gains come from the representation or an additional budget.

The Recurrent Depth branch implements this within the model's main computation: first obtain an input representation e=P(x), repeatedly execute a shared module s_r=R(e,s_(r-1)), and finally produce predictions through an output module. The paper trains a 3.5B model with randomized recurrent depth and uses techniques such as truncated backpropagation to control training cost; the number of recurrences can be adjusted at inference. This introduces a budget axis distinct from output tokens: how many computational iterations occur internally at the same position. Scaling up Test-Time Compute with Latent Reasoning, §3-4

Reusing weights increases compute per problem, but does not proportionally increase the number of independent parameters available to store knowledge. Deeper iteration also requires stable state dynamics; arbitrarily many loops do not necessarily improve performance. “Compute equivalent to a larger model” must therefore not be rewritten as “the same knowledge capacity as that larger model.” The main question in this branch has shifted from how to sample more answers to how to train internal state dynamics that remain useful when computation continues at test time.

8. Failure Experiments: Complexity, Output Protocols, and Knowledge Gaps

The value of The Illusion of Thinking lies in changing the independent variable in evaluation: instead of summarizing capability with one exam score, it increases problem complexity in controllable tasks such as Hanoi and Blocks World. The authors observe that ordinary models use compute more efficiently at low complexity, thinking models benefit at medium complexity, and both collapse at high complexity. Some models also reduce their thinking in the failure region. This shows that existing empirical curves cannot be extrapolated unconditionally to harder combinatorial tasks. Shojaee et al., v3, §4.

However, inferring from failure under a particular output protocol that “models cannot reason” requires additional evidence. Lawsen's commentary questions solution representation, the requirement to output complete action sequences, and the solvability of some tasks, and tests an alternative that outputs executable generating functions. Its v2 corrects v1's mistaken output-length estimates and acknowledges that many collapses occur before the theoretical length limit; the alternative-representation experiments are also limited in scale. The original paper therefore cannot be read as a universal impossibility theorem, nor can one commentary be said to overturn every observed failure. Lawsen, v2, §4-5.

The appropriate research split is to measure three things separately: whether an algorithm is found, whether a long sequence can be executed faithfully, and whether a verifiable result can be delivered through a given interface within a given budget. Allowing code execution changes the system under evaluation; direct model answers and model-plus-tool performance should be reported separately.

Another boundary comes from knowledge tasks. Zhao et al. find that extending thinking does not consistently improve factual accuracy on SimpleQA and FRAMES when external retrieval is prohibited. Changes in hallucination rate sometimes arise from greater willingness to answer or abstain. The authors also find that enabling thinking can still help compared with not thinking. This sharpens the conclusion: **the benefit of having thinking and the marginal benefit of adding more thinking are different questions**. These results also do not cover full test-time scaling that increases retrieval and tool calls. Knowledge-Intensive Tasks, §3-5.

These disagreements should be read alongside Evaluation and the Limits of Extrapolation: changing the output interface or adding tools changes the system being measured. Algorithm discovery, long-sequence execution, and final delivery under a given budget must be distinguished.

9. Putting the Branches Back into One Problem Map

This Mermaid diagram is available in the Markdown edition; it is not rendered here.

The cycles in the diagram show that training compute and inference compute can be converted into one another, but are not freely interchangeable resources. If an expensive search helps only one request, its benefit belongs to inference; if it produces reliable examples and trains a generally better policy, its cost may be amortized over future requests. Conversely, using more pretraining to improve knowledge in parameters does not guarantee a substitute for the tool calls or combinatorial search needed by the current problem. The optimal budget depends on request volume, task distribution, feedback availability, and the cost of errors.

Observed improvement	Confound to rule out first	Control that advances the conclusion
Higher pass@1 after RL	Changes in the base model, template, length, or test sampling	Pass@k and error types with the same base model and budget
Better best-of-N	Oracle selection or additional verifier cost	Actual selection success and full-pipeline compute
More accurate PRM scores	Conflating step correctness with future recoverability	Test error localization and search gains separately
Longer reasoning chains	Normalization bias, repetition, or truncation rules	Accuracy and length distributions at fixed total FLOPs
Strong results from a small model	Uncounted teacher, search, or training-data generation costs	Complete budget accounting and controls for teacher information
Gains from latent depth	Changed training distribution or an unmatched explicit-CoT baseline	Fixed data, forward computation, and stopping rules

These checks do not require every method to use the same experiment; they require each causal claim to have a corresponding control. So far, the evidence better supports several conditional scaling relationships: sample volume depends on the distribution of problem success probabilities, search on evaluation quality, RL on the base model and useful reward signals, and implicit depth on training stability. Compressing all of these into “scaling laws still work” or “scaling laws have failed” loses the mechanisms that actually need explanation.

To connect with the pretraining chapter, we can use an accounting identity rather than a new empirical law:

$$C_{\text{lifecycle}} = C_{\text{pre}} + C_{\text{post}} + \sum_{i=1}^{Q} (C_{\text{generate},i} + C_{\text{verify},i} + C_{\text{tool},i}).$$

The same optimization is also constrained by latency, memory, and throughput. This points to the real unresolved problem of joint allocation: a stronger base model may improve the starting point for exploration, better post-training may improve the return on each unit of inference budget, and better verification may convert candidate potential into actual success. All three require comparison under matched costs and task distributions. Existing results have not yet provided universal exchange rates.

Together with Post-Training and Reinforcement Learning, this chapter forms a training-inference feedback loop. Future updates should prioritize evidence that can change the causal interpretation, such as controlled RL scaling with the same base model, cross-distribution tests of independent verifiers, budget allocation under real total costs, and stable extrapolation of continuous depth at larger scales.

Chapter 6 - Theory and Capability Evaluation: Why Do Power Laws Arise, and What Can They Predict?

Updated: 2026-10-12. Problem thread T6. This chapter places explanations of loss scaling and predictions of task capability within the same chain of evidence, while treating them as distinct claims. Connections between chapters are this survey's synthesis of research questions; relationships to prior work count as historical influence only when the source explicitly discusses them.

Chapter 1 established empirical relationships between resources and training outcomes; the data chapter and architecture chapter explained how changing the recipe changes those relationships. Successful fitting nevertheless leaves two questions: why can very different models exhibit power laws? And even if average prediction error is predictable, can we know when a model will solve a particular class of problems or pass a particular test? The first requires a mechanistic explanation; the second requires a bridge between measurement and transfer.

These questions must constrain each other. A theory that reproduces smooth curves without explaining which interventions would change them has limited explanatory power. A capability prediction that fits historical leaderboards but has not been tested on unseen scales and training recipes cannot directly guide the next large investment either.

6.1 From Fitting to Explanation: Different Mechanisms Can Produce the Same Power Law

Early empirical work showed that increasing model size, data, and compute can often reduce test loss consistently. Here, “consistently” refers to a relationship within a particular training recipe and measurement distribution. Before treating this as a law of nature, we need to ask whether the power law arises from a long-tailed data distribution, the representational resolution of a finite model, parameter-estimation variance, or an optimization process that gradually learns features of different difficulty.

Bahri et al.'s Explaining Neural Scaling Laws distinguishes regimes such as variance-limited and resolution-limited scaling: the former concerns fluctuations introduced by finite sampling or finite models; the latter concerns how much structure can be resolved as resources increase. Different assumptions about smooth manifolds, kernel spectra, and target functions yield different relationships between exponents. The contribution is to separate mechanisms, not to assign a single fixed exponent to every deep learning system. The theoretical conditions still need to be matched to actual data and training regimes. Bahri et al., 2021

This chapter therefore follows two routes toward more specific explanations. The first uses long tails and kernel spectra to describe how much information remains unlearned. The second incorporates training time and feature evolution to ask how quickly that information becomes learnable. Both can produce power laws, but that does not mean both apply simultaneously to the same network.

Research question. Observing L(x)-L_infinity proportional to x^(-alpha) does not establish any particular microscopic mechanism. We need to intervene on the data distribution, task difficulty, model representation, or optimization method, then check whether exponents and transitions change as the theory predicts.

6.2 A Solvable Starting Point: How Does a Long-Tailed Distribution Become a Learning Curve?

Hutter's Learning Curve Theory begins with a deliberately simplified model. Let the occurrence probability of a category or “feature” be p_i, with noiseless labels. The learner memorizes categories it has seen and still makes errors on new ones. It cannot transfer knowledge to similar categories, so the only question is: after n independent observations, what is the probability that the next sample belongs to an unseen category? For category i, the probability of remaining unseen is (1-p_i)^n, and its probability of appearing at test time is p_i. The expected error is therefore

$$\mathbb{E}[E_n] = \sum_{i=1}^{\infty} p_i (1-p_i)^n.$$

This expression makes concrete why more data can remain useful: common categories are learned early, and increasingly rare categories gradually dominate the remaining error. If p_i proportional to i^(-(1+alpha)), with alpha>0, approximating the sum by an integral gives

$$\mathbb{E}[E_n] \propto n^{-\beta}, \qquad \beta = \frac{\alpha}{1+\alpha}.$$

In other words, a long tail in the data-frequency distribution can generate a smooth learning curve in a model with no deep representation learning at all. This does not imply that “neural networks merely memorize.” It is a counterexample reminding us that **the shape of a curve alone cannot identify the mechanism of intelligence.** The same model produces exponential decay for finitely many equiprobable categories. In the example that restricts categories to a finite vocabulary, the truncated approximate power law also departs from that form at larger sample sizes; this does not imply that a finite token vocabulary makes all text sequences or skill categories the same kind of finite set. The model also shows that instantaneous error and error averaged over time have different variances, so smoothing can sometimes change the curve people see. Hutter, 2021, §2-5 and Appendix B

This account explains one source of slow learning rates, but at the cost of strong simplifications: independent categories, deterministic labels, no compositional generalization, and no joint bottleneck involving trainable parameters and compute budgets. An extension to noisy labels in the appendix adds irreducible noise and parameter-estimation error, but still does not establish a predictive theory for modern Transformers. The author explicitly leaves model size, compute, and real deep networks as open questions. Same paper, §6-7 and Appendix C

Michaud et al.'s Quantization Model connects to this line of inquiry: if skills can be divided into discrete knowledge units with different frequencies, local changes as those units are acquired one by one can coexist with smooth aggregate loss. “Quantization” here is a hypothesis about knowledge units, not low-bit weight quantization. It removes the apparent logical contradiction between aggregate smoothness and local jumps, while leaving open whether skills are sufficiently discrete, whether they can be

independent, and whether different representations can realize the same capability. Michaud et al., 2023, Discussion

6.3 From Information Coverage to Learning Dynamics: Time Is Not Another Measure of Data

The long-tail memorization model explains neither representational capacity nor the training process. Kernel methods offer another decomposition: project the target function onto spectral modes, then express learning error as the sum of what remains unlearned in each mode. Intuitively, some directions are easier for the model and optimizer to capture, while others require more samples or training time. Spectral decay and the distribution of target energy jointly determine the aggregate learning curve. Bordelon et al.'s 2020 kernel-regression study, Sharma and Kaplan's manifold-dimension account, and Maloney et al.'s solvable random-feature model belong to this background. For these three papers, this edition has verified only the abstracts; their derivations remain candidates for subsequent close reading. Kernel-spectrum source, manifold source, random-feature source

To address compute allocation, we must further distinguish how much a model can eventually represent from how much it has learned within finite time. Bordelon, Atanasov, and Pehlevan's A Dynamical Model of Neural Scaling Laws explicitly analyzes gradient descent in a random-feature model. It characterizes the task through the decay of kernel eigenvalues and target energy, and gives errors when training time, the number of independent samples, or model size is the limiting resource. Finite time amounts to learning only a subset of effective spectral modes. Finite models and finite data also impose capacity or rank bottlenecks, but these three resources constrain modes in different ways, so the time exponent need not equal the parameter exponent. Bordelon et al., 2024, §4

This explains a distinction that empirical formulas can obscure: more training steps can repeatedly process the same samples without increasing the amount of independent data. Data reuse gradually separates training and test losses; treating steps as fresh data loses this mechanism. Likewise, even if a static model has a symmetric form in data size and parameter count, optimal allocation at finite training time need not require symmetric growth. These are mechanisms in a controlled model, not grounds for revising the measured exponents of large language models.

Yet the kernel model exposes its own limitation. Comparing linearized networks with networks that actually learn features, the paper finds that the latter can improve at a better compute-scaling rate. A fixed kernel explains some curve shapes but understates the value of representations changing during training. This is the limitation that the follow-up How Feature Learning Can Improve Neural Scaling Laws explicitly seeks to address. Same paper, §5.1-6

6.4 When Does Feature Learning Change the Exponent Rather Than Simply Shift the Curve?

The follow-up retains solvability while allowing features to evolve during training. It expresses a finite model's features as learnable linear combinations of an initial feature basis. A parameter controls the rate of feature change; as this rate approaches zero, the model returns to the fixed-feature case. A projection constraint preserves the finite-model bottleneck: a freely varying matrix cannot give the model access to the entire infinite feature space at no cost. Bordelon et al., 2024 / ICLR 2025, §2

The key observation is that a “hard task” must be defined relative to the kernel at initialization. If the target function lies within the reproducing kernel Hilbert space of the initial kernel, the model already has a relatively suitable representation; feature learning can improve constants without necessarily changing the asymptotic exponent. If the target is sufficiently difficult for the initial representation, training strengthens directions that were initially hard to learn and changes the optimization dynamics. Using the paper's source exponent beta to express difficulty, in its infinite-width limit and under the corresponding training conditions, time scaling in the hard regime 0<beta<1 improves from t^(-beta) to t^(-2beta/(1+beta)). This beta and the exponent in Hutter's model above have different definitions and cannot be compared directly. Same paper, §4

The significance is not that “exponents universally double,” but that the result makes an intervention prediction: changing the target's difficulty relative to the initial representation should change the benefit of feature learning to the exponent. The paper tests this using target functions with different Fourier spectra on a circle and nonlinear MLPs, then examines vision tasks. The theory describes the dynamics well in some experiments; late in training on CIFAR-5M, actual networks still improve faster than predicted, indicating that the current feature-combination model does not capture every mechanism. Same paper, §5-6

This also changes the connection to Chapter 1: if an intervention can change the time exponent, the optimal allocation between parameters and training duration must be solved again, rather than merely shifting the old curve downward. Unresolved aspects include cross-entropy, adaptive optimizers such as Adam, and finite deep networks' ability to create representations beyond the span of the initial features. Current theory should be understood as progressively narrowing the mechanistic gap, not as having already derived frontier LLM budget recipes from first principles.

Explanatory route	What produces the power law	Testable prediction	What remains missing
Long-tail coverage	Unobserved probability mass	Changing the frequency tail should change the data-scaling curve	Compositional generalization, parameters, and optimization
Resolution / spectral modes	Unresolved representational directions	Changing data geometry and the target spectrum should change exponents	Representation evolution during training
Training dynamics	Joint constraints from finite time and finite resources	Time, sample, and model bottlenecks can be asymmetric	More general networks and optimizers
Feature learning	Training changes the effective kernel and learning directions	Exponents may change for tasks that are harder under the initial representation	Quantitative, advance predictions for modern LLMs

Another Return to Depth: Selective Retrieval and Error Cancellation

Spectral explanations still leave a question: can nonlinear attention change what depth does? Peng et al., 2026, §2-4 and §6 compare linear and softmax attention in controlled in-context learning: copying a matching token's label or averaging the two most relevant tokens' labels. On the latter task, nonlinear models show approximately inverse-depth decay of excess loss above a fitted floor across tested spectra. Selective retrieval lets layers form similar but nonidentical predictions: aggregation cancels some layer-specific errors, while shared error sets a plateau.

With separate input and label streams and context fixed across layers, the controlled model can be written as a parallel average of layer predictions. In this representation, the mean squared error at depth L satisfies

$$\mathcal{R}_L = C_L + \frac{A_L - C_L}{L}.$$

C_L is the cross-layer shared-error term, and A_L-C_L measures prediction disagreement. This identity alone does not prove an exponent of one; both terms' depth dependence needs checking. Appendix D.4's convergence fits provide an empirical consistency check. A counterexample matters: in the isotropic-spectrum extension, with context updates enabled, top-16 quickly saturates while top-2 retains depth gains. The mechanism supplements spectral accounts without replacing all such theories, and establishes neither real-language generality nor joint width, data, and compute optima.

6.5 From Loss to Capability: A Function Is Still Missing from the Prediction Chain

The preceding discussion primarily concerns error or loss. Practical decisions concern whether a program runs, an answer is correct, or a long task is completed. Average cross-entropy does not identify the samples on which errors concentrate, nor does it specify prompting, decoding, or scoring. The mapping from resources to capability therefore needs at least two stages: resources and the training recipe determine some model state; that state produces outcomes under a particular task and measurement procedure.

The GPT-4 technical report provides a useful bridging experiment. Researchers used runs with less compute to predict in advance the final loss on an internal held-out code set; separately, they predicted average log pass rates for subsets of HumanEval problems. These predictions target different quantities, and the latter is also limited by problem selection and finite sampling: the 15 hardest problems were excluded from the bucketed predictions, and some buckets deviated from the forecasts. Model size, absolute compute, and data construction were not fully disclosed, limiting external reproducibility. This supports advance prediction of some capability metrics; it does not establish that every capability follows directly from a single average loss. OpenAI, 2023, §2-3

Du et al. ask a further question: once training conditions are fixed, is loss a better coordinate for capability than parameter count? Keeping corpus composition, tokenizer, and architecture family fixed, they train models of different sizes for different token counts and examine intermediate checkpoints. On many tasks, performance points align more closely with a common trend when plotted against loss than against model size. This addresses the ambiguity on the parameter axis introduced when a smaller but

sufficiently trained model outperforms a larger one: parameter count is not a sufficient statistic for how much has been learned. Du et al., 2024, §2

But this coordinate is conditional. The paper's own limitations note that losses from different tokenizers or corpora are not directly comparable; architectures and optimizers were not comprehensively swept either. The study concerns task performance under prompting and does not automatically cover systems subjected to different fine-tuning, RL, or search procedures. The next step is therefore not to seek a universal loss threshold, but to specify the fixed conditions under which loss can reliably compress the model's state.

6.6 The Emergence Debate: Scoring Functions, Chance Baselines, and Prediction Boundaries

Curves on which small models remain near zero before larger models suddenly improve prompted discussion of emergent abilities. BIG-bench and the work of Wei et al. provide observations across tasks. This edition has verified their abstracts but does not count them among the newly completed close readings. BIG-bench, Emergent Abilities

Schaeffer et al. pose a more basic identification question: can the scoring rule create a steep curve even when output quality improves smoothly? In a simplified independent-token model, if each token is correct with probability p, the probability that an answer of length L is entirely correct is approximately p^L. Smooth token-level improvement can therefore appear as a long period near zero followed by a rapid rise when transformed by exact-match scoring. Discrete scoring, finite samples, and sparse model scales further affect the visible shape. By changing metrics on fixed outputs, the paper shows some apparent emergence disappearing, addressing the inference that a steep curve necessarily indicates an abrupt internal mechanism change. Schaeffer et al., 2023, §2-4

Du et al.'s response goes beyond simply switching to a continuous metric. In their experiments on MMLU, C-Eval, and other tasks, correct-option probability and Brier score still exhibit transitions relative to a chance baseline. In a four-option task, a predictor that ignores the question and always assigns uniform probability to all four options has a Brier score of 0.75 when the score is not normalized by the number of options. Improving from worse calibration to this level does not establish that the model has learned to distinguish the answers. An increase in the correct option's probability must also be considered alongside changes in incorrect-option probabilities to establish improved discrimination. Du et al., §3 and Appendix C

These findings need not cancel each other out. Schaeffer et al. establish that measurement can produce some visible transitions, without ruling out all real transitions. Du et al. show that a continuous metric alone is insufficient to rule out a transition, without proving that every future capability must appear at a universal threshold. Together, they demand a more precise question: does the change occur in the model's output distribution, task capability relative to chance, or the final utility metric?

This survey therefore separates “emergence” into three independently testable claims: a metric exhibits a transition; that transition cannot be explained by the measurement transformations tested; and the transition can be predicted from small-scale data specified in advance. The first does not imply the other two. A real program may indeed need to be entirely correct to be useful, so the utility of exact match

should not be dismissed because it is nonlinear. Intermediate continuous signals and final success rates should be reported together.

6.7 When Data and Tasks Are Misaligned, Even Downstream Loss Can Mislead

The previous section discussed loss with the corpus fixed, whereas the data chapter explicitly allows corpus composition to change. Isik et al. investigate the resulting gap: does more pretraining necessarily improve a task after fine-tuning? They pretrain T5 models on different language mixtures, then fine-tune on machine translation, measuring downstream cross-entropy, BLEU, and COMET. Pretraining volume, fine-tuning data volume, and language composition are examined as separate variables. Isik et al., 2024 / ICLR 2025, §3-4

A key experimental control is that each pretraining example is used only once, allowing checkpoint token counts to correspond to the amount of data used; learning rates are selected separately for fine-tuning. When translation languages align relatively well with the pretraining language distribution, task scores and downstream cross-entropy generally improve together. The authors propose a logarithmic form for task scores, fit it using the first four points with smaller pretraining volumes, and validate on the remaining points. Yet in some English-to-French tasks, when pretraining lacks the target language and fine-tuning data are insufficient, more pretraining can make BLEU or COMET fluctuate or even decline **while downstream cross-entropy continues to improve**. Same paper, §4-5 and Appendix C

This is stronger than a mismatch between pretraining loss and some distant task: even when measured on the downstream distribution, an improvement in average probability mass does not guarantee a corresponding improvement in the task evaluation of generated text. Examining BLEU's components, the paper attributes the measured nonmonotonicity to n-gram precision rather than the brevity penalty, but does not provide a complete linguistic mechanism. A heuristic language-alignment score should not be treated as an all-determining causal variable: some weakly aligned settings still scale normally, and not every pretraining setup missing a language fails.

Another boundary concerns the amount of fine-tuning data. In the translation tasks tested, when supervised data are already sufficiently abundant, the incremental benefit of pretraining shrinks. This connects to the post-training chapter: pretraining and fine-tuning are not independent additions to performance; their informational contributions to the same task may partly overlap. The stability of these relationships in decoder-only models, complex reasoning, different decoding strategies, and modern RL post-training remains to be tested. The study's most reliable practical lesson is to retain target-task outcomes throughout training, rather than using one loss proxy to decide which data to keep increasing.

6.8 Prediction Across Families: From Training Scale to Observable Capability Coordinates

Training multiple model sizes to study each capability is expensive. Public models are more numerous, but their recipes differ, so they cannot simply be placed on one compute-scaling curve. Ruan et al.'s Observational Scaling Laws connects these limitations: first construct low-dimensional capability coordinates from standard benchmark scores, then use those coordinates to predict more complex target tasks. Their main analysis of base models covers 77 models from 21 families; the first three principal components explain approximately 97% of the variation in the selected benchmarks. Ruan et al., 2024, §3

The method involves two distinct mappings. Within a family, capability coordinates are approximately related to log compute. Across families, target-task scores are mapped through shared capability coordinates. This allows recipes to differ in how efficiently they convert compute into capability and includes models without disclosed training FLOPs. When training cost is unknown, the predictive evidence comes from existing benchmark performance; this is not a formula that predicts any future model from its training budget alone.

The paper uses held-out splits between weaker and stronger models and fits preprocessing, including PCA and missing-value handling, only on the training set to avoid leaking information from stronger models into predictions. It also selects a small set of models spanning different capabilities through experimental design, reducing the cost of evaluating new methods. This provides tools for studying scaling trends in CoT, self-consistency, and simple agent scaffolds, connecting to the inference chapter: method comparisons need not rely solely on three model sizes released by the same company. Same paper, §4-5

The boundaries are equally clear. The meaning of a principal component depends on the chosen models and benchmarks; it is not a directly measured cognitive organ. Benchmark contamination and heterogeneous recipes within a family can break the relationship. The paper primarily studies prompting and relatively simple inference interventions, so its results cannot be transferred directly to large-scale RL, intensive search, or every fine-tuning setting. Kearns's 2026 work on latent capabilities and measurement error continues to ask what leaderboards measure, but its observational, exploratory factor analysis does not establish a causal mechanism either. Ruan et al., §7, Kearns, 2026

6.9 More Important Than a Good-Looking Curve: Extrapolation Requires Independent Tests

Alabdulmohsin et al. point out that multiple parameter combinations can almost coincide over the observed small-scale range, yet diverge substantially at larger scales. Minimizing in-sample error therefore need not identify the exponent that extrapolates best. Their approach constructs a functional form compatible with near-chance behavior at small scales, an intermediate transition, and a later power-law regime, and evaluates estimators by their errors on larger held-out scales. Alabdulmohsin et al., 2022, §3-4

Caballero et al.'s Broken Neural Scaling Laws further allow several smoothly connected power-law segments, helping describe plateaus, acceleration, double descent, and other shapes. The paper also

explicitly shows that predicting a transition often requires observations near or across that region. A more flexible function does not imply the ability to foresee any new regime from far away. Caballero et al., 2022 / ICLR 2023, §4-6

Candidate literature on inverse scaling, U-shaped reversals, and double descent should also be understood in this context. These works propose different sources of nonmonotonicity and should not be assigned a common mechanism simply because their curves look similar. In particular, deterioration on some problems should not be generalized into a failure of pretraining scaling as a whole. This edition retains these entries as branches awaiting close reading, rather than substituting abstracts for mechanistic review.

This chapter uses the following distinctions between kinds of evidence; future updates will also use them to determine where a new result belongs:

Evidence type	What it can support	What it cannot directly support
Derivation within a theoretical model	Exponents and intervention predictions under the stated assumptions	Applicability to arbitrary modern networks
Fit to known scale points	Compatibility between the chosen function and historical data	Correct prediction at unseen scales
Validation on larger held-out scales	Predictive power over a specified extrapolation range	Validity for arbitrary new recipes or distant extrapolation
Preregistration followed by validation	Reduced post hoc selection and hindsight bias	A universal law across all tasks
Benchmark correlations across models	Shared variation and predictive signals within the sample	Causal effects of training variables on capability

The remaining open questions can be made concrete: can we measure changes in a model's representation to predict exponent changes in advance? Can we separate the effects of lower loss, different data composition, and better task alignment under a fixed budget? Can we forecast capability transitions without leaking target-task information? Can independent evaluation confirm the progress identified by automated evaluators? These questions reappear in the multimodal chapter: there is likewise no automatically valid conversion between classification error, image quality, action error, and closed-loop success.

Reading records are available in the new primary-source metadata and the expanded verification log (Chinese). The main text and all appendix text of Hutter, Isik, and Du have been read; targeted-reading or abstract-only candidate status for the other new sources is recorded individually. Reading scope for the existing GPT-4, Schaeffer, BNSL, Bahri, Michaud, and Kearns sources remains as recorded in the corresponding source files.

Chapter 7 - Vision, Multimodality, Diffusion, and Robotics: What Should Be Scaled to Improve the Target Capability?

Updated: 2026-10-19. Problem thread T7. What transfers across domains is the experimental methodology and the resource-allocation question, not an assumed shared exponent. Core sources read in full include mixed-modal scaling, diffusion inference search, and robotic data scaling; frontier or historical additions checked only at the abstract level are marked separately.

Parameter counts and token counts in language models provide a convenient starting point, but they are not naturally shared units across every domain. An image can become hundreds of patches, thousands of discrete codes, or a continuous latent representation; a robotic trajectory also carries correlations involving environments, objects, actions, and time. Plotting different domains on log-log axes therefore does not establish that they follow the same resource relationships.

This chapter follows four connected questions: can visual supervision scale? When do modalities become synergistic within a shared model? How can a trained generative model continue to use inference compute? And in closed-loop robotics, should increasing sample size mean collecting more repeated demonstrations or covering more environments and tasks? Each expansion of the capability scope requires resources, losses, and ultimate success to be defined again.

7.1 Beyond Text: Similar Laws Do Not Make Metrics Interchangeable

Henighan et al. extend autoregressive generation to images, video, image-text, and mathematical data to test whether smooth resource-loss relationships extend beyond text. Under controlled conditions, several cross-entropies can be described by a power law plus a constant; downstream image classification can also continue improving as generative loss approaches a plateau. This is evidence for similar empirical relationships across modalities, not a conclusion that similar losses imply identical capabilities. Resolution, tokenizer, and the particular architecture all condition these results. Henighan et al., 2020, §1 and §3-4

On the route that evaluates visual tasks directly, Scaling Vision Transformers jointly studies model size, data, and training duration: fixing one can obscure the benefits of the other two, and the error-compute frontier depends on their combination. Its curves also account for saturation in low- and high-compute regimes. But the fitted floor of classification error is not irreducible entropy in the information-theoretic sense. Splitting images into more patches also changes the cost of each forward pass, so comparing only the number of training images is insufficient. Zhai et al., 2021 / CVPR 2022

The original ViT, large-scale ViT-22B, and MAE point to three scalable routes: reducing architectural dependence on convolution, addressing training stability at large scale, and using masked self-supervision to reduce annotation requirements and encoder computation. Their original abstracts have been verified, but this edition has not fully checked their methods and ablations, so they remain historical follow-up candidates. A title containing “scale” does not by itself make a paper a complete law of optimal resource allocation. ViT, ViT-22B, MAE

The first shift along this vision route is from showing that a larger network scores higher to asking what optimal frontier different combinations of data, model size, and training jointly form. A second shift concerns supervision itself: human-provided closed-set labels cannot expand indefinitely, so could natural language supply a training signal with broader coverage?

7.2 Natural-Language Supervision Expands Coverage but Returns the Bottleneck to the Data Distribution

CLIP converts web image-text pairs into a contrastive learning signal, maps images and text into a shared representation, and uses textual descriptions to define zero-shot classification. Its significance is not merely a new architecture; it changes the source of scalable supervision. The objective also determines the trade-off: matching images to text does not mean that counting, fine-grained recognition, or every out-of-distribution task has been solved. The paper explicitly discusses these capability boundaries. Radford et al., 2021, §2 and §6

Closed data, however, leave an attribution gap: is a new CLIP model stronger because it is larger, has seen more independent image-text pairs, repeats training for longer, or uses filtering better suited to the evaluated tasks? Cherti et al. systematically sweep these axes with public LAION data and OpenCLIP. The experiments distinguish **the size of the independent dataset** from **the cumulative number of image-text samples seen during training**, and set a separate cosine schedule for each training length. This matches the lesson about training recipes in Chapter 1: an intermediate checkpoint from one long training run is not necessarily equivalent to a complete run optimized for a shorter budget. Cherti et al., 2022 / CVPR 2023, §3

This design reveals alternating bottlenecks. At fixed cumulative training volume, the benefit of more independent samples can be small; it becomes apparent after training volume increases. Conversely, with little independent data, further repetition may stop helping, with gains resuming only after switching to a larger dataset. It is therefore insufficient to report that “more data did not help” or “longer training helped” without specifying the levels of the other two resources. Same paper, §5

A more important limitation is that OpenAI CLIP and OpenCLIP have different relative scaling advantages in zero-shot classification and retrieval. The authors suggest differing affinities between pretraining data selection and downstream tasks as a possible explanation, without establishing it as the sole cause: large-scale experiments are sparsely sampled, hyperparameter search is limited, and OpenAI's private data cannot be freely rerun. This connects to task alignment in Chapter 6: sharing a model family and similar objectives does not remove the effect of data distribution on transfer curves.

DataComp further places data selection within a controlled comparison: hold the training procedure and compute budget fixed, compare selection strategies on a candidate image-text pool, and evaluate across 38 downstream datasets. This gives “high-quality data” a comparable operational definition. Yet even a strong correlation between ImageNet and average performance cannot replace every task; some tasks have markedly different relationships. Gadre et al., 2023, §3 and §5

This branch leaves two distinct follow-up directions. SigLIP uses a pairwise sigmoid objective to reduce reliance on global softmax normalization, making the roles of batch size and positive/negative pairs easier to study separately. JEST instead points out that an example's value under a contrastive objective depends on the other examples in its batch, so independent scoring can be inferior to joint selection.

They change the objective and the unit of data selection, respectively. This edition has verified the abstracts but has not completed a full cost and ablation review at matched compute budgets, so it does not repeat their results as confirmed universal speedup factors. SigLIP, JEST

Interim conclusion. Scalable supervision addresses where labels come from; public controlled experiments address which variable causes an improvement. The next question is whether multiple modalities sharing a model share knowledge or compete for limited representation and optimization capacity.

7.3 Mixed-Modal Scaling: When Do Additional Modalities Shift from Competition to Synergy?

Turning language, images, speech, and other modalities into tokens allows one generative model to process modality sequences in arbitrary arrangements. A unified representation, however, only solves the input-format problem; it does not guarantee that joint training outperforms separate training. A small model may learn multiple distributions simultaneously yet learn each of them insufficiently.

Aghajanyan et al.'s key change is to explicitly add a **modality-interaction term** to the unimodal resource functions. The study covers seven modalities or modality combinations defined as distinct distributions, using a discrete image tokenizer, HuBERT speech units, and text BPE. The principal sweeps of unimodal and bimodal models range from 8M to 6.7B parameters, with an additional 30B model used to check extrapolation. The study first estimates each modality's own loss as parameters and data vary, then examines seven bimodal pairings. Each modality supplies half the tokens in the bimodal experiments; this is not an exhaustive sweep of arbitrary ratios across all modalities. Aghajanyan et al., 2023, §3-5

To highlight the idea, we can write the paper's interaction component as

$$L_{ij} = \frac{L_i(N, D/2) + L_j(N, D/2)}{2} + \Delta_{ij}, \qquad \Delta_{ij} = -C_{ij} + A_{ij}N^{-\alpha_{ij}} + B_{ij}D^{-\beta_{ij}}.$$

Here, D is the total number of mixed-training tokens, with each modality contributing D/2. Following Equation 4 and the paper's comparison at matched per-modality token counts, the baseline consists of two unimodal models: each has N parameters and is trained on D/2 tokens of its own modality, and their losses are averaged equally. The mixed model also has N parameters but is trained on D tokens in total. The expression rewrites the original L(N,D_i) and L(N,D_j) for this equal-mixture setting; both model size and per-modality data volume must remain explicit. -C_(ij) represents potential synergy, while the final two terms represent competition costs under finite capacity and training resources. When competition exceeds synergy, the shared model has worse average loss. As resources increase, competition costs decline, potentially allowing the model to cross the boundary Delta_(ij)=0. This differs fundamentally from adding unimodal data volumes and reusing the old formula: an extra modality can both provide information and consume limited resources. Same paper, Equations 4-6

The researchers go beyond fitting historical observations. For the Speech/Text combination, the fitted compute-optimal point near the boundary is approximately 28.35B parameters and 45.12B tokens. The actual test uses the available 30B architecture and 50B tokens, allowing some margin in data volume. The result crosses the competition boundary defined in the paper. Both numbers should be retained: **45.12B is the fitted prediction; 50B is the training setting of the validation run.** A successful

validation for one speech-text pairing does not establish that every new modality will naturally become synergistic at some scale. Same paper, §5.2.2

This interaction view also explains why average loss can hide imbalances within training. The paper observes that, while the shared model's overall perplexity decreases smoothly, an individual submodality can stall for a long training interval before improving again. Larger models can reduce some such stalls; interaction parameters also correlate with batch settings and the number of gradient spikes. These correlations do not establish that the parameters themselves are causal mechanisms, nor do they guarantee that increasing model size alone solves all training instability. Same paper, §6

Unresolved boundaries concern representations, mixture ratios, and objectives. An image tokenizer's compression ratio changes sequence length, visual detail, and the actual cost per image; speech tokens introduce their own information loss. Equal token counts therefore should not be interpreted as equal semantic information. The appendix's comparisons of visual and speech reconstruction underscore that the tokenizer is part of the entire scaling system. Extending the result to unequal mixtures, more modalities, long video, different task losses, and perception-action loops requires identifying the interaction terms again, rather than copying these coefficients.

MM1's abstract offers an engineering route to follow: image encoders, resolution, image-token counts, and mixtures of data types affect its models more than some connector designs. This agrees with the question raised here-resource and data bottlenecks remain after format unification-but MM1's detailed ablations remain candidate reading in this edition. Its abstract cannot justify a universal architectural ranking for all MLLMs. MM1, 2024

Unequal mixtures: transfer gains must be weighed against shared-capacity costs

Equal-ratio interactions leave modality supply and repetition unresolved. Khandelwal et al., 2026, §§3-6 and Appendices G-H use 268 training runs to separate directional transfer, shared-capacity competition, and repetition saturation in effective data and parameter models. The fixed MoE family spans 457M-8.3B total parameters, including inactive experts and embeddings; these counts cannot directly represent active computation per token. Vision and audio validation losses also include associated text, rather than measuring pure perceptual quality.

Three validations without refitting cover larger-model training from scratch and two continued-pretraining mixtures. Eight of nine modality losses have relative prediction errors within 2.51%, but text error reaches 6.73% for the non-text-heavy continuation. Longer training is a proposed explanation for resolving this discrepancy, not a demonstrated fix. Repetition saturates asymptotically rather than becoming worthless after two epochs; optimal repetition still depends on supply and budget. Transfer from audio to text and vision is near zero and unstable across folds; this does not eliminate capacity costs. This extends resource accounting, but fixed tokenizers, mostly single-seed runs, and trade-offs across ablation metrics do not establish universal coefficients or unrestricted extrapolation.

Pairing Is Also a Resource: Why Equal Modality Totals Are Insufficient

Interaction terms leave another variable unidentified: whether modalities occur together in training examples. Ma and Narayanan, 2026, §3-6 hold per-modality counts and training updates fixed while varying pairing, measuring cross-entropy in Digits, VQA classification heads over frozen encoders, and NLVR2 fine-tuning. Their four-channel model separates redundant, modality-unique, and jointly available information; paired counts drive the last component, with a gate allowing learning to start only after sufficient paired exposure. This adds an experimental axis: more unpaired examples do not equal more cross-modal connections.

However, more pairing means fewer distinct labeled rows and more epochs; equal updates also do not establish equal FLOPs across architectures. NLVR2 curves exclude seeds that never escape chance, and test loss uses validation-set temperature calibration, so these curves do not establish unconditional training success. The abstract's 3.2% versus 10.4% refers to Table 3's full-pairing holdout, not aggregate error across all extrapolation tests. The study supports counting pairing as a resource; unlabeled generative pretraining and frontier model scales remain untested.

After Aggregate Gains: What Evidence Establishes Subtask Regressions?

Lower mixture loss or higher aggregate scores need not improve every target subtask. Saxena et al., 2026, §3-5 and Appendices D/L compare 19 released VLMs in four families using paired correctness flips: smaller-correct/larger-wrong minus the reverse. Requiring at least 20 questions and at least five net regressions selects 375 cases from 10,434 eligible comparisons; 16 remain significant after BH multiple-testing correction (FDR 0.05) across all 10,434. These are not 375 independent capability failures; larger models win most comparisons.

Rerunning both models with CoT leaves 96 of the 375 candidates meeting the same threshold. However, the token cap rises from 2,048 to 4,096, and smaller-model accuracy falls in 251 cases; narrowing gaps do not all represent larger-model recovery. Training data, encoders, and alignment also differ, preventing a causal interpretation of parameter increases. This connects Chapter 6's measurement limits to multimodal model selection: aggregate laws can guide allocation, while target subtasks still require paired evaluation and multiple-testing controls.

7.4 From Understanding to Generation: Which Kind of Compute Does Diffusion Scale?

Vision-language understanding is often evaluated through classification, retrieval, or question answering. Image generation additionally concerns realism, satisfaction of text conditions, and overall diversity. These can move in different directions under the same intervention. Diffusion scaling must therefore retain the training objective, sampling process, and final evaluation together, rather than reporting only a lower FID.

Three groups of work whose abstracts have been verified illustrate the evolution on the training side. Improved DDPM studies reverse-process variances while improving likelihood and generation quality, allowing quality to be maintained with fewer sampling forward passes. Latent diffusion moves the main generative process into a compressed space, reducing pixel-space computation. DiT replaces the U-Net with a latent-patch Transformer and compares how depth, width, and token count affect forward-pass

GFLOPs and FID. These change the sampling parameterization, representation space, and backbone architecture, respectively; their gains cannot all be attributed to more parameters. Improved DDPM, Latent Diffusion, DiT

Scaling Rectified Flow Transformers further brings noise-sampling strategies and image-text Transformer architecture into large-scale experiments, connecting validation loss with generation quality. These are training-side milestones worth reading more closely. This edition has checked only their abstracts and versions, and does not derive a unified compute-optimal training exponent from them. Esser et al., 2024

On the inference side, a more direct tension appears: diffusion generation already calls the network repeatedly, so adding denoising steps naturally adds compute, but the gains often saturate quickly. Must a further budget increase bring almost no return? This is the same resource-allocation question raised in Chapter 5, except that the search objects are now noise and continuous generation trajectories rather than textual reasoning paths.

7.5 Beyond Denoising Steps: Noise Search, Path Selection, and Overfitting the Evaluator

Ma et al. start from the observation that, with the model and sampling procedure fixed, initial noise affects the final image, and different noise samples are not equally suitable for the objective. Extra compute can therefore search for better noise instead of merely integrating the same generation trajectory more accurately. The paper divides the design space into two axes: **the verifier defines what makes a better candidate; the search algorithm determines how to find candidates.** Ma et al., 2025, §2-3

Random search is the simplest approach: generate multiple complete candidates and retain the one with the highest evaluator score. It corresponds to best-of-N in language models and shares a risk: the evaluator may reward only part of the task. The paper first uses an oracle verifier that knows the final evaluation procedure for a proof of concept. Such results can show that better samples exist among the candidates, but they do not represent a capability directly available in general deployment, because real tasks may not permit using evaluation-set statistics to select generated samples.

More realistic verifiers use DINO, CLIP, or models associated with human preferences. Counterexamples become crucial here: stronger selection based on each image's classification confidence gradually concentrates samples in high-scoring regions preferred by the evaluator. Precision of the generated sample set rises while recall falls, reflecting a trade-off between fidelity and distributional coverage. The paper observes improved Inception Score accompanied by worse FID at larger search budgets, interpreting this as the selection process losing overall diversity. Proxy-objective overoptimization can occur even without updating model parameters. Same paper, §3.1 and Appendix B

The subsequent changes therefore alter search geometry as well as candidate count. Zero-order local search starts from a noise sample, compares nearby candidates, and moves to a better location. Path search adds noise again at intermediate noise levels, denoises, and selects which branches to retain. Locality can partly slow the concentration of the entire sample set into a few high-scoring regions, but this is not a permanent guarantee: similar diversity problems can return when neighborhoods expand. It addresses part of the limitation caused by overly rapid concentration under global selection, while leaving open how to design verifiers that account for both individual-sample quality and population diversity. Same paper, §3.2 and Appendices A-C

Text-to-image generation further exposes dependence on the objective. Aesthetic scores favor style and visual presentation; CLIPScore emphasizes image-text matching; ImageReward comes from a different preference-training signal. A better-looking image can comply less well with a complex prompt. An evaluator combination useful in general DrawBench evaluation may also be less effective on T2I-CompBench, which emphasizes attribute binding, spatial relations, and counting. Thus, combining more evaluators is not necessarily better; evaluators must match the deliverable's objective. Same paper, §4

Budget comparisons also have three levels. For a fixed model, the number of network function evaluations, NFE, can compare search with denoising. Across models, each forward pass has a different cost, requiring a FLOPs comparison. Real systems must additionally account for verifier calls, scheduling, and latency. The paper uses estimated GFLOPs for comparisons across model sizes, finding that a smaller model with search can outperform a larger model without search in some budget regions, while noting limited gains when the base model is too weak. It does not support the claim that any small model can replace training with unlimited search. Automated scorers, model judges, and estimated costs are also insufficient substitutes for full human-preference experiments or deployment accounting. Same paper, §5 and Appendix A.5

How the inference budget increases	Where the extra compute goes	Main bottleneck
More denoising steps	Numerical solution along the same sampling trajectory	Reaching a plateau jointly determined by model and solver error
More complete candidates	Broader coverage of initial noise	Verifier quality, selection bias, and diversity
Local noise search	Gradual improvement within a candidate region	Local optima, neighborhood settings, and evaluator bias
Intermediate path search	Branching and selection during generation	Difficulty of evaluating intermediate states and extra call costs

The 2025 classical-search work connects local Langevin search with breadth- and depth-based tree search. The 2026 IPR work instead attempts revision without an external verifier: during sequential mixed-noise generation, it adds noise again to selected regions and revises them conditioned on the remaining regions, improving consistency with global constraints. These new directions, verified at the abstract level, ask how to search more effectively and whether revision is possible without an evaluator, respectively. This edition retains them as candidates and does not take results on constrained visual puzzles as establishing a new universal law for general image generation. Classical Search, IPR

7.6 Into Robotics: More Trajectories Do Not Mean More Independent Information

Robotics closes the loop between inputs, outputs, and the environment: actions change the next observation, errors can accumulate, and adjacent frames within a demonstration are highly correlated. One million action tokens are neither one million independent samples nor a statement of how many environments, objects, or skills are covered. The starting question therefore shifts from quantity to data composition and the generalization protocol.

RT-1, RT-2, and Open X-Embodiment offer three connected routes: using a shared high-capacity policy to absorb multitask data; representing actions as tokens to transfer web vision-language knowledge; and standardizing data from multiple robots to study positive transfer across embodiments. This edition has checked only their original abstracts and versions, retaining them as historical nodes awaiting close reading. They address data sharing, semantic transfer, and the use of heterogeneous data, respectively, but do not individually answer which real trajectories should be added under a fixed collection budget. RT-1, RT-2, Open X-Embodiment

Lin et al.'s Data Scaling Laws in Imitation Learning for Robotic Manipulation narrows the question to controlled single-task generalization: which is most effective-increasing the number of environments, the number of objects, or demonstrations per condition? The researchers collect demonstrations using a UMI handheld gripper and learn actions with a DINOv2 visual encoder plus Diffusion Policy. They first sweep data axes on a pouring motion and mouse-placement task, then validate the collection strategy on towel folding and charger unplugging. Lin et al., 2024 / ICLR 2025, §1-4

The study does not merely evaluate action MSE after splitting a dataset. Models execute closed-loop rollouts in unseen environments, with unseen objects, or with both changed. Tasks are divided into stages with explicit criteria and assigned normalized scores. When evaluating the same set of policies, the order is randomized and the initial conditions of the objects and robot are kept as consistent as possible. The comparison therefore concerns generalization in actual execution, not merely static action-prediction error.

The results support a finding relevant to collection decisions: generalization scores improve with greater environment and object diversity, whereas increasing demonstrations at fixed conditions saturates earlier. The appendix also reconnects points with approximately equal total demonstration counts and still finds a diversity benefit. However, this is an approximate matched-sample comparison constructed from existing experiments, not a new set of experiments with exactly equal counts in every condition; the strength of control should not be overstated. Same paper, §4 and Appendix G.2

7.7 Where Are the Boundaries of a Robotic “Scaling Law”?

What deserves to be retained from this study is not only its positive results but also its account of fitting boundaries. The vertical axis of the power-law fit is 1-normalized score, not training loss or final success rate. The horizontal axis counts environments, objects, or environment-object combinations. The main fit has only six scale points, so the authors do not also fit an irreducible-error term. Extrapolating from this fit, the paper estimates that approximately 1,191 environment-object combinations would be needed for a normalized score of 0.99 on the mouse task, but **this prediction has not been validated by collecting data at that scale**. Same paper, §4.2

A more firmly supported result is the local transfer of the collection strategy: in settings with similar task difficulty, the recommendation to collect in more environments, pair each environment with a new object, and use approximately 50 demonstrations per combination is applied to two new tasks. Across four tasks in eight new environments, average success rates range from approximately 85% to 92.5%; these figures are not guarantees for every new task or arbitrary environment. There are still marked differences between new environments-for example, the pouring task reaches only 40% success in one environment. Broad generalization language in the paper must be bounded by its actual tasks and test scope. Same paper, §5 and Appendix G.4

Task definitions also matter. The “pouring” test keeps the bottle cap screwed shut and does not actually pour water; scoring measures grasping, aligning the bottle opening with the cup, and placing the bottle back. The towel-folding task also has conditions such as prior folding and a bounded range of initial orientations. These protocols can test clearly specified motor skills, but they cannot be rewritten as full validation of fluid manipulation or arranging cloth from arbitrary states. Same paper, Appendix D

Experiments in the same paper also connect directly to evaluation in Chapter 6: the LoRA setting has lower action MSE but lower closed-loop scores than full fine-tuning. Enlarging the visual encoder improves the tested tasks, while enlarging the action U-Net does not bring the same gains. Different modules within one system already face different bottlenecks, which total parameter count cannot summarize. Lower offline proxy loss also cannot replace evaluation of actual execution. Same paper, §6 and Appendix E.1

The resulting research chain is: larger datasets create the potential for policy generalization → diversity must be separated from repetition → testing must use closed-loop execution in unseen environments → the data relationship must then be checked across tasks, algorithms, and hardware. The authors explicitly do not resolve task-level generalization, RL data scaling, or complex dexterous manipulation, and their main conclusions depend on one policy-learning route. AXIS in 2026 proposes a new engineering direction for broader task coverage through community collection, automatic task generation, and task-snapshot evaluation. This edition has verified only its abstract and retains it as a data-engine candidate, not as cross-task validation of the preceding single-task power law. Lin et al., §7, AXIS, 2026

A New Branch: Forecasting Human-Data Transfer Is Not Forecasting Robotic Success

Figure's September 17, 2026 Helix 2.5 report extends the collection question to human-to-humanoid transfer. Four nested Index subsets span an 8-fold data range, with model size and downstream training fixed. Smaller runs forecast the largest run's held-out action-prediction loss; the reported error is 0.54% of the loss variation across that range, not an error in success rate. A separate controlled comparison reports that pretraining raises whole-task success from 9% to 56% across three behaviors in 30 unseen homes. Zero-shot refers to environments and objects; task-specific fine-tuning data are collected elsewhere. Figure's official report, scaling, evaluation, and appendix

This extends the question of which data to expand, while keeping offline prediction and closed-loop execution as separate validation targets. Developer-reported results over a limited data range and three behaviors do not establish a power law for arbitrary task success or joint model-data-compute optimality. This survey has not independently reproduced the experiments.

Inference-Time Search Also Depends on the Evaluation Path

Closed-loop testing leaves another question: does extra search expose useful choices, and is the measurement itself reliable? Liu et al., 2026, §§3-6 and Supplement §§9-10 audit Cosmos Policy on RoboCasa. With eight candidates and two shared continuation seeds, a single candidate switch followed by a fixed base policy yields a value-selector result 1.66 percentage points below random. The hindsight oracle's 1.40-point gain has an episode-bootstrap interval spanning zero; it is neither deployable nor a reliable universal ceiling.

The efficiency audit separates forward-pass savings from complete-query timing: removing 47.9% of forward FLOPs gives at most 1.152× speedup on the tested A6000 eager deployed complete-query path. This latency-only pruning test does not assess action quality. The smoke sample fails its power gate, so the full joint-allocation factorial was not launched; the result remains inconclusive. The authors also report rendering corruption concentrated in a nonstandard planning path, with small direct-policy impact; the authors did not seek maintainer confirmation of the root cause. These results add three prerequisites-candidate separability, realized latency, and evaluation integrity-without establishing that search fails across other models, simulators, or real robots.

7.8 A Shared Bottleneck Across Domains: Average Progress Versus Progress on the Target

The four routes in this chapter repeatedly exhibit the same structure. After CLIP expands coverage, the bottleneck shifts to the data distribution and transfer objective. After shared generative models add modalities, it shifts to competition and synergy. After diffusion inference adds search, it shifts to evaluators and diversity. After robotics adds trajectories, it shifts to environment coverage, data collection, and closed-loop evaluation.

These are not four unrelated cases. Together they show that resources must be specified with their quality, structure, and purpose: independent image-text pairs differ from repeated exposure; equal token counts can correspond to different information compression; more candidates require reliable selection; and more action records may not expand environment coverage. They also show that proxy objectives and final capabilities each require their own evidence.

Sarridis et al.'s 2026 analysis of 194 public vision-language model checkpoints offers another reminder: the relationship between scale and overall ImageNet accuracy need not extend consistently to worst-group accuracy on UrbanCars, where multiple spurious correlations are present. The result is limited to the models tested and a small set of benchmarks, and cannot characterize every generative VLM. Its value is to show that average improvement may conceal particular failures. Sarridis et al., 2026, §4-5

Branch	Resource units to record	Final objectives requiring separate validation	Priority questions for updates
Visual / image-text representations	Independent image-text pairs, cumulative exposure, both encoders, resolution	Task-by-task transfer, out-of-distribution and group performance	Do data selection and objective changes reshape the frontier?
Mixed-modal generation	Per-modality tokens, mixture ratios, tokenizers, shared capacity	Unimodal degradation and joint-task capability	Can competition boundaries extrapolate across combinations and ratios?
Diffusion generation and search	Training compute, generation calls, search and verifier costs	Diversity, prompt compliance, independent preference evaluation	When does more search begin to overoptimize the proxy objective?
Robotics	Environments, objects, tasks, embodiments, independent trajectories	Closed-loop success and generalization to unseen conditions	Do single-task relationships extend to multiple tasks and new platforms?

The next update should not merely add larger model releases. Evidence more likely to change the narrative includes a prediction succeeding or failing on a larger held-out experiment; a new data or architecture choice changing the bottleneck at the same budget; evaluation across objectives or groups overturning the ranking implied by average scores; or a method turning previously expensive feedback into a scalable signal. These are the results that can explain why the next research branch emerges.

Primary sources and reading scope are recorded in the new metadata and verification log (Chinese). For Aghajanyan, Ma, and Lin, the complete main text and all appendix text, tables, and captions have been read; not every image has been visually inspected, and the experiments have not been reproduced. Cherti received targeted reading of the main text. The September 21 update adds targeted reading of Figure's official report, recorded in editorial sources. Other new nodes above explicitly retain abstract-only candidate status; existing sources retain the reading records in extensions.json.

Community Radar: Which Debates Should Change the Scaling-Law Survey?

Initial search date: **September 16, 2026**. This round established **18 traceable records: 4 from X, 6 from YouTube, and 8 from Reddit**. Eight meet the screening threshold, seven provide instructional or contrasting context, and engagement counts remain unverified for three X posts. The date identifies this search round; **it does not mean that views or votes in the search index are live counts from that day**.

Community discussion exposes unresolved questions among researchers and practitioners. Technical conclusions still require papers, author code, and official reports: high view counts indicate reach, not correctness, and vigorous debate is not independent replication. Full fields appear in community.json; searches and leads not included appear in community-search-log.md.

The September 21, September 28, October 12, and October 26 additions bring the cumulative record count to 22. Earlier counters retain their original observation dates; see Sections 6-8 and 10 for the new records.

1. Operationalizing “High Attention”

The initial round uses explicit, conservative editorial thresholds: a YouTube search-index view count of at least 100,000 or a Reddit post score of at least 100 receives the label high_attention_proxy. This is a signal for reading priority, not a statistical definition of popularity or an exhaustive platform ranking. For X, an explicitly labeled view count of at least 100,000 provides the same priority signal. Only X-04 currently has an indexed original-post view count, below that threshold; the other three remain unverified. Counts claimed by third-party mirrors are not entered as X metrics.

context_below_screening_threshold is retained for items that clarify technical disagreements, such as misunderstandings of Chinchilla optimality and rebuttals to the Apple paper. This captures broadly circulated topics while preserving less prominent but necessary counterarguments. The threshold controls search priority, not scientific value.

Missing counts are recorded as null, not zero. Reddit scores differ from total likes and comment counts, and an upvoted reply cannot supply the parent post's metrics. Reliable total comment counts are unavailable in this round, so “visible attention signals” is more accurate than a claim to have measured the full intensity of discussion. Views, likes, and scores are neither summed across platforms nor combined into a universal ranking.

2. Observed Examples of Circulation

All counts below are **index snapshots returned during searches on September 16, 2026**; the time when the counts themselves were collected is unknown. YouTube inspection covered metadata and descriptions, not complete video viewing or comment analysis.

ID / platform	Post or video	Visible attention signal	Status and role in the survey
YT-02 / YouTube	Dave Plummer explains Deepseek R1, 2025-01-27	2,436,922 views; 128,000 likes	Meets threshold; a lead into R1 efficiency narratives
YT-03 / YouTube	Fireship: Did DeepSeek R1 just pop the AI bubble?, 2025-01-27	3,884,394 views; 152,000 likes	Meets threshold; distinguish efficiency gains from claims that scaling has failed
YT-01 / YouTube	AI Coffee Break: s1 / “wait…”, 2025-03-23	5,544 views; 335 likes	Below threshold; a technical explanation example

ID / platform	Post or video	Visible attention signal	Status and role in the survey
RD-02 / Reddit	DeepSeek R1 has been officially released!, 2025-01-20	Score +301	Meets threshold; benchmarks, practical coding, and distillation
RD-03 / Reddit	Discussion of Apple's The Illusion of Thinking, 2025-06-06	Score +103	Meets threshold; reasoning boundaries and evaluation confounds
RD-04 / Reddit	Discussion of the rebuttal to The Illusion of Thinking, 2025-06-14	Score +58	Below threshold; a necessary alternative interpretation
RD-01 / Reddit	New Llama scaling laws?, 2024-08-12	Score +48	Below threshold; common misunderstandings of Chinchilla
RD-05 / Reddit	There Will Be a Scientific Theory of Deep Learning, 2026-04-24	Score +263	Meets threshold; empirical regularities versus mechanistic theory
RD-06 / Reddit	Thoughts About Scaling Law - Z.ai, 2026-08-19	Score +519	Meets threshold; new claims await primary-source verification

Additional records cover research talks and engineering reproduction discussions directly relevant to scaling laws. The value of a lecture does not depend on crossing an attention threshold; only metadata and descriptions were verified for all videos.

ID / platform	Post or video	Indexed attention signal	Research connection
YT-04 / YouTube	Jared Kaplan's scaling talk at YC, published 2025-07-29	60,347 views; 965 likes	How a researcher connects pretraining, RL, and compute efficiency; the talk took place on 2025-06-16
YT-05 / YouTube	Stanford CS336 2026 Lecture 9, 2026-04-30	12,604 views; 173 likes	An entry point into foundational course material
YT-06 / YouTube	Stanford CS336 2026 Lecture 11, 2026-05-19	8,699 views; 128 likes	A later lecture in the same course
RD-07 / Reddit	HF authors share search reproduction and DVTS, 2024-12-16	Score +507	High attention; compare generators, verifiers, and costs together
RD-08 / Reddit	Discussion of o3 and test-time scaling, 2024-12-20	Score +142	High attention; returns to budget and local deployment constraints

X posts are listed separately so that author authority is not confused with engagement:

ID	Original post and associated primary source	Scope verified in this round
X-01	Niklas Muennighoff announces s1; author repository, paper	The author repository links directly to the post; the original returned 403; publication time, likes, replies, reposts, and views remain null
X-02	DeepSeek announces R1-Lite-Preview; official announcement	The post URL is cited by s1; the official announcement was readable in the search index; the post returned 403; all counts remain null
X-03	Jie Tang: Thoughts About Scaling Law; associated official blog	Reddit and secondary coverage trace back to the same post; X returned 403 and the official blog did not return readable body text; all counts remain null
X-04	A post circulating the HF search results, 2024-12-18	The original-post index shows 7,143 Views; the direct page returned no body text; unlabeled numbers are not interpreted as likes or reposts. This is the same research event as RD-07

3. Turn Discussion into a Problem-Driven Research Narrative

3.1 Does Continued Training of Smaller Models Overturn Chinchilla?

RD-01 is useful because it exposes a confusion between objectives: fixed FLOPs for one training run and a fixed deployment size serving many requests define different optimization problems. The survey should move naturally from allocating parameters and data under a training budget to allocating resources across training and serving. Chinchilla provides an empirical model for the former; Beyond Chinchilla-Optimal incorporates inference demand and examines extrapolation errors at very high tokens-per-parameter ratios.

The resulting connection is **an objective that omits deployment cost → a revised cost objective → longer training of smaller models becomes reasonable → fits require recalibration in extreme regimes**. It is not simply a small-model counterexample overturning an old law. The primary sources support this chain; Reddit identifies an accessible entry point into the confusion.

3.2 If R1 or s1 Is Inexpensive, Does More Compute Stop Mattering?

The two R1 videos with millions of indexed views and the release discussion show how rapidly methodological efficiency and cost narratives circulate. They do not independently establish that scaling has failed. The DeepSeek-R1 report connects RL training, inference behavior, and capability transfer to smaller models. s1 offers another route to controlled test-time computation through an existing Qwen base, selected reasoning trajectories, and budget forcing.

The next question worth integrating is: **should an additional budget go to base-model pretraining, RL, teacher-trajectory generation, or extra computation on each test request?** A small count of SFT examples is not a substitute for full-pipeline cost accounting. R1-Lite-Preview and the released R1 also cannot be treated as the same experiment. This radar does not extract training-cost figures or leaderboard conclusions from news videos.

3.3 Under What Conditions Does More Thinking Stop Helping?

Apple's original study distinguishes low-, medium-, and high-complexity behavior in controlled puzzles. Lawsen's Comment v2 raises confounds involving output length, scoring, and unsolvable instances. The RD-03/RD-04 debate should become a set of verifiable questions: are budgets comparable, are instances solvable, does the answer representation require exponentially long outputs, and does tool use change the task definition?

This branch cannot be compressed into either “LLMs cannot reason” or “the rebuttal proves there are no boundaries.” The experiment and response constrain the scope of their respective evidence; the response's second version also notes corrections to its earlier version. A review must retain versions and distinguish **the return curve for more computation** from **a failure point under one evaluation protocol**.

3.4 A Question Added in 2026: Can Empirical Regularities Become Mechanistic Explanations?

The 2026 position paper linked by RD-05 advocates connecting solvable models, tractable limits, macroscopic empirical regularities, hyperparameter theory, and universal phenomena. It motivates a theoretical branch asking why particular exponents arise and when they transfer. It is a research agenda, not evidence that a unified theory is complete.

RD-06 and X-03 are recent candidates connecting deployment conditions, MoE, and post-training budgets. The associated official blog did not provide readable body text in this search round, so **we record the questions without adopting specific claims about new-model performance or architectural mechanisms**. Appearance on two platforms indicates cross-platform circulation, not two independent technical validations.

3.5 What Exactly Does “3B Beats 70B” Compare?

RD-07 provides a more useful research conversation than a repeated headline. After the authors shared open-model search experiments, readers asked how large the verifier was and whether 256 candidates actually saved compute relative to one generation from a larger model. The author replied that a strict FLOPs comparison had not been completed and suggested that extensive sampling with the 3B system might be less compute-efficient than 70B, while its memory requirements made deployment more accessible. This is an author's clarification of their own experiment and should still be read alongside the technical material. Original post and author replies.

In the authors' technical article, the generator, verifier, and search algorithm together constitute the evaluated system. The case raises the question of **how to combine generation and verification within memory and acceptable-latency constraints**, rather than demonstrating that parameters no longer matter. It shares the cost-accounting issue in architecture and deployment and directly connects to search allocation in inference-time budgets.

RD-08 connects these changes to public discussion of o3, but community speculation about undisclosed training procedures lacks sufficient evidence. The survey retains the explicit budget questions, not inferences such as “pretraining has stopped improving.” X-04's headline similarly compresses experimental conditions. A complete update should therefore connect **headline → author experiment →**

reader questions → author qualifications → survey revision, rather than merely adding a popular link.

4. Preserve Comparability Across Updates

Each update begins with searches over the preceding 7-30 days, followed by author responses, corrections, and independent reproductions. Merge records of the same event using DOI, arXiv ID, and canonical URL. Preserve earlier observations and append new counts, access status, and evidence timestamps; do not overwrite historical observations with current values.

Once stable counts are available, medians, quantiles, and growth rates could be reported within a platform, time window, and comparable set of channels or communities. The current selective seed sample does not support reliable “top 10%” rankings or engagement-growth estimates. Platform APIs or readable pages authorized by the user may later fill gaps in comments, reposts, and views. Failed access continues to produce null, not guessed values.

A candidate enters the technical narrative only after the primary source establishes which earlier limitation it addresses, what the experiment controls, and which trade-offs remain. Updates prioritize counterevidence and corrections that change existing conclusions, then independent reproduction, then new allocation evidence, and only afterward high-exposure opinion. Reposts that do not change understanding update the radar rather than repeatedly expanding the prose.

5. Limitations of This Round

This is a purposive seed search concentrated on English-speaking communities, language models, and selected turning points. Search visibility, channel size, community population, emotionally framed headlines, and accumulated attention all affect counts. X login restrictions and missing YouTube comments make platform coverage uneven. Chinese and other non-English discussions, negative results, smaller research groups, and work outside social media may be underrepresented. The table supports decisions about what to verify next, not a ranking of the most popular papers across the field or a claim of community consensus.

6. Observation Added on 2026-09-21: The Boundary of Zero-Shot Transfer in Robotics

This search covered September 2-21, 2026. The observation below is a separate addition; the preceding September 16 sample and counts retain their original meaning. No newly verified high-attention item was added this round. One discussion below the screening threshold is included for its value in clarifying technical claims.

ID	Discussion and associated primary source	Publication-date evidence	Visible attention and classification
RD-09	The Birth of the Robotics Scaling Law (Helix 2.5); Figure's official report	This round's indexed snapshot explicitly displayed September 17, 2026	Indexed score of 52, below the editorial Reddit threshold of 100; counter measurement time unknown

The record preserves two distinct observations. During retrieval on September 21, the lead researcher saw an indexed result with an explicit date and a score of 52. Directly opening the page exposed only the relative time “5h ago,” with no readable counter. Its reference time was unavailable, so it cannot revise the publication date or be combined with the indexed result to infer live engagement changes. Replies were not read comprehensively, and no community consensus is inferred.

The headline raises a research question: **which distribution shift do the gains from more robot data actually cross?** The official report defines zero-shot in terms of unseen environments and objects; tasks are fine-tuned using data collected elsewhere. Its control uses random initialization under the same downstream settings, so this experiment does not directly establish superiority over general-purpose VLM pretraining. The data-scaling curve measures action-prediction loss, which cannot simply be equated with a power law for closed-loop task success. These qualifications come from the report itself; independent replication remains necessary.

Related X and YouTube URLs were corroborated through public cross-links, but this round did not obtain verifiable platform publication dates or engagement metrics. They remain search-log candidates rather than high-attention evidence. See the weekly search log for queries, access limitations, and excluded candidates, and the maintained community records for both RD-09 observations.

7. Observation Added on 2026-09-28: Separate Current Evaluation Failures from Long-Term Extrapolation

The indexed result for RD-10: RSI discussion explicitly gives September 14, 2026 and a score of 288, meeting this radar's editorial Reddit threshold. It was retrieved on September 28; the counter's measurement time is unknown. The title is the poster's interpretation. Only the abstract and version history of the linked study v2 were checked this round: observations from two cases cannot establish that RSI is impossible. The disagreement motivates recording models, tasks, budgets, and human intervention before extrapolating; comments are not technical evidence. Direct page reading exposed no counter, and no new verifiable X or YouTube attention metrics were obtained. Historical counts remain unchanged.

8. October 12, 2026 Addition: Small Parameter Counts Do Not Establish Low System Cost

The search window runs from September 21 through October 12, 2026.

ID	Discussion and primary sources	Publication-date evidence	Visible attention and classification
RD-11	FrogNano 4B release discussion; paper v5, official model card	Index explicitly shows 2026-10-02	Indexed score 223; observed October 12, measurement time unknown; meets the Reddit editorial screening threshold

The discussion connects local deployment of a small model with task curricula and tool interfaces. The official model card distinguishes not imitating stronger-model solver trajectories from still using AI-generated tasks, reference patches and hidden tests. This motivates tracking generation, training and

verification costs alongside interface conditions; small parameter count does not establish compute optimality. This round checked the abstract, version history, model card and selected method passages, without reproduction. Version 1 appeared September 7 and v5 was revised October 5; a complete revision diff was not performed. The October 2 post predates v5 and cannot be interpreted as feedback on that revision. The direct page exposed no counters, and selected replies do not establish community consensus. The item is included only in the radar and source index.

X post identities, dates, or labeled counters remain unverified. This round obtained no qualifying new YouTube sample, and the tested prior candidate failed to open directly. These access limits do not establish an absence of new discussion. See the complete weekly log.

9. Search on 2026-10-19: No New Qualifying Observation

The 2026-09-28 to 2026-10-19 window yielded no formally eligible addition across 32 queries covering the three platforms. All 21 existing records and historical counters remain unchanged. Goodfire discussion stays a candidate: the Reddit original lacked a verifiable absolute date and score, the X original returned 403, and third-party view claims were not adopted as platform metrics. A YouTube transcript date does not establish the upload date; missing counters remain empty. These access and verification limits do not establish an absence of discussion or warrant changes to technical conclusions about superposition or scaling. See the complete search record.

10. October 26 Addition: Do Readable Labels Explain Computation?

The October 5-26 window produced one adopted addition from 24 queries across the three platforms. The previously unverified Goodfire thread now has an explicit October 15 date and indexed score of 887. This is not a live count; the existing 21 historical observations remain unchanged. The author's essay presents a working hypothesis, makes readability proportions threshold-dependent, and still describes its technical companion as forthcoming. Leo Gao's October 17 response argues for testing explanations through downstream behavior after replacing activations, and distinguishes explaining neurons from understanding computation between them. This dispute over evidence does not establish that interpretability is solved or prove a universal scaling mechanism. The X original still returned 403; the previously identified YouTube candidate returned Cache miss. Missing metrics do not mean zero attention. See the complete search record.

Synthesis: How Has the Research Focus Shifted, and What Evidence Matters Next?

Reducing scaling laws to “larger models perform better” cannot explain why identical compute budgets produce models of different quality, why smaller models may warrant longer training, or why longer thinking sometimes reduces accuracy. The preceding chapters show several overlapping shifts in the field's focus.

First, **from observed improvements to predictable returns**. Early learning curves made the benefits of larger experiments estimable, allowing smaller experiments to inform larger investments. The remaining issues concern functional form and extrapolation range: low fitting error alone cannot guarantee reliable

predictions across orders of magnitude. Second, **from increasing one resource to allocating resources jointly**. Kaplan, Chinchilla, and reanalysis studies examined the relationship between parameters, data, and training computation, showing that the optimal configuration depends on experimental design and cost conventions. Hestness; Kaplan; Chinchilla; Porian.

Third, **from nominal resources to effective resources**. Repeated tokens are not new information; selected data need not be more valuable for every task; total MoE parameters do not equal active computation per step; and FLOPs are not latency or monetary cost. Resource optimization must therefore specify the data distribution, architecture, hardware, and model lifecycle. Data-constrained scaling; DataComp-LM; Routed LMs; Beyond Chinchilla.

Fourth, **from predicting average loss to optimizing task systems**. Inference-time sampling and search expand the candidate pool, verifiers turn that potential into selectable answers, and post-training changes how models explore and use their budgets. These approaches complement one another, while verifiable rewards, selector reliability, and task difficulty become new bottlenecks. Existing results support conditional cost frontiers, not a fixed universal exchange rate among pretraining, RL, and inference compute. Snell; DeepSeek-R1; s1.

These shifts do not imply that earlier approaches have been completely replaced. Better base models can increase the probability of successful RL exploration; better data can improve both pretraining and verification; and reliable inference trajectories can feed into distillation. These cross-branch relationships explain more than a straight narrative in which pretraining ends and inference takes over.

Turn Debates into Testable Questions

Common debate	A testable formulation	Evidence that would change the narrative
Has scaling hit a wall?	For which objective, budget range, data distribution, and system configuration do returns diminish?	Reproduction across scales with consistent accounting, plus counterfactual comparisons after changing constraints
What happens when data run out?	At fixed unique data, total compute, and target distribution, what gains come from repetition, selection, and synthesis?	Accounting for generation and selection costs, real-data controls, and tail evaluation
Can smaller models replace larger ones by thinking longer?	Which problems permit such substitution at equal end-to-end budget and latency?	Complete compute accounting, difficulty stratification, realistic selectors, and reproduction across tasks
Does RL create new capabilities?	How does the solvable set change after matching prompts, sampling, length, and checkers?	Full pass@k curves, disclosure of external information, and replication across base models
Does long CoT establish reasoning?	Does increasing the budget causally improve delivered outcomes, and does the conclusion survive interface changes?	Randomized budget interventions, separation of action execution from algorithmic representation, and tool-cost accounting
Do multimodal models inherit language-model laws?	How do optimal proportions of visual data, encoders, and language backbones change with the objective?	Independent variation of each axis, with distribution-specific and worst-group results

This table is a research agenda, not a claim that every question has been answered. It also sets a threshold for maintenance: a new benchmark point can first enter the reference index; evidence that changes an assumption, supplies a counterexample, establishes a new cost convention, or repairs a critical failure should alter the corresponding connection in the survey.

What Remains Missing

This edition extends the discussion to diffusion models and robotics under their distinct data and feedback constraints. Scientific foundation models, long-horizon agents, hardware energy use, and economic scaling still need fuller synthesis. Limited disclosure of data and costs in industrial training often prevents direct comparison across organizations. Much of the 2026 literature awaits further replication, and community search indexes have temporal lags and visibility biases.

Future maintenance should first upgrade verified abstract-only candidates into reviews of methods and evidence, track publicly reproducible post-training and inference cost frontiers, and expand tool-use and cross-distribution evaluation. The result should be an increasingly accurate map of research questions, not merely a longer list of papers.

Cross-Chapter Relationships Matter More Than a Single Optimal Exponent

At least four interactions connect data, architecture, feedback, and evaluation. Data selection changes the training distribution and therefore the regions a verifier can judge reliably. Architecture and caching change how many candidates fit within an inference budget, moving the practical cost frontier of search. Reliable search trajectories can enter post-training, converting a one-time inference expense into policy capabilities that can subsequently be amortized. Changes in evaluation protocol may then reduce or erase those gains on new tasks. These relationships organize the evidence reviewed above; they are not a single precise dynamical model shared by all the papers.

This yields a more concrete way to judge new work: does it reach a larger scale on an existing curve, change the curve itself, change the price of available resources, or change the objective being measured? All four can matter, but they require different controlled comparisons. Updates should first revise the affected question and assumptions, then add the relevant papers. Later readers can then understand why a conclusion held, why its scope subsequently narrowed, and which branches remain worth pursuing.

Variables, Metrics, and Cost Conventions

The equations in this survey explain mechanisms and support experimental comparisons. The same symbol can have different definitions in different papers; the conventions below are reading aids, not replacements for the original definitions.

Symbol or metric	Convention in this survey	Conditions that comparisons must retain
N	Number of model parameters	Whether embeddings are included; report both total and active parameters for sparse models
D	Total training tokens processed	Tokenizer, unique data, repeated epochs, and context length
U	Size of the unique training data	Token, document, and semantic deduplication define different quantities
C	Compute budget with a stated accounting boundary	Whether training or inference, forward and backward passes, routing, verifiers, tools, and data generation are included
L	Loss under a specified distribution and scoring rule	Cross-entropy, normalized error, and FID cannot directly share fitted parameters
Q	Number of requests over the model's lifecycle	Request length, concurrency, target quality, hardware, and latency requirements
pass@1	Probability, or an estimate, that one sample is correct	Sampling temperature, prompt, length limit, and checker

Symbol or metric	Convention in this survey	Conditions that comparisons must retain
pass@k	Probability, or an estimate, that at least one of k candidates is correct	Usually evaluated using a checker; does not imply that a deployed selector can identify the correct candidate
maj@k	Accuracy after majority aggregation of candidate answers	Answer normalization, correlated errors, and tie handling
best-of-N	A strategy that selects candidates with a scorer	N is a strategy hyperparameter; final accuracy depends on both the generator and scorer
GPU-hours	Total time occupying accelerators	Device type, utilization, precision, communication, and waiting overhead
Latency / throughput	Waiting time per request / completed work per unit time	Batching can make their optimization objectives diverge

The common language-model approximation C_(train) ~= 6ND is a training-cost model for dense Transformers. Estimating inference compute from parameters and generated tokens likewise has conditions. Long-context attention, KV caches, prefill and decode, sparse routing, and hardware utilization require separate examination. Two systems generating the same token count do not necessarily use equal computation, cost, or time. Chinchilla; HF experiments on inference compute with open models.

“Compute-optimal” must specify both the objective and constraints. Minimizing validation loss at fixed training FLOPs, maximizing quality at fixed model size, and minimizing lifecycle cost for a specified serving demand are three different problems. Beyond Chinchilla-Optimal.

“Overtraining” here means training beyond a particular training-compute-optimal allocation; it does not automatically imply statistical overfitting. “Emergence” describes the shape of change under a measurement procedure; it does not by itself prove a discrete transition in an internal mechanism. Nor does every larger model with a higher score establish a “scaling law”: a meaningful relationship also needs resource definitions, multiple scales, a functional form, a fitting range, and extrapolation tests. Schaeffer et al..

“Capability boundaries” are particularly easy to conflate. Failure within a finite sampling budget means success was not observed under that protocol and budget; it does not prove that the probability of a correct solution is exactly zero. Conversely, one success after enormous sampling does not establish reliable delivery within a practical budget. The text therefore discusses candidate coverage, candidate selection, and final reliability separately. Research on RL capability boundaries; inference-time compute allocation.

The Reference Survey's Method and the Gaps in Our First Edition: A Full-Text Reading Audit

Audit date: 2026-09-16. The reference is JonnesLin/post-training-survey. This document analyzes its organization and how that organization is implemented; it does not treat the technical conclusions it contains as independently verified primary-source facts.

Reading Scope and Reproducible Counts

The first round sampled chapter structure only and did not meet the user's request for careful reading. In this round, the locally obtained Chinese source was read in full, in chunks: the 72-line preface, 311-line introduction, 520-line conclusion, and all body text, tables, and figure source in the seven chapters below. Passages truncated in terminal output were read separately. The design document (130 lines) and implementation plan (431 lines) were also read in full. This is a full-text reading, not an independent verification of every cited paper. The English edition, research notes, and the complete set of proposal attachments are not counted as read.

The counts come directly from the source snapshot. Chinese characters are counted using Unicode U+4E00-U+9FFF, excluding English, formulas, and punctuation. Citation counts merge and deduplicate keys appearing in cite/citep/citet/landmark. Source-character counts must not be presented as PDF page counts.

Chinese chapter	Lines	Chinese characters	Unique citation keys in chapter	landmark occurrences	crossref occurrences
01 SFT	865	7,550	54	8	11
02 Preference	1,040	7,389	61	12	7
03 Safety	915	8,956	49	8	11
04 Reasoning	1,080	8,977	51	12	11
05 Multimodal	1,123	8,632	103	8	15
06 Agentic	960	11,820	56	10	22
07 Efficiency	809	6,539	49	11	13

The seven chapters total 6,792 lines, 239,108 source characters, and 59,863 Chinese characters. references.bib contains **468 entries with unique keys**. All the Chinese chapters read actually use **440 unique citation keys**, all present in the bibliography; another 28 bibliography entries do not appear in those chapters. The seven chapters contain 69 landmark markers in total, including repeated papers and shorter entries; this cannot be converted into 69 papers each receiving two pages of in-depth analysis. All Chinese chapters together contain 115 crossref occurrences. The seven main chapters have 21 Problem, 14 Solution, and 42 Open Problem boxes. These figures indicate the investment in coverage and organization, but cannot replace an assessment of factual accuracy.

Structural Choices Worth Adopting

The design document explicitly uses seven problem threads, rather than isolated introductions to seven methods. Each chapter proceeds through problem origins, foundational approaches, limitations and branches, convergence and frontiers, and open problems. Cross-references between chapters record the origins, migration, and dependencies of methods. The planned depth is 1-2 pages for landmarks, one paragraph for important follow-ups, and 1-2 sentences for minor work. The actual source does not mechanically deliver a fixed page count per paper; instead, key nodes combine problem formalization, method mechanisms, experimental comparisons, and new limitations.

The smallest unit of this writing style is a contestable research claim and its evidence, rather than an abstract with a link. The FLAN section first explains the construction of instructions and held-out task clusters, then separately discusses ablations involving the number of tasks, the number of models, and instructions. The reader therefore learns not just that FLAN works, but how the experiments distinguish task transfer from memorization of training data. The passage spans 52 lines and 1,757 source characters, including substantial English and 480 Chinese characters. It is followed by architectural and scale comparisons for T0, then real-user data in InstructGPT. These provide complementary evidence and should not be forced into a story in which the later paper fixes the earlier one.

DPO is a stronger example of mechanistic development. Lines 375-444 of the source begin with the KL-regularized objective, then present the optimal policy, reparameterization, cancellation of the partition function, the final preference loss, and an explanation of gradient weights. The subsequent variants are classified by the assumptions they modify, and the next branch turns to the mismatch between the offline distribution and the current policy. This progression places training simplification and insufficient sampling coverage in the same research conversation. A scaling-law survey should correspondingly develop the derivation of optimal allocation under a budget constraint, the effects of exponents on decisions, and why different fitting protocols lead to different decisions, rather than showing one formula and jumping to the next paper.

The Safety chapter does not impose a single timeline: CAI addresses scaling annotation, Safe RLHF addresses conflicting objectives, attack research continually exposes failure modes from outside, and representation engineering changes the level at which control is applied. The 75-line Safe RLHF section connects decoupled annotation, separate reward and cost models, CMDPs, dual variables, and iterative data collection. For scaling laws, data, architectures, verifiers, and system costs should likewise form parallel routes around different bottlenecks; not every arrow means that a successor defeats its predecessor.

The LLaVA section develops the visual interface, data generation, two-stage freezing and unfreezing, and evaluation over 92 lines and 3,255 characters, then compares information retention and token overhead with an instruction-aware Q-Former. The multimodal chapter's ending naturally connects perception and localization capabilities to screen interaction in the next chapter. This suggests that a scaling survey should move from visual-token budgets to robotic trajectories and environment coverage by first explaining how the unit of measurement has changed, then discussing whether transferable relationships exist.

The Efficiency chapter is not a list of LoRA methods either. It starts from three costs-memory, annotation, and iteration time-derives low-rank updates, explains QLoRA's three complementary components, then

expands to data selection, distillation, and RL overhead. Its GRPO discussion leaves the formulas to the Preference chapter and explains only their cost implications here, avoiding duplication across chapters. When the same scaling paper appears in multiple chapters, each appearance should likewise make clear the new question it addresses, rather than repeat its abstract.

How Figures, Boxes, Tables, and Conclusions Contribute to the Argument

Problem boxes identify concrete tensions left by current methods; Solution boxes condense mechanisms; Evolution arrows should carry the reason for a relationship; Open Problem boxes should identify what the evidence has not yet ruled out. These elements serve the reasoning in the prose rather than replace it. Columns in chapter-end comparison tables change with the question: the preference chapter concerns learned reward models and online versus offline training; the vision chapter concerns the bridge; the agent chapter concerns action spaces; and the efficiency chapter concerns resource types. A scaling survey should not impose “title-year-contribution” on every chapter. It should compare independent variables, controlled variables, metrics, extrapolation ranges, and cost accounting.

The introduction establishes field boundaries, assumptions about readers, the seven threads, and a cross-thread map. The preface offers three entry points: linear reading, jumping between threads, and navigation through the overview. The conclusion is more than a chapter summary: it first presents broad trends, then abstracts recurring patterns such as reformulation, data quality, self-improvement, technique migration, and evaluation bottlenecks, and finally derives joint research questions from those patterns. Our conclusion must also reason back from the evidence in the preceding chapters, avoiding new claims that appear without an established basis in the main text.

The generation plan's actual sequence starts with typesetting and semantic macros, followed by the introduction and overview figure, chapter-by-chapter research and expansion of a unified bibliography, and final checks of citations, figures, and compilation. It requires looking up papers' methods, formulas, and results. The execution commands in the file document the reference author's workflow; they are not instructions for this project. Parallel division of labor is an implementation option for this project, but the plan alone does not establish that the reference repository was actually generated in parallel.

Where the First Edition Fell Short and How to Assess the Rewrite

The first edition's 45 research sources made it closer to a sourced reading guide; insufficient coverage was only one symptom. The main gaps included theoretical mechanisms reduced to short passages, vision and embodied AI treated mainly as pointers to extensions, missing comparisons of the assumptions behind different extrapolation models, insufficient analysis of how data, tasks, and evaluation protocols affect conclusions, and cross-chapter connections that were mostly verbal reminders. Individual chapters in the reference already contain 49-103 unique citation keys, so it was inappropriate to describe the first edition as having comparable depth.

The unit of acceptance for the rewrite should be a complete chain of argument: each core thread needs landmarks developed at the mechanism level, comparison papers that explain disagreements, counterexamples or failure analyses with explicit scope, passages that reconnect branches, and concrete open questions. The theory chapter must distinguish different mechanisms that produce power laws. The capability chapter must distinguish loss, metrics, and actual task utility. The multimodal and embodied

chapters must redefine data and cost units. New candidates do not automatically count toward synthesized coverage, abstract verification does not pass for full-text reading, and bibliography size is not the criterion for completion of quality work.

The Reference Also Requires Critical Reading

Its structure is worth learning from, but visible internal inconsistencies and overgeneralizations remain. For example, the multimodal chapter gives different versions of LLaVA-OneVision's three training stages around lines 498 and 528. The Preference chapter first says GRPO does not require a separate reward model, then explicitly allows rewards from an ORM or PRM. The Reasoning and Preference chapters explain DAPO dynamic sampling differently. The SFT chapter presents data-constrained language-modeling results as an instruction-tuning relationship, while the Efficiency chapter states that the work primarily studies pretraining. These are internal comparisons of the source, not a completed paper-by-paper adjudication on the author's behalf.

Especially dangerous for this topic are describing phi-1's advantage on particular tasks as “breaking scaling laws,” or directly calling benchmark improvements over calendar time an independent scaling law. Efficiency gains after changing the training distribution do not automatically overturn a relationship under a fixed distribution; years cannot substitute for controllable resource variables either. The rewrite should retain the first edition's careful verification of sources, versions, and applicability while adding the density of mechanisms and research conversation found in the reference. Adopting a structure and verifying technical facts must be done separately.

References

Full metadata are available in the reference index, BibTeX, and structured catalog. Technical claims in the text link to their primary sources.

Year	Work and primary source	Reading depth	Version / type	Chapters
2017	Deep Learning Scaling is Predictable, Empirically	Selected full-text sections	v1 / Paper / preprint	01
2018	Universal Transformers	Selected full-text sections	v3 / Paper / preprint	05
2019	A Constructive Prediction of the Generalization Error Across Scales	Abstract / metadata	v2 / Paper / preprint	Catalog only
2019	Deep Double Descent: Where Bigger Models and More Data Hurt	Abstract / metadata	v1 / Paper / preprint	Catalog only
2019	Fast Transformer Decoding: One Write-Head is All You Need	Selected full-text sections	v1 / Paper / preprint	03

Year	Work and primary source	Reading depth	Version / type	Chapters
2019	Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism	Selected full-text sections	v4 / Paper / preprint	03
2019	ZeRO: Memory Optimizations Toward Training Trillion Parameter Models	Selected full-text sections	v3 / Paper / preprint	03
2020	A Neural Scaling Law from the Dimension of the Data Manifold	Abstract / metadata	v1 / Paper / preprint	06
2020	An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale	Abstract / metadata	v2 / Paper / preprint	07
2020	Feature Learning in Infinite-Width Neural Networks	Selected full-text sections	v3 / Paper / preprint	01
2020	GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding	Selected full-text sections	v1 / Paper / preprint	03
2020	Language Models are Few-Shot Learners	Selected full-text sections	v4 / Paper / preprint	01
2020	Scaling Laws for Autoregressive Generative Modeling	Selected full-text sections	v2 / Paper / preprint	07
2020	Scaling Laws for Neural Language Models	Selected full-text sections	v1 / Paper / preprint	01
2020	Spectrum Dependent Learning Curves in Kernel Regression and Wide Neural Networks	Abstract / metadata	v7 / Paper / preprint	06
2021	Deduplicating Training Data Makes Language Models Better	Selected full-text sections	v2 / Paper / preprint	02
2021	Explaining Neural Scaling Laws	Selected full-text sections	See metadata / Paper / preprint	06
2021	High-Resolution Image Synthesis with Latent Diffusion Models	Abstract / metadata	v2 / Paper / preprint	07
2021	Improved Denoising Diffusion Probabilistic Models	Abstract / metadata	v1 / Paper / preprint	07
2021	Learning Curve Theory	Full text	v1 / Paper / preprint	06

Year	Work and primary source	Reading depth	Version / type	Chapters
2021	Learning Transferable Visual Models From Natural Language Supervision	Selected full-text sections	v1 / Paper / preprint	07
2021	Masked Autoencoders Are Scalable Vision Learners	Abstract / metadata	v3 / Paper / preprint	07
2021	Scaling Language Models: Methods, Analysis & Insights from Training Gopher	Selected full-text sections	v2 / Paper / preprint	01
2021	Scaling Laws for Transfer	Abstract / metadata	v1 / Paper / preprint	Catalog only
2021	Scaling Vision Transformers	Selected full-text sections	v2 / Paper / preprint	07
2021	Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity	Selected full-text sections	v3 / Paper / preprint	03
2021	Training Verifiers to Solve Math Word Problems	Selected full-text sections	v2 / Paper / preprint	04
2022	A Solvable Model of Neural Scaling Laws	Abstract / metadata	v1 / Paper / preprint	06
2022	Beyond neural scaling laws: beating power law scaling via data pruning	Selected full-text sections	See metadata / Paper / preprint	02
2022	Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models	Abstract / metadata	v3 / Paper / preprint	06
2022	Broken Neural Scaling Laws	Selected full-text sections	v17 / Paper / preprint	06
2022	Chain-of-Thought Prompting Elicits Reasoning in Large Language Models	Abstract / metadata	v6 / Paper / preprint	05
2022	Efficiently Scaling Transformer Inference	Selected full-text sections	v1 / Paper / preprint	03
2022	Emergent Abilities of Large Language Models	Abstract / metadata	v2 / Paper / preprint	06

Year	Work and primary source	Reading depth	Version / type	Chapters
2022	FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness	Selected full-text sections	v2 / Paper / preprint	03
2022	Inverse scaling can become U-shaped	Abstract / metadata	v5 / Paper / preprint	Catalog only
2022	OPT: Open Pre-trained Transformer Language Models	Selected full-text sections	v4 / Paper / preprint	01
2022	PaLM: Scaling Language Modeling with Pathways	Abstract / introduction	v5 / Paper / preprint	Catalog only
2022	RT-1: Robotics Transformer for Real-World Control at Scale	Abstract / metadata	v2 / Paper / preprint	07
2022	Reproducible scaling laws for contrastive language-image learning	Selected full-text sections	v2 / Paper / preprint	07
2022	Revisiting Neural Scaling Laws in Language and Vision	Selected full-text sections	v2 / Paper / preprint	06
2022	STaR: Bootstrapping Reasoning With Reasoning	Selected full-text sections	v2 / Paper / preprint	04
2022	Scalable Diffusion Models with Transformers	Abstract / metadata	v2 / Paper / preprint	07
2022	Scaling Laws for Reward Model Overoptimization	Selected full-text sections	v1 / Paper / preprint	04
2022	Self-Consistency Improves Chain of Thought Reasoning in Language Models	Abstract / metadata	v4 / Paper / preprint	05
2022	Solving math word problems with process- and outcome-based feedback	Selected full-text sections	v1 / Paper / preprint	04
2022	Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer	Selected full-text sections	v2 / Paper / preprint	01
2022	Training Compute-Optimal Large Language Models	Selected full-text sections	v1 / Paper / preprint	01, 02

Year	Work and primary source	Reading depth	Version / type	Chapters
2022	Training language models to follow instructions with human feedback	Selected full-text sections	v1 / Paper / preprint	04
2022	Unified Scaling Laws for Routed Language Models	Selected full-text sections	See metadata / Paper / preprint	03
2022	Will we run out of data? Limits of LLM scaling based on human-generated data	Selected full-text sections	v2 / Paper / preprint	02
2023	Are Emergent Abilities of Large Language Models a Mirage?	Selected full-text sections	v2 / Paper / preprint	06
2023	Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws	Selected full-text sections	See metadata / Paper / preprint	03
2023	Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models	Selected full-text sections	See metadata / Paper / preprint	04
2023	Data Selection for Language Models via Importance Resampling	Selected full-text sections	v3 / Paper / preprint	02
2023	DataComp: In search of the next generation of multimodal datasets	Selected full-text sections	v5 / Paper / preprint	07
2023	Direct Preference Optimization: Your Language Model is Secretly a Reward Model	Selected full-text sections	v3 / Paper / preprint	04
2023	DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining	Selected full-text sections	v4 / Paper / preprint	02
2023	Effective Long-Context Scaling of Foundation Models	Selected full-text sections	v3 / Paper / preprint	03
2023	Efficient Memory Management for Large Language Model Serving with PagedAttention	Selected full-text sections	v1 / Paper / preprint	03
2023	FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning	Selected full-text sections	v1 / Paper / preprint	03

Year	Work and primary source	Reading depth	Version / type	Chapters
2023	GPT-4 Technical Report	Selected full-text sections	v6 / Report	06
2023	GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints	Selected full-text sections	v3 / Paper / preprint	03
2023	Inverse Scaling: When Bigger Isn't Better	Abstract / metadata	v2 / Paper / preprint	Catalog only
2023	LLaMA: Open and Efficient Foundation Language Models	Selected full-text sections	v1 / Paper / preprint	01
2023	Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models	Selected full-text sections	v3 / Paper / preprint	05
2023	Let's Verify Step by Step	Abstract / metadata	See metadata / Paper / preprint	04
2023	Llemma: An Open Language Model For Mathematics	Selected full-text sections	See metadata / Paper / preprint	04
2023	Mamba: Linear-Time Sequence Modeling with Selective State Spaces	Selected full-text sections	v2 / Paper / preprint	03
2023	Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations	Selected full-text sections	v3 / Paper / preprint	04
2023	MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models	Selected full-text sections	v4 / Paper / preprint	04
2023	Open X-Embodiment: Robotic Learning Datasets and RT-X Models	Abstract / metadata	v9 / Paper / preprint	07
2023	Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling	Selected full-text sections	v2 / Paper / preprint	01
2023	RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control	Abstract / metadata	v1 / Paper / preprint	07
2023	Reinforced Self-Training (ReST) for Language Modeling	Selected full-text sections	v2 / Paper / preprint	04

Year	Work and primary source	Reading depth	Version / type	Chapters
2023	Scaling Data-Constrained Language Models	Selected full-text sections	See metadata / Paper / preprint	02
2023	Scaling Laws for Generative Mixed-Modal Language Models	Full text	v1 / Paper / preprint	07
2023	Scaling Vision Transformers to 22 Billion Parameters	Abstract / metadata	v1 / Paper / preprint	07
2023	Sigmoid Loss for Language Image Pre-Training	Abstract / metadata	v4 / Paper / preprint	07
2023	The Curse of Recursion: Training on Generated Data Makes Models Forget	Selected full-text sections	v3 / Paper / preprint	02
2023	The Quantization Model of Neural Scaling	Selected full-text sections	v3 / Paper / preprint	06
2023	TinyStories: How Small Can Language Models Be and Still Speak Coherent English?	Abstract / introduction	v2 / Paper / preprint	Catalog only
2023	Tree of Thoughts: Deliberate Problem Solving with Large Language Models	Selected full-text sections	v2 / Paper / preprint	05
2024	2 OLMo 2 Furious	Selected full-text sections	v3 / Paper / preprint	04
2024	A Dynamical Model of Neural Scaling Laws	Selected full-text sections	v4 / Paper / preprint	06
2024	CREAM: Consistency Regularized Self-Rewarding Language Models	Selected full-text sections	v5 / Paper / preprint	04
2024	Chinchilla Scaling: A replication attempt	Selected full-text sections	v2 / Paper / preprint	01
2024	Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance	Abstract / introduction	v2 / Paper / preprint	02
2024	Data Scaling Laws in Imitation Learning for Robotic Manipulation	Full text	v4 / Paper / preprint	07
2024	Data curation via joint example selection further accelerates multimodal learning	Abstract / metadata	v1 / Paper / preprint	07

Year	Work and primary source	Reading depth	Version / type	Chapters
2024	DataComp-LM: In search of the next generation of training sets for language models	Selected full-text sections	v4 / Paper / preprint	02
2024	DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search	Selected full-text sections	v1 / Paper / preprint	05
2024	DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model	Selected full-text sections	v5 / Report	03
2024	DeepSeek-V3 Technical Report	Selected full-text sections	v2 / Report	03
2024	DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models	Selected full-text sections	v3 / Paper / preprint	04
2024	DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models	Selected full-text sections	v1 / Report	03
2024	Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs	Selected full-text sections	v2 / Paper / preprint	05
2024	Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research	Selected full-text sections	v2 / Paper / preprint	02
2024	How Feature Learning Can Improve Neural Scaling Laws	Selected full-text sections	v2 / Paper / preprint	06
2024	How predictable is language model benchmark performance?	Abstract / metadata	v1 / Paper / preprint	Catalog only
2024	Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for LLM Problem-Solving	Selected full-text sections	v3 / Paper / preprint	05
2024	Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data	Selected full-text sections	See metadata / Paper / preprint	02
2024	Language models scale reliably with over-training and on downstream tasks	Selected full-text sections	v2 / Paper / preprint	01

Year	Work and primary source	Reading depth	Version / type	Chapters
2024	Large Language Monkeys: Scaling Inference Compute with Repeated Sampling	Selected full-text sections	v3 / Paper / preprint	05
2024	Learning to reason with LLMs	Official report sections	See metadata / Technical article	04
2024	Let's Think Dot by Dot: Hidden Computation in Transformer Language Models	Selected full-text sections	v1 / Paper / preprint	05
2024	MM1: Methods, Analysis & Insights from Multimodal LLM Pre-training	Abstract / metadata	v4 / Paper / preprint	07
2024	Mixtral of Experts	Selected full-text sections	v1 / Paper / preprint	03
2024	Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solvers	Selected full-text sections	v1 / Paper / preprint	05
2024	OLMo: Accelerating the Science of Language Models	Selected full-text sections	v4 / Paper / preprint	01
2024	Observational Scaling Laws and the Predictability of Language Model Performance	Selected full-text sections	v3 / Paper / preprint	06
2024	Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone	Abstract / metadata	v4 / Report	Catalog only
2024	Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking	Selected full-text sections	v2 / Paper / preprint	05
2024	ReST-MCTS: LLM Self-Training via Process Reward Guided Tree Search	Selected full-text sections	v3 / Paper / preprint	05
2024	RegMix: Data Mixture as Regression for Language Model Pre-training	Selected full-text sections	v2 / Paper / preprint	02
2024	Resolving Discrepancies in Compute-Optimal Scaling of Language Models	Selected full-text sections	v4 / Paper / preprint	01
2024	Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters	Selected full-text sections	v1 / Paper / preprint	05

Year	Work and primary source	Reading depth	Version / type	Chapters
2024	Scaling Laws for Downstream Task Performance of Large Language Models	Full text	v3 / Paper / preprint	06
2024	Scaling Laws for Fine-Grained Mixture of Experts	Selected full-text sections	v1 / Paper / preprint	03
2024	Scaling Rectified Flow Transformers for High-Resolution Image Synthesis	Abstract / metadata	v1 / Paper / preprint	07
2024	Scaling test-time compute with open models	Full text	Web snapshot / Technical article	05
2024	Self-Rewarding Language Models	Selected full-text sections	v3 / Paper / preprint	04
2024	The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale	Selected full-text sections	v2 / Paper / preprint	02
2024	The Llama 3 Herd of Models	Selected full-text sections	v3 / Report	01
2024	Training Large Language Models to Reason in a Continuous Latent Space	Selected full-text sections	v4 / Paper / preprint	05
2024	Tulu 3: Pushing Frontiers in Open Language Model Post-Training	Selected full-text sections	See metadata / Paper / preprint	04
2024	Understanding Emergent Abilities of Language Models from the Loss Perspective	Full text	v3 / Paper / preprint	06
2025	Comment on The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity	Full text	v2 / Paper / preprint	05
2025	DAPO: An Open-Source LLM Reinforcement Learning System at Scale	Selected full-text sections	v2 / Paper / preprint	04
2025	DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning	Selected full-text sections	v1 / Report	04
2025	DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning	Selected full-text sections	v1 / Paper / preprint	04

Year	Work and primary source	Reading depth	Version / type	Chapters
2025	Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?	Selected full-text sections	v5 / Paper / preprint	04
2025	Group Sequence Policy Optimization	Selected full-text sections	v2 / Paper / preprint	04
2025	How Do Large Language Monkeys Get Their Power (Laws)?	Selected full-text sections	v1 / Paper / preprint	05
2025	Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps	Full text	v1 / Paper / preprint	07
2025	Inference-time Scaling of Diffusion Models through Classical Search	Abstract / metadata	v2 / Paper / preprint	07
2025	Learning to Reason under Off-Policy Guidance	Selected full-text sections	v5 / Paper / preprint	04
2025	Open-Reasoner-Zero: An Open Source Approach to Scaling Up Reinforcement Learning on the Base Model	Selected full-text sections	See metadata / Paper / preprint	04
2025	Process Reward Models That Think	Selected full-text sections	See metadata / Paper / preprint	04
2025	Scaling Behaviors of LLM Reinforcement Learning Post-Training: An Empirical Study in Mathematical Reasoning	Selected full-text sections	v1 / Paper / preprint	04
2025	Scaling Laws for Optimal Data Mixtures	Selected full-text sections	v2 / Paper / preprint	02
2025	Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach	Selected full-text sections	v2 / Paper / preprint	05
2025	SimpleRL-Zoo: Investigating and Taming Zero Reinforcement Learning for Open Base Models in the Wild	Selected full-text sections	v3 / Paper / preprint	04
2025	SmolLM2: When Smol Goes Big -- Data-Centric Training of a Small Language Model	Selected full-text sections	v1 / Report	02

Year	Work and primary source	Reading depth	Version / type	Chapters
2025	Test-Time Scaling in Reasoning Models Is Not Effective for Knowledge-Intensive Tasks Yet	Selected full-text sections	v1 / Paper / preprint	05
2025	The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity	Selected full-text sections	v3 / Paper / preprint	05
2025	The Lessons of Developing Process Reward Models in Mathematical Reasoning	Selected full-text sections	v2 / Paper / preprint	04
2025	ThinkPrune: Pruning Long Chain-of-Thought of LLMs via Reinforcement Learning	Selected full-text sections	v1 / Paper / preprint	05
2025	Towards Thinking-Optimal Scaling of Test-Time Compute for LLM Reasoning	Selected full-text sections	v1 / Paper / preprint	05
2025	Understanding R1-Zero-Like Training: A Critical Perspective	Selected full-text sections	v2 / Paper / preprint	04
2025	VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks	Selected full-text sections	v3 / Paper / preprint	04
2025	rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking	Selected full-text sections	v1 / Paper / preprint	05
2025	s1: Simple test-time scaling	Selected full-text sections	v1 / Paper / preprint	05
2026	A study of sequence weighting at scale	Full text	Web snapshot / Technical article	02
2026	AXIS: A Growable Community-Driven Data Engine for Scalable Robot Manipulation	Abstract / metadata	v1 / Paper / preprint	07
2026	Adaptive Test-Time Compute Allocation for Reasoning LLMs via Constrained Policy Optimization	Selected full-text sections	v1 / Paper / preprint	05
2026	Ambiguous Fits: The Rashomon Effect and Challenges in Scaling Law Estimation	Selected full-text sections	v1 / Paper / preprint	01

Year	Work and primary source	Reading depth	Version / type	Chapters
2026	Capability Scaling-Down Laws for LLM Compression	Selected full-text sections	v1 / Paper / preprint	03
2026	Curriculum Reinforcement Learning Can Incentivize Reasoning Capacity in LLMs Beyond the Base Model	Selected full-text sections	v1 / Paper / preprint	04
2026	Emergent Inverse-Depth Scaling From Nonlinearity In Attention	Selected full-text sections	v1 / Paper / preprint	06
2026	FrogNano: Training a 4B Coding Agent via Online Task Synthesis	Selected full-text sections	v5 / Paper / preprint	Catalog only
2026	Frontier Pre-training at Microsoft AI	Official report sections	Web snapshot / Report	01
2026	Harness Scaling for Generalizable Agentic Reinforcement Learning	Selected full-text sections	v2 / Paper / preprint	05
2026	Helix 2.5: Zero-Shot 30-Home Generalization	Official report sections	Web snapshot / Report	07
2026	How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents	Selected full-text sections	v2 / Paper / preprint	03
2026	How Much Can Language Models Gain from Test-Time Computation?	Selected full-text sections	v2 / Paper / preprint	05
2026	How Much Is an AI Token Worth? Scaling Laws for Wild AI-Generated Web Text	Selected full-text sections	v1 / Paper / preprint	02
2026	Hyperparameter Scaling Laws Across MoE Sparsity	Selected full-text sections	v1 / Paper / preprint	03
2026	Inference-Time Scaling in Diffusion Models through Iterative Partial Refinement	Abstract / metadata	v1 / Paper / preprint	07
2026	IronMind: Scaling Humanoid Dexterous Manipulation via Camera-Space Ego-Centric Pretraining	Selected full-text sections	v1 / Paper / preprint	Catalog only
2026	Li²SAO: Lifting Light Single-Rollout Asynchronous RL Optimization via Group-Relative Actor-Critic Bootstrapping	Selected full-text sections	v1 / Paper / preprint	04

Year	Work and primary source	Reading depth	Version / type	Chapters
2026	Mixed Precision Scaling Laws for Quantized Pretraining	Selected full-text sections	v1 / Paper / preprint	03
2026	More Than Seeing: Vision as Scalable Anchors in Native Multimodal Pre-Training	Selected full-text sections	v1 / Paper / preprint	Catalog only
2026	On the Limits of Scale: Inverse Scaling in Vision-Language Models	Selected full-text sections	v1 / Paper / preprint	07
2026	Paired Multimodal Scaling Laws	Selected full-text sections	v1 / Paper / preprint	07
2026	Planned Test-Time Scaling with Coordinated Reasoning Paths	Selected full-text sections	v1 / Paper / preprint	05
2026	Quantifying construct validity in large language model evaluations	Selected full-text sections	v1 / Paper / preprint	06
2026	Rethinking the Evaluation of Harness Evolution for Agents	Selected full-text sections	v4 / Paper / preprint	05
2026	Revisiting scaling laws for reward optimization	Selected full-text sections	v1 / Paper / preprint	04
2026	Sample Count Is Not Enough: Candidate-Generation Strategy Shapes the Energy and Performance of LLM Test-Time Scaling	Selected full-text sections	v2 / Paper / preprint	05
2026	Scaling Laws for Generative Reward Models	Selected full-text sections	v1 / Paper / preprint	04
2026	Scaling Laws for Multimodal Data Mixtures	Selected full-text sections	v1 / Paper / preprint	07
2026	Scaling Laws for Tool-Use Agents via Controllable Environment Synthesis	Selected full-text sections	v1 / Paper / preprint	05
2026	Scaling Misuse Detection with Self-Assessment	Selected full-text sections	Web snapshot / Technical article	Catalog only

Year	Work and primary source	Reading depth	Version / type	Chapters
2026	Scaling Multilingual Mixture-of-Experts: How Language Coverage Changes Capacity and Compute Allocation	Selected full-text sections	v3 / Paper / preprint	03
2026	Scaling Repeated-Data Pretraining: Batch Size, Regularization, and GPU-Time Tradeoffs	Selected full-text sections	v1 / Paper / preprint	02
2026	Scaling Vision-Language Models Is Not Enough to Mitigate Bias	Selected full-text sections	v1 / Paper / preprint	07
2026	Test-Time Scaling in Reasoning LLMs: Inference Regimes, Evaluation, and Reproducibility	Selected full-text sections	v2 / Paper / preprint	05
2026	Thinking Long, but Short: Stable Sequential Test-Time Scaling for Large Reasoning Models	Selected full-text sections	v1 / Paper / preprint	05
2026	What Is a Repeated Token Worth? The Scaling Geometry of Multi-Epoch Pretraining	Selected full-text sections	v1 / Paper / preprint	02
2026	When Does Test-Time Search Help World Action Models? An Empirical Audit of Candidate Selection and Simulator Integrity	Selected full-text sections	v1 / Paper / preprint	07
2026	Your LR Schedule May Not Be Principled: Effective Learning Rate Scheduling in Weight-Normalized Training	Selected full-text sections	v1 / Paper / preprint	01